Cooperative Robotics

v4.4 17-03-2023

Enrico Simetti
DIBRIS - University of Genova
ISME - Interuniversity Research Center on Integrated Systems for Marine Environment
Via Opera Pia 13, Genova, Italy
enrico.simetti@unige.it

Contents

Contents ii

1 Introduction 1
1.1 Centralized versus Decentralized Control Architectures . . . 1

2 Fundamentals 3
2.1 Geometric Fundamentals . . . 3
2.1.1 World Coordinate System and Related Objects . . . 3
2.1.2 Other Coordinate Systems . . . 4
2.1.3 Transformation Matrices . . . 6
2.1.4 Free from Coordinate Notation . . . 7
2.1.5 Universes of Frames and Reference Systems . . . 7
2.1.6 Three-parameter Representation of Orientation Matrices . . . 8
2.2 Kinematics Fundamentals . . . 10
2.2.1 Time Derivative of Orientation and Transformation Matrices . . . 10
2.2.2 Composition of Angular Velocity Vectors . . . 11
2.2.3 Time Derivative of Vectors . . . 12
2.2.4 Time Derivative of Constant Module Vectors . . . 13
2.2.5 Time Derivative of Generic Vectors . . . 13
2.2.6 Time Derivative of Rotation Vectors . . . 14
2.2.7 Time Derivative of Points . . . 16
2.2.8 Time Derivative of Distance Vectors . . . 17

3 Single Agent Control: a Task Priority Approach 19
3.1 Case Study: Underwater Floating Manipulator Systems . . . 19
3.1.1 Notation . . . 19
3.1.2 Degrees of Freedom of an UVMS . . . 19
3.1.3 Reference Frames . . . 20
3.1.4 Configuration of the System . . . 20
3.1.5 Sensory Equipment of an UVMS . . . 21
3.2 General Control Framework Architecture . . . 22
3.3 Control Objectives . . . 23
3.3.1 Objective Definition . . . 23
3.3.2 Categories of Objectives . . . 24
3.3.3 Examples of Control Objectives for an UVMS . . . 24
3.4 Control Tasks . . . 27
3.4.1 Closed Loop Feedback Reference Rates . . . 27
3.4.2 Tasks and Their Relationship with Control Variables . . . 28
3.4.3 Reactive and non-reactive Control Task Definitions . . . 28
3.5 Actions . . . 29
3.6 Task Activation Functions . . . 29
3.7 Priorities between Control Tasks . . . 30
3.8 Task Priority Inverse Kinematics . . . 31
3.8.1 Preliminaries on Least Squares Problems . . . 31
3.8.2 Classic Task Priority Algorithm . . . 33
3.8.3 Modern Task Priority Algorithms . . . 34

3.8.4 Building a Mission through Action Sequencing . . . 34
3.8.5 Remarks on Conflicting Objectives . . . 35
3.8.6 Control in Presence of Vehicle Underactuations . . . 36
3.8.7 Arm-Vehicle Coordination Scheme . . . 37
3.8.8 Arm-Vehicle Multi-rate Control . . . 38
3.9 Multi Arm UVMS Control . . . 38
3.10 Exploiting the Task Priority Approach: Actions and Possible Applications . . . 40
3.10.1 Safe Waypoint Navigation . . . 40
3.10.2 Assisted Teleoperation of UVMS Degrees of Freedom . . . 41
3.10.3 Grasping . . . 42
3.10.4 Assisted End-Effector Teleoperation . . . 42
3.10.5 Inspection . . . 43
3.10.6 Landing . . . 46
3.10.7 Case Study Examples . . . 46

4 Cooperative Control of Multiple Agents 49
4.1 Cooperative Transportation and Manipulation . . . 49
4.1.1 Introduction . . . 49
4.1.2 Coordination Policy . . . 52
4.1.3 Experimental Results . . . 54

A Pseudo Inverses and SVD 59
A.1 Moore-Penrose Pseudo Inverse . . . 59
A.2 Singular Value Decomposition . . . 59
A.2.1 Geometric Interpretation of SVD . . . 60
A.2.2 Pseudo Inversion through SVD . . . 61
A.2.3 Singular Value Oriented Regularization . . . 61

B Inverse Kinematics: a Geometric Interpretation 63
B.1 Full Row Rank Jacobian case . . . 63
B.2 Full Column Rank Jacobian Case . . . 64

C Task Priority Inverse Kinematics: Detailed Developments 67
C.1 Solving the First Priority Level . . . 67
C.1.1 Task Oriented Regularization . . . 67
C.2 Extension to Multiple Priority Levels . . . 69
C.2.1 Removing the Discontinuities in the Prioritized Control . . . 69
C.2.2 Minimization of the Control Vector as the Final Task . . . 70
C.2.3 Unifying Formula for the Pseudo Inverse . . . 70
C.2.4 Extension to Any Number of Priority Levels . . . 70
C.3 Residual Orthogonality Properties of the Projector Matrix . . . 70

Chapter 1

Introduction

The employment of cooperating robots (or collaborative or coordinating) for executing complex activities is receiving a great interest by the scientific and industrial communities. Robots can cooperate to map an area, to execute underwater surveys, to assemble a piece, or to transport an object in a new location just to name but a few examples.

In a dual-arm work cell, the two arms needs to be coordinated to assemble a piece. In a mapping scenario, two vehicles might coordinate themselves to achieve the mapping in a more efficient way, rather than proceeding without coordination. Arms and vehicles constitute the *basic robotic structures*, or basic units, which could be employed either as individual agents or as coordinated operating parts within more complex organizations, characterized by enhanced overall operational capabilities. Therefore, a mobile manipulator, i.e. a vehicle endowed with an arm, is a more powerful robotic agent, as it benefits from the capabilities of each of its composing subsystems. Two or more mobile manipulators can then cooperate to transport a big object toward a new location.

While each basic robotic unit is accomplishing his main goal, it also has to take care of many other objectives. While mapping an area, a vehicle need to avoid possible obstacles, whereas a robotic arm doing manipulation on some object needs to comply with the constraints dictated by their joint limits.

Therefore, to simplify the cooperation of multiple robotic units, each basic unit (arm or vehicle) should be supported by a unified functional and algorithmic architecture, satisfying the following requirements: a) automatically guaranteeing all safety and operational-enabling unit-specific conditions; b) best-accomplishing to externally provided commands; c) operating without structural changes also when assembled with other ones.

For the aforementioned reasons, these notes will first focus on the development of a task priority based approach to the control of each basic unit.

1.1 Centralized versus Decentralized Control Architectures

An *Individual Robotic Agent* (for short an agent) is any robotic structure having its motion governed by an associated, suitably designed, centralized agent control system (agent-controller). The Agent-controller exploits, without any restriction: a) all information relevant to the whole geometric and kinematic model of the underlying robotic structure; b) the real-time measurements of any needed variable from the inside of the robotic structure; c) the real-time acquisition of any additional environmental information provided by endowed sensors; and the operational commands to which the agent has to obey at best.

A *Multi Robotic Agent* (multi-agent) is instead intended as the aggregation, or collection, of different agents, which results when multiple agents are employed for executing, cooperative activities, via the aid of a suitably implemented *coordination policy*, established among their individual agent-controllers, supported by a real-time information exchanges with them.

In the multi-agent case, the information exchange should be a proper *subset* of the complete one that would correspond to a full-exchange of the individual information of each agent. If this were not the case, we would re-enter in the centralized case, as the coordination policy would have access to all information.

Let us consider a paradigmatic case, depicted in Fig. 1.1 . A mobile manipulator has its own agent-controller. A second arm has a standalone single-agent controller. If the mobile manipulator is

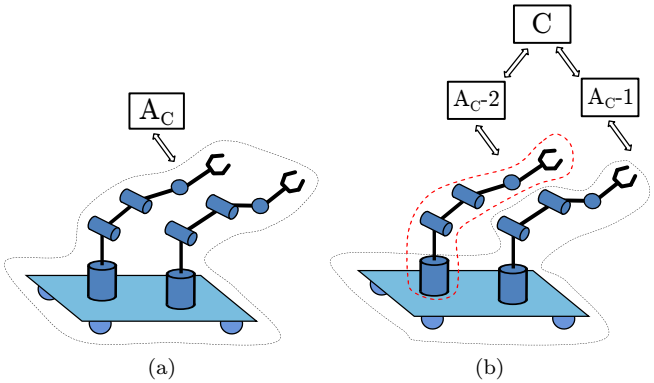


Figure 1.1: (a) single-agent controller (b) multi-agent controller

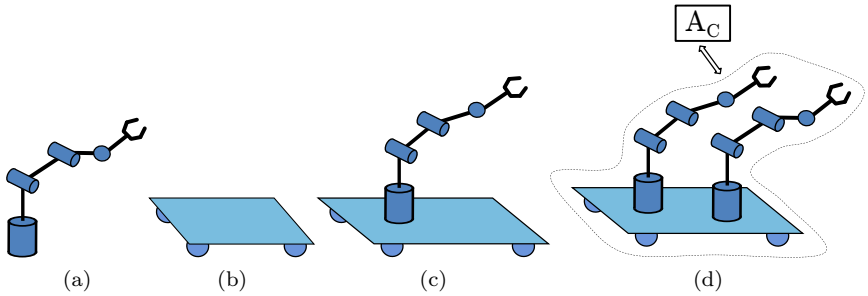


Figure 1.2: (a) arm (b) vehicle (c) mobile manipulator (d) dual-arm mobile manipulator

upgraded to become a dual-arm mobile manipulator, should the two original single-agent controller be coordinated? or should a new single-agent controller be devised? Clearly, the multi-agent controller cannot have the same performances as the centralized one.

Looking at Fig. 1.2 it is clear how the dual arm mobile manipulator can be seen as a general case, from which all the others can descent. In these notes we will develop a control framework, based on task-priority approach, which is suited to handle the general case and all its downscaled ones. To this aim, chapter 2 will first recall the fundamentals of geometry and kinematics of robots. Then, chapter 3 will introduce the task priority approach applied to a single agent. Finally, chapter 4 will extend the approach to cooperative agents.

Chapter 2

Fundamentals

2.1 Geometric Fundamentals

The present chapter will review all basic geometric concepts and related algorithmic tools needed for later managing geometric problems regarding any type of robotic structure.

For sure, many of the concepts and formalisms presented in the initial sections are well known to the reader, possibly from previous courses. Even in this case, the reading of the chapter is recommended, at least for getting acquainted with the used notations.

2.1.1 World Coordinate System and Related Objects

A so-called *world coordinate system*, otherwise termed as the *world frame* $\langle 0 \rangle$, is represented by the *ordered triple* of *orthogonal* and *oriented axes* $x - y - z$, corresponding to a so-called right frame (i.e. with the orientation of the z axis obtained via the application of the "right-hand-rule" from x to y) that *ideal observers* have *agreed* to use for parameterizing the surrounding space.

The concept of "ideal observer" corresponds to an ideal entity capable of monitoring the evolution of the involved geometric or physical phenomena in whatever needed details, and capable of recording them with no storage limits, as well as capable of instantaneously communicating with other ideal observers via infinite capacity and speed channels. In the practice, instead, "non-ideal (real) observers" typically correspond to hardware/software subsystem organizations that, possibly distributed within the environment, perform measurements, data acquisition, data processing, data logging, data transmission, etc., but with obvious practical limitations with respect to the ideal ones.

With that in mind, as indicated in Fig. **??**, the position of a point P (we shall always use capital letters for indicating points in space) with respect to the agreed frame $\langle 0 \rangle$ is termed as *point P projected on frame* $\langle 0 \rangle$ and is denoted as ${}^0P$, and is represented by the algebraic-vector of its ordered components, respectively on the $x - y - z$ axes of $\langle 0 \rangle$, that is:

$$ {}^0P \triangleq \begin{bmatrix} x_{P,0} \\ y_{P,0} \\ z_{P,0} \end{bmatrix}, \tag{2.1} $$

where the indexes indicating the point P and the frame $\langle 0 \rangle$, in correspondence of each component of ${}^0P$ itself, only serve for recalling how such components are defined, and will be omitted in the rest of the notes, whenever the context is clearly understood. Moreover, let us trivially note that the position ${}^0O_0$ of the origin of frame $\langle 0 \rangle$, with respect to frame $\langle 0 \rangle$ itself, corresponds to the null position (i.e. the null algebraic vector).

Within such a parameterized space, other than points, we also have the so-called *applied projected vectors* on frame $\langle 0 \rangle$, simply defined and denoted as

$$ {}^0\left[P - Q\right] \triangleq \left[{}^0P - {}^0Q\right], \tag{2.2} $$

corresponding to the algebraic vector of the components on $\langle 0 \rangle$ of the oriented segment that starting from point Q reaches point P.

Accordingly with its definition, an applied projected vector cannot be parallel shifted. Indeed, if we were instead parallel shifting it into another ${}^0\left[P' - Q'\right]$, then we should consider it as a *different*

vector. Finally, let us note how a projected point ${}^0P$ can be seen as an applied projected vector on frame $\langle 0 \rangle$, with starting point the origin of frame $\langle 0 \rangle$:

$$ {}^0P = {}^0 \left[P - O_0 \right]. \tag{2.3} $$

An important extension is the concept of *projected vector* on frame $\langle 0 \rangle$ denoted as

$$ {}^0 \left(P - Q \right) \triangleq \left({}^0P - {}^0Q \right), \tag{2.4} $$

which now includes the *entire class* of applied projected vectors on frame $\langle 0 \rangle$, which are each one a parallel shift of a given seminal one ${}^0 \left[P - Q \right]$. The use of round brackets, in lieu of squared ones, underlines this difference. Moreover, let us introduce a more compact vector notation:

$$ {}^0\boldsymbol{v}_{P,Q} \triangleq \left({}^0P - {}^0Q \right), \tag{2.5} $$

where the indication of the seminal points is maintained. Whenever the context is clear, the projected vector will be indicated simply as ${}^0\boldsymbol{v}$. Moreover, accordingly with the so-called *Grassman rule*, note that a position point ${}^0P$ can also be represented as

$$ {}^0P = {}^0Q + {}^0\boldsymbol{v}, \tag{2.6} $$

i.e. as a position point translated toward the desired position, accordingly with the translation provided by a given projected vector.

Finally, within the space parameterized by $\langle 0 \rangle$, a special ordered triple of projected vectors can be further introduced as

$$ \begin{aligned} {}^0\boldsymbol{i}_0 &\triangleq \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^\top \\ {}^0\boldsymbol{j}_0 &\triangleq \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}^\top \\ {}^0\boldsymbol{k}_0 &\triangleq \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^\top \end{aligned} \tag{2.7} $$

corresponding to the well known so-called projected unit vectors of the x-y-z axes of frame $\langle 0 \rangle$, projected on frame $\langle 0 \rangle$ itself.

2.1.2 Other Coordinate Systems

Once the ideal observers have agreed about the world coordinate system $\langle 0 \rangle$, they can also agree on the use of other coordinated systems, of which at least one of them is positioned with respect to the originally agreed frame $\langle 0 \rangle$; and each one of the others still positioned with respect to $\langle 0 \rangle$ (directly or indirectly through other frames).

Fig. **??** is an example of a graph indicating how a possible positioning of different frames $\langle 1 \rangle$, $\langle 2 \rangle$, $\langle 3 \rangle$ has been progressively performed by starting from the seminal one $\langle 0 \rangle$.

Within such a graph, the tips of the arrows joining one frame with another are used for indicating a corresponding positioned frame, otherwise termed as a *referring frame*, while the tails indicate the corresponding *referred frame*, i.e. the one with respect to which the frame at the tip has been positioned. By proceeding in this way a so-called tree of frames (a connected graph) can consequently be constructed, as agreed by all the observers, termed as the *agreed universe of frames*. Let us note how, when positioning a referring frame $\langle b \rangle$ with respect to a referred one $\langle a \rangle$, then the point of view of an observer fixed with the referred frame $\langle a \rangle$ (i.e. just looking to where actually is located with respect to him) is de-facto implicitly adopted.

To describe the positioning of a generic frame $\langle b \rangle$ with respect to another one $\langle a \rangle$, the following set of information data is needed:

$$ \left\{ {}^aO_b; \left({}^a\boldsymbol{i}_b, {}^a\boldsymbol{j}_b, {}^a\boldsymbol{k}_b \right) \right\}, \tag{2.8} $$

composed by the where the origin O_b of frame $\langle b \rangle$ is located w.r.t. $\langle a \rangle$, i.e. the projection of O_b on the referred frame $\langle a \rangle$, indicated as ${}^aO_b$; and the ordered triple of unit vectors of $\langle b \rangle$, each one represented with components on frame $\langle a \rangle$. The second set of information serves for establishing which is the *orientation* of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$.

2.1.2.1 Orientation Matrices

Let us represent the orientation information described above in a 3×3 matrix, termed *orientation matrix* ${}^a_bR$, where the above defined vectors are orderly inserted as:

$$ {}^a_b\boldsymbol{R} \triangleq \begin{bmatrix} {}^a\boldsymbol{i}_b & {}^a\boldsymbol{j}_b & {}^a\boldsymbol{k}_b \end{bmatrix}. \tag{2.9} $$

Such a matrix turns out to be of orthonormal type, since its columns are orthogonal unitary algebraic vectors, hence it satisfies

$$ \det({}^a_b\boldsymbol{R}) = 1 \tag{2.10} $$

Furthermore, still due to the orthonormal property, the following equations hold

$$ {}^a_b\boldsymbol{R}^a_b\boldsymbol{R}^\top = \boldsymbol{I}_{3\times 3} \tag{2.11} $$

$$ {}^a_b\boldsymbol{R}^\top {}^a_b\boldsymbol{R} = \boldsymbol{I}_{3\times 3} \tag{2.12} $$

implying that

$$ {}^a_b\boldsymbol{R}^\top = {}^a_b\boldsymbol{R}^{-1}. \tag{2.13} $$

Let us now consider two frames $\langle a \rangle$, $\langle b \rangle$, for which it is known the orientation matrix ${}^a_b\boldsymbol{R}$ of the second one with respect to the first one. Further consider a generic geometric vector $\boldsymbol{v}$, whose projection ${}^b\boldsymbol{v}$ is it also known. Let us now perform a change of projection coordinate for vector $\boldsymbol{v}$, i.e. let us devise a formula for deducing its projection ${}^a\boldsymbol{v}$ on frame $\langle a \rangle$. To this aim, let us formerly *express* the geometric vector $\boldsymbol{v}$ as the linear combination of the unit vectors of frame $\langle b \rangle$:

$$ \boldsymbol{v} = \boldsymbol{i}_b x_b + \boldsymbol{j}_b y_b + \boldsymbol{k}_b z_b. \tag{2.14} $$

After projecting on frame $\langle a \rangle$ one obtains

$$ {}^a\boldsymbol{v} = {}^a\boldsymbol{i}_b x_b + {}^a\boldsymbol{j}_b y_b + {}^a\boldsymbol{k}_b z_b, \tag{2.15} $$

which rearranged in matrix form becomes

$$ {}^a\boldsymbol{v} = \begin{bmatrix} {}^a\boldsymbol{i}_b & {}^a\boldsymbol{j}_b & {}^a\boldsymbol{k}_b \end{bmatrix} \begin{bmatrix} x_b \\ y_b \\ z_b \end{bmatrix} = {}^a_b\boldsymbol{R}^b\boldsymbol{v}, \tag{2.16} $$

which represents the formula for the change of coordinates of a projected vector.

Now, by inverting the above we get

$$ {}^b\boldsymbol{v} = {}^a_b\boldsymbol{R}^{-1\,a}\boldsymbol{v} = {}^a_b\boldsymbol{R}^\top {}^a\boldsymbol{v}. \tag{2.17} $$

However, if the change of coordinate problem had been casted with the roles of frames $\langle a \rangle$ and $\langle b \rangle$ swapped, we would have obtained

$$ {}^b\boldsymbol{v} = {}^b_a\boldsymbol{R}^a\boldsymbol{v}, \tag{2.18} $$

then, due to the arbitrariness of $\boldsymbol{v}$, it follows that we must necessarily have that

$$ {}^a_b\boldsymbol{R}^\top = {}^b_a\boldsymbol{R}, \tag{2.19} $$

which represents the simple formula for knowing the orientation of a first frame with respect to a second one, once the orientation of the second one is known with respect to the first. In particular, the above equation implies the following inner structure of ${}^a_b\boldsymbol{R}$

$$ {}^a_b\boldsymbol{R} = \begin{bmatrix} {}^a\boldsymbol{i}_b & {}^a\boldsymbol{j}_b & {}^a\boldsymbol{k}_b \end{bmatrix} = \begin{bmatrix} {}^b\boldsymbol{i}_a^\top \\ {}^b\boldsymbol{j}_a^\top \\ {}^b\boldsymbol{k}_a^\top \end{bmatrix}, \tag{2.20} $$

which provides a very easy mean for immediately looking at the mutual orientations between two given frames.

2.1.2.2 Evaluation of Orientation Matrices within Trees of Frames

Let us consider a chain of $n+1$ frames, as depicted in Fig. **??**, for which we only evidence their cascaded orientation matrices ${}^{i-1}_{i}\boldsymbol{R}, i = 1, \ldots, n$. We want to deduce the orientation matrix ${}^{0}_{n}\boldsymbol{R}$ from the last frame with respect to the first one. To this aim, let us extend the problem considered in the previous section, and let us assume to know the projection ${}^{n}\boldsymbol{v}$ of a generic vector $\boldsymbol{v}$ on the last frame, and then let us evaluate its projection ${}^{0}\boldsymbol{v}$ on the first one. Repeating backwardly the change of coordinate formula developed in the previous section, one obtains:

$$ {}^{0}\boldsymbol{v} = \left({}^{0}_{1}\boldsymbol{R}{}^{1}_{2}\boldsymbol{R} \cdots {}^{i-1}_{i}\boldsymbol{R} \cdots {}^{n-1}_{n}\boldsymbol{R}\right) {}^{n}\boldsymbol{v}, \tag{2.21} $$

that, accordingly with the arbitrariness of $\boldsymbol{v}$, allows stating that

$$ {}^{0}_{n}\boldsymbol{R} = {}^{0}_{1}\boldsymbol{R}{}^{1}_{2}\boldsymbol{R} \cdots {}^{i-1}_{i}\boldsymbol{R} \cdots {}^{n-1}_{n}\boldsymbol{R}. \tag{2.22} $$

2.1.3 Transformation Matrices

In the previous sections we have focused on the representation of the mutual orientation between two frames, neglecting the position information represented by ${}^{a}O_{b}$. Let us now introduce a compact notation that takes into account both the positioning information and the orientation one in a 4×4 matrix

$$ {}^{a}_{b}\boldsymbol{T} \triangleq \begin{bmatrix} {}^{a}_{b}\boldsymbol{R} & {}^{a}O_{b} \\ \boldsymbol{0}_{3\times 1} & 1 \end{bmatrix}, \tag{2.23} $$

which is termed *Transformation Matrix* of frame $\langle b \rangle$ with respect to frame $\langle a \rangle$. The reasons why the above matrix is constructed to be 4×4, with the first three elements of the fourth row always zero, and the last one always unitary, will be clarified in the sequel.

Let us consider a frame $\langle b \rangle$, positioned with respect to $\langle a \rangle$ within an agreed universe, via its transformation matrix ${}^{a}_{b}\boldsymbol{T}$. Let us further assume to know the position ${}^{b}P$ of a generic point P with respect to frame $\langle b \rangle$ and let us compute its position ${}^{a}P$ with respect to frame $\langle a \rangle$.

The above stated problem can be trivially solved, by observing that it will necessarily be

$$ {}^{a}P = {}^{a}O_{b} + {}^{a}_{b}\boldsymbol{R}{}^{b}P \tag{2.24} $$

as changing the coordinates of projected point from one frame to another one always requires its reference origin to be changed accordingly, as Fig. **??** shows.

Let us now represent projected points via the so called *homogeneous coordinates*, i.e. via the following 4×1 algebraic vector

$$ {}^{b}\bar{P} = \begin{bmatrix} {}^{b}P \\ 1 \end{bmatrix}, \tag{2.25} $$

where the scalar value 1 is always added as the fourth component. Then, exploiting this representation, the change of coordinates formula for a point can be more compactly rewritten as

$$ {}^{b}\bar{P} = {}^{a}_{b}\boldsymbol{T}{}^{a}\bar{P}. \tag{2.26} $$

Following similar steps as done for orientation matrices, it is possible to show that

$$ {}^{a}_{b}\boldsymbol{T}^{-1} = {}^{b}_{a}\boldsymbol{T}. \tag{2.27} $$

Moreover, the inversion of a transformation matrix can be found as:

$$ {}^{a}_{b}\boldsymbol{T}^{-1} = \begin{bmatrix} {}^{a}_{b}\boldsymbol{R}^{\top} & -{}^{a}_{b}\boldsymbol{R}^{\top}{}^{a}O_{b} \\ \boldsymbol{0}_{3\times 1} & 1 \end{bmatrix}. \tag{2.28} $$

2.1.3.1 Evaluation of Transformation Matrices within Trees of Frames

Let us again consider a chain of $n+1$ frames, as depicted in Fig. **??**. Following the same idea exploited for orientation matrices, consider as known the position of a point P with respect to the last frame, i.e. ${}^{n}\bar{P}$. Then, its position w.r.t. the first frame $\langle 0 \rangle$ can be found by repeatedly using the change of coordinates formula

$$ {}^{0}\bar{P} = \left({}^{0}_{1}\boldsymbol{T}{}^{1}_{2}\boldsymbol{T} \cdots {}^{i-1}_{i}\boldsymbol{T} \cdots {}^{n-1}_{n}\boldsymbol{T}\right) {}^{n}\bar{P}, \tag{2.29} $$

that, accordingly with the arbitrariness of P, allows stating that

$$ {}^{0}_{n}\boldsymbol{T} = {}^{0}_{1}\boldsymbol{T}{}^{1}_{2}\boldsymbol{T} \cdots {}^{i-1}_{i}\boldsymbol{T} \cdots {}^{n-1}_{n}\boldsymbol{T}. \tag{2.30} $$

2.1.4 Free from Coordinate Notation

In the previous sections, we have shown how to change the projection of points, vectors and applied vectors to a different frame within an agreed universe of frames. It was shown how a frame can be described via different equivalent positioning with respect to the other frames. This makes the original agreed world frame $\langle 0 \rangle$ actually loosing its character of privilege, which was only apparently assigned to it.

Therefore, let us introduce the so-called *free from coordinate* notation, for indicating points, vectors and applied vectors without mentioning any projection frame since, in force of existing transformation equivalence, it can be any one among the agreed ones.

More precisely, in terms of free from coordinate notation, points, vectors, and applied vectors, will be more simply denoted as

- P - Capital letters for points;
- $\boldsymbol{v} = (P - Q)$ - bold lower case letters, or round parentheses difference of points, for vectors;
- $[P - Q]$ - square bracketed difference of points, or a vector-point couple for applied vectors;

where the introduced notation identifies, for each item, the entire class of the associated projections, which transform each other via coordinate changes.

It must be explicitly noted how, with the use of such a notation, the corresponding items result into *symbolic* items, each one of them generally termed as a *geometric-vector*, or a *geometric-point*. Instead, their projected equivalent forms are *algebraic-vectors* (i.e. 3×1 matrices, or 4×1 in case of homogeneous coordinates).

Finally note how even the notation for denoting frames, say for instance a generic frame $\langle a \rangle$, can be it also interpreted as a free-from-coordinates notation, where the explicit indication of its referring frame is avoided, since it could actually be any one of the other frames within the agreed universe.

2.1.5 Universes of Frames and Reference Systems

The present subsection is more of conceptual nature than technical. It will fix some further concepts (mainly the concept of Reference System) that will permeate the contents of all the successive chapters.

We can start the discussion by noticing how the agreed universe of frames (i.e. coordinate systems) as well as the Euclidean space (i.e. the whole set of points parameterized by each frame of the agreed universe) has been de-facto presented as if they were "static entities" not subjected to any change after their establishment. Of course, this is definitely not true, since if we now explicitly consider the time, and also assume the frames of a universe maintaining their "individuality along time", then we certainly have situations where at least some of the universe frames are changing their transformation matrices along time, with respect to at least one of the other frames of the universe itself.

So, if we have a frame $\langle \alpha \rangle$ of the universe exhibiting such a behavior, at a first glance we might consequently say that is therefore "moving" inside the universe, with respect to each one of the frames with respect to which it is changing its transformation matrices.

However, if we conversely consider one or more of such other frames (i.e. those with respect to which has been at a first glance considered "moving") we could say that each one of them are moving with respect to $\langle \alpha \rangle$. This in total accordance with the trivial well known fact that the concept of motion is a *relative* one.

As the consequence of the motion relativity, when at least a frame of a universe changes in time its transformation matrix with respect to other frames, we can only say that it is the configuration of the entire universe which is actually changing. The interpretation for such changes is let to the "adopted point of view", i.e. the frame from which such changes are observed.

For a given frame $\langle \alpha \rangle$, we can consequently adopt the free-from-coordinates notation $\langle \alpha \rangle(t)$ for representing it within the time varying context induced by its supporting entire class of now time varying transformation matrices ${}^{i}_{\alpha}\boldsymbol{T}$, $i = 1, \ldots, n$, with respect to all other frames of the universe. If we also consider points characterized by their own "individuality" along time, we can use the free-from-coordinate notation $P(t)$ to denote them. A point that does not change its position with respect to a frame $\langle \alpha \rangle(t)$ is a pointed "attached" to $\langle \alpha \rangle(t)$; nevertheless, it can still be a point moving within the universe, if and only if $\langle \alpha \rangle(t)$ is moving in the previously specified relative sense.

Motions within the universe are also allowed for "individual vectors" (applied or not), as the results of the motion assigned to their identifying points.

At this point, let us consider the set of individual points which are not varying their position with respect to $\langle\alpha\rangle(t)$ itself, and let us term such a rigid space as the *Reference System* established by frame $\langle\alpha\rangle(t)$, to be for instance denote as $\Sigma_\alpha(t)$.

Let us note that any other frame $\langle b\rangle(t)$ exhibiting a constant position and orientation with respect to $\langle\alpha\rangle(t)$, shares with it the same rigid space. Hence, a reference system can also be intended as established by the entire class of additional frames that maintain constant position and orientation with respect to the given one.

Intrinsic to the definition of reference system (or rigid space) is the possibility of enlarging the original universe of frames with the infinite continuous sets of frames belonging, each one of them, to the reference systems of the original ones. This possibility makes the universe becoming continuous also with respect to frames, as it was already as regard points, vectors and applied vectors. Furthermore, the originally agreed *discrete set* of frames actually lose its character of privilege. Indeed, any collection of frames within the different reference systems could be used for constructing (or in a sense reconstructing) the same universe.

Finally, let us explicitly note how there exists a general tendency to consider the three distinct terms "frames" (i.e. "coordinate systems"), "reference systems/rigid spaces" and "ideal observers", as synonymous one of the other, notwithstanding the fact that they are slightly differently originated concepts. For instance, an observer should be more properly assigned to a reference system rather than to a specific frame, i.e. it should be delocalized within the assigned reference system. However, its attachment to a specific frame is in any case accepted, within the obvious understanding of considering it equivalent to any other attached to a frame of the same reference system (i.e. they all are de-facto the same observer).

2.1.6 Three-parameter Representation of Orientation Matrices

For the subsequent developments, it is now important to introduce other more compact ways of representing an orientation matrix ${}^a_b\boldsymbol{R}$ between two generic frames $\langle a\rangle$, $\langle b\rangle$.

To this aim, let us recall that orientation matrices are orthonormal, therefore their 9 elements must satisfy the following 6 constraints:

- Unimodularity. Its three columns (rows) are unit vectors, that is

$$\begin{aligned} {}^a\boldsymbol{i}_b^{\top\,a}\boldsymbol{i}_b &= 1, \\ {}^a\boldsymbol{j}_b^{\top\,a}\boldsymbol{j}_b &= 1, \\ {}^a\boldsymbol{k}_b^{\top\,a}\boldsymbol{k}_b &= 1. \end{aligned} \tag{2.31}$$

- Orthogonality. Its three columns (rows) are orthogonal to each other, that is

$$\begin{aligned} {}^a\boldsymbol{i}_b^{\top\,a}\boldsymbol{j}_b &= 0, \\ {}^a\boldsymbol{j}_b^{\top\,a}\boldsymbol{k}_b &= 0, \\ {}^a\boldsymbol{k}_b^{\top\,a}\boldsymbol{i}_b &= 0. \end{aligned} \tag{2.32}$$

Therefore, due to the existence of the 6 constraints shown above, it is possible to represent ${}^a_b\boldsymbol{R}$ with three parameters only, even if in a not unique manner.

2.1.6.1 Euler angles

Any orientation can be achieved by composing three elementary rotations in sequence. These can occur either about the axes of a fixed coordinate system (extrinsic rotations), or about the axes of a rotating coordinate system (intrinsic rotations) initially aligned with the fixed one. Usually the rotating frame is assumed attached to a rigid body. We have the followinig distinction:

- Proper Euler angles: x-z-x, y-z-y, ...
- Tait-Bryan angles: z-y-x, x-y-z, ...

The notation z-y'-x" denotes an intrinsic rotation, first around the axis z, then about the newly found axis y' and finally about the axis x". The sequence corresponds to the yaw-pitch-roll sequence, which is the most used.

$$ {}^{a}_{b}\boldsymbol{R} = \boldsymbol{R}_z(\psi)\boldsymbol{R}_y(\theta)\boldsymbol{R}_x(\phi) \tag{2.33} $$

2.1.6.2 Exponential (Angle-Axis) Representation

A particularly interesting minimal representation of 3D rotation matrices is the so called *exponential* or *angle-axis* representation. To introduce such a parameterization, let us recall the definition of the exponential of a matrix operator. Given a real square matrix $\boldsymbol{M} \in \mathbb{R}^{n\times n}$ and $\theta \in \mathbb{R}$, the exponential matrix of $\theta\boldsymbol{M}$ belongs to $\mathbb{R}^{n\times n}$ and is defined as

$$ e^{\theta\boldsymbol{M}} = \sum_{k=0}^{\infty} \frac{(\theta\boldsymbol{M})^k}{k!}. \tag{2.34} $$

Then, let us consider two frames $\langle\alpha\rangle$, $\langle\beta\rangle$, initially coinciding. Let us further consider an applied geometric unit vector $(\boldsymbol{v}, O_\alpha) = (\boldsymbol{v}, O_\beta)$ passing through the common origin of the two frames, whose initial projection on $\langle\alpha\rangle$ is the same of that on $\langle\beta\rangle$. Then let us consider that frame $\langle\beta\rangle$ (and consequently its associated reference system) is purely rotated around $\boldsymbol{v}$ of an angle θ, even negative, accordingly with the right-hand rule, as indicated in Fig. **??**.

So doing, we note that the axis-line defined by $(\boldsymbol{v}, O_\alpha) = (\boldsymbol{v}, O_\beta)$ remains common to both the reference systems of the two frames $\langle\alpha\rangle$, $\langle\beta\rangle$, (like the axis of the hinge of a door, which belongs to the door whatever is its rotation, as well as to the wall) we have that

$$ {}^{\alpha}\boldsymbol{v} = {}^{\beta}\boldsymbol{v} \triangleq {}^{*}\boldsymbol{v},\ \forall\theta, \tag{2.35} $$

where the introduced notation ${}^{*}\boldsymbol{v}$ recalls that the geometric vector $\boldsymbol{v}$ can be indifferently projected on frames $\langle\alpha\rangle$ or $\langle\beta\rangle$.

Then, the orientation matrix constructed in the above way is said to be represented by its equivalent angle-axis representation that admits the following equivalent analytical expressions (Rodrigues formula)

$$ \boldsymbol{R}({}^{*}\boldsymbol{v}, \theta) = e^{[{}^{*}\boldsymbol{v}\wedge]\theta} = \boldsymbol{I}_{3\times 3} + [{}^{*}\boldsymbol{v}\wedge]\sin\theta + [{}^{*}\boldsymbol{v}\wedge]^2(1-\cos\theta), \tag{2.36} $$

where the last expression is the direct consequence of the fact that the matrix exponent is actually a skew symmetric one.

2.1.6.3 Unit Vector Lemma

The angle-axis vector can be computed using the following "unit vector lemma". Given two frames $\langle a\rangle$, $\langle b\rangle$, the following equivalence holds:

$$ \begin{cases} (\boldsymbol{i}_a \wedge \boldsymbol{i}_b) + (\boldsymbol{j}_a \wedge \boldsymbol{j}_b) + (\boldsymbol{k}_a \wedge \boldsymbol{k}_b) = 2\boldsymbol{v}\sin\theta \\ (\boldsymbol{i}_a \cdot \boldsymbol{i}_b) + (\boldsymbol{j}_a \cdot \boldsymbol{j}_b) + (\boldsymbol{k}_a \cdot \boldsymbol{k}_b) = 1 + 2\cos\theta \end{cases} \tag{2.37} $$

These equations allow us to compute the angle-axis vector $\boldsymbol{v}, \theta$ from the knowledge of the rotation matrix ${}^{a}_{b}\boldsymbol{R}$.

Let us express (2.37) in a frame of our choice, say $\langle s\rangle$, obtaining

$$ \begin{cases} [{}^{s}\boldsymbol{i}_a\wedge]{}^{s}\boldsymbol{i}_b + [{}^{s}\boldsymbol{j}_a\wedge]{}^{s}\boldsymbol{j}_b + [{}^{s}\boldsymbol{k}_a\wedge]{}^{s}\boldsymbol{k}_b = 2{}^{s}\boldsymbol{v}\sin\theta \\ ({}^{s}\boldsymbol{i}_a^{\top}{}^{s}\boldsymbol{i}_b) + ({}^{s}\boldsymbol{j}_a^{\top}{}^{s}\boldsymbol{j}_b) + ({}^{s}\boldsymbol{k}_a^{\top}{}^{s}\boldsymbol{k}_b) = 1 + 2\cos\theta \end{cases} \tag{2.38} $$

where we have defined the skew matrix as

$$ [{}^{s}\boldsymbol{i}_a\wedge] = \begin{bmatrix} 0 & -{}^{s}\boldsymbol{i}_{a_z} & {}^{s}\boldsymbol{i}_{a_y} \\ {}^{s}\boldsymbol{i}_{a_z} & 0 & -{}^{s}\boldsymbol{i}_{a_x} \\ -{}^{s}\boldsymbol{i}_{a_y} & {}^{s}\boldsymbol{i}_{a_x} & 0 \end{bmatrix}. \tag{2.39} $$

Solving (2.38) gives us ${}^{s}\boldsymbol{v}, \theta$.

Note that:

$$ {}^{s}_{a}\boldsymbol{R} = [{}^{s}\boldsymbol{i}_{a}|...] \qquad {}^{s}_{b}\boldsymbol{R} = [{}^{s}\boldsymbol{i}_{b}|...] \tag{2.40} $$

This means that ${}^{s}_{a}\boldsymbol{R}$ and ${}^{s}_{b}\boldsymbol{R}$ contain all the vectors needed in the projected versor lemma equation. From their knowledge it is possible to compute the angle-axis representation, which will be projected on the common frame $\langle s \rangle$.

2.2 Kinematics Fundamentals

2.2.1 Time Derivative of Orientation and Transformation Matrices

Let us start by considering the rate of change of a generic transformation matrix between two frames, within a given universe. This analysis, and in particular the analysis regarding its orientation part, will allow us to introduce the important concept of angular velocity vector.

To these aims, let us start by considering two frames $\langle a \rangle$, $\langle b \rangle$, extracted from a given universe, which are assumed to be moving inside it, and in general moving one with respect to the other. This is described by the, time varying, transformation matrix ${}^{a}_{b}\boldsymbol{T}(t)$. In particular, let us adopt the point of view of an observer at all times fixed with frame $\langle a \rangle$ (i.e. an observer in the reference system of frame $\langle a \rangle$).

Then, the time evolution of ${}^{a}_{b}\boldsymbol{T}(t)$ is governed by its time derivative ${}^{a}_{b}\dot{\boldsymbol{T}}(t)$ taking the form

$$ {}^{a}_{b}\dot{\boldsymbol{T}}(t) = \begin{bmatrix} {}^{a}_{b}\dot{\boldsymbol{R}}(t) & {}^{a}\dot{O}_{b}(t) \\ \boldsymbol{0}_{3\times 1} & 0 \end{bmatrix}, \tag{2.41} $$

from which ${}^{a}_{b}\boldsymbol{T}(t)$ follows via time integration, from an initial condition ${}^{a}_{b}\boldsymbol{T}(0)$, of the following *separate* time integrations:

$$ {}^{a}_{b}\boldsymbol{R}(t) = \int_{0}^{t} {}^{a}_{b}\dot{\boldsymbol{R}}(t)dt + {}^{a}_{b}\boldsymbol{R}(0) \tag{2.42} $$

$$ {}^{a}O_{b}(t) = \int_{0}^{t} {}^{a}\dot{O}_{b}(t)dt + {}^{a}O_{b}(0). \tag{2.43} $$

The above equations imply and are implied by the following expressions termed *progressive-sums of infinitesimal contributions* associated to the corresponding time integrals:

$$ {}^{a}_{b}\boldsymbol{R}(t+dt) = {}^{a}_{b}\boldsymbol{R}(t) + {}^{a}_{b}\dot{\boldsymbol{R}}(t)dt + O(dt) \tag{2.44} $$

$$ {}^{a}O_{b}(t+dt) = {}^{a}O_{b}(t) + {}^{a}\dot{O}_{b}(t)dt + O(dt), \tag{2.45} $$

in which the notation $O(dt)$ means "infinitesimal terms of order greater than dt".

Let us remark how in (2.44) the sum of infinitesimal contributions ${}^{a}_{b}\dot{\boldsymbol{R}}(t)dt$ does not, in general, preserves the orthogonality of the resulting matrix. Indeed, let us consider the *exact* form of *progressive products of infinitesimal-rotation matrices*, for representing the evolution ${}^{a}_{b}\boldsymbol{R}(t)$ via a sequence of infinitesimal rotations:

$$ {}^{a}_{b}\boldsymbol{R}(t+dt) = {}^{a}_{b}\boldsymbol{R}(t)e^{\left[{}^{b}\boldsymbol{d\theta}_{b/a}\wedge\right]} = e^{\left[{}^{a}\boldsymbol{d\theta}_{b/a}\wedge\right]}{}^{a}_{b}\boldsymbol{R}(t), \tag{2.46} $$

where the infinitesimal rotation vector $\boldsymbol{d\theta}_{b/a}$ has been directly defined, without any relationship with any pre-existing finite one. Note that the lower-right index b/a, appearing in both projected vectors and related free-from-coordinate abstraction, has to be read as "frame $\langle b \rangle$ with respect to $\langle a \rangle$". This serves for recalling which is the referred frame ($\langle a \rangle$), with respect to which the referring one ($\langle b \rangle$) is currently infinitesimally changing its orientation.

Let us now represent the geometric vector $\boldsymbol{d\theta}_{b/a}$ via the following geometric differential form

$$ \boldsymbol{d\theta}_{b/a} \triangleq \boldsymbol{\omega}_{b/a}dt, \tag{2.47} $$

where the introduced geometric vector $\boldsymbol{\omega}_{b/a}$ has the following properties:

- its unit vector $\boldsymbol{u}_{b/a}$ identifies at time t the instantaneous axis around which frame $\langle b \rangle$ can be considered as instantaneously rotating with respect to $\langle a \rangle$;
- its norm $\dot{\alpha} \triangleq \boldsymbol{u}_{b/a} \cdot \boldsymbol{\omega}_{b/a}$ identifies the instantaneous rotation rate at time instant t.

Such a geometric vector is termed the *angular velocity vector* of frame $\langle b \rangle$ with respect to $\langle a \rangle$. Again, let us remark how the term "velocity" does not mean, in general, that the vector corresponds to the time-derivative of a pre-existing vector.

By formerly substituting the definition of angular velocity (2.47) in (2.46) and then in (2.44) one obtains

$$ {}^a_b\boldsymbol{R}(t)e^{\left[{}^b\boldsymbol{\omega}_{b/a}\wedge\right]dt} = e^{\left[{}^a\boldsymbol{\omega}_{b/a}\wedge\right]dt}{}^a_b\boldsymbol{R}(t) = {}^a_b\boldsymbol{R}(t) + {}^a_b\dot{\boldsymbol{R}}(t)dt + O(dt). \tag{2.48} $$

Rearranging some terms allow to find out

$$ {}^a_b\dot{\boldsymbol{R}}(t) = \frac{{}^a_b\boldsymbol{R}(t)\left(e^{\left[{}^b\boldsymbol{\omega}_{b/a}\wedge\right]dt} - \boldsymbol{I}_{3\times 3}\right)}{dt} - \frac{O(dt)}{dt} = \frac{\left(e^{\left[{}^a\boldsymbol{\omega}_{b/a}\wedge\right]dt} - \boldsymbol{I}_{3\times 3}\right){}^a_b\boldsymbol{R}(t)}{dt} - \frac{O(dt)}{dt} \tag{2.49} $$

that for $dt \to 0$ finally leads to the following (hint: represent the matrix-exponential as its power-series and then note how the ratios with dt of all terms of greater infinitesimal order, necessarily go to zero):

$$ {}^a_b\dot{\boldsymbol{R}}(t) = {}^a_b\boldsymbol{R}(t)\left[{}^b\boldsymbol{\omega}_{b/a}\wedge\right] = \left[{}^a\boldsymbol{\omega}_{b/a}\wedge\right]{}^a_b\boldsymbol{R}(t), \tag{2.50} $$

which are matrix differential equations termed (for historical reasons) *Strap-down equations* of the first and second type, respectively. Such equations establish the searched structural constraints to which ${}^a_b\dot{\boldsymbol{R}}(t)$ obeys and highlight that the *sole cause* for the time change of a rotation matrix, of a frame with respect to another one, is just the angular velocity vector of the referring frame with respect to the referred one.

2.2.2 Composition of Angular Velocity Vectors

Let us consider three frames $\langle a \rangle$, $\langle b \rangle$, $\langle c \rangle$ for which we assume to know $\boldsymbol{\omega}_{b/a}, \boldsymbol{\omega}_{c/b}$. We wish to evaluate $\boldsymbol{\omega}_{c/a}$. In this situation we have that

$$ {}^a_c\boldsymbol{R}(t+dt) = {}^a_b\boldsymbol{R}(t+dt){}^b_c\boldsymbol{R}(t+dt) = {}^a_b\boldsymbol{R}(t)e^{\left[{}^b\boldsymbol{\omega}_{b/a}\wedge\right]dt}{}^b_c\boldsymbol{R}(t)e^{\left[{}^c\boldsymbol{\omega}_{c/b}\wedge\right]dt}. \tag{2.51} $$

Now, using the r.h.s equality in (2.46), let us transfer the infinitesimal orientation matrices to the right, i.e.

$$ {}^a_c\boldsymbol{R}(t+dt) = {}^a_b\boldsymbol{R}(t){}^b_c\boldsymbol{R}(t)e^{\left[{}^c\boldsymbol{\omega}_{b/a}\wedge\right]dt}e^{\left[{}^c\boldsymbol{\omega}_{c/b}\wedge\right]dt} = {}^a_c\boldsymbol{R}(t)e^{\left[{}^c\boldsymbol{\omega}_{b/a}\wedge\right]dt}e^{\left[{}^c\boldsymbol{\omega}_{c/b}\wedge\right]dt}. \tag{2.52} $$

From the previous equation, let us compute the associated incremental ratio at time t as

$$ \frac{{}^a_c\boldsymbol{R}(t+dt) - {}^a_c\boldsymbol{R}(t)}{dt} = \frac{{}^a_c\boldsymbol{R}(t)\left(e^{\left[{}^c\boldsymbol{\omega}_{b/a}\wedge\right]dt}e^{\left[{}^c\boldsymbol{\omega}_{c/b}\wedge\right]dt} - \boldsymbol{I}_{3\times 3}\right)}{dt}, \tag{2.53} $$

whose limit for $dt \to 0$ provides the following result (hint: represent the matrix exponential as series expansions; then imagine to execute the resulting countable products; and then, as usual, keep into account that all ratios by dt of infinitesimals of order greater than dt tend to zero for $dt \to 0$):

$$ {}^a_c\dot{\boldsymbol{R}}(t) = {}^a_c\boldsymbol{R}(t)\left[{}^c\left(\boldsymbol{\omega}_{b/a} + \boldsymbol{\omega}_{c/b}\right)\wedge\right], \tag{2.54} $$

where we recognize the strap-down equation of the first type for frame $\langle c \rangle$ with respect to frame $\langle a \rangle$. Consequently, we have that

$$ \boldsymbol{\omega}_{c/a} = \boldsymbol{\omega}_{b/a} + \boldsymbol{\omega}_{c/b}, \tag{2.55} $$

which is commonly known as the *law of composition (or addition) of angular velocity vectors*. Finally, if we consider $\langle c \rangle \equiv \langle a \rangle$ we immediately get the well know result

$$ \boldsymbol{\omega}_{a/b} = -\boldsymbol{\omega}_{b/a}. \tag{2.56} $$

As a direct consequence of the law of addition for angular velocity vectors, let us remark how the angular velocity vector $\boldsymbol{\omega}_{b/a}$ between two frames $\langle a \rangle$, $\langle b \rangle$, in its geometric form is actually *common* to that of any other couples of frames $\langle \bar{a} \rangle$, $\langle \bar{b} \rangle$ respectively belonging to their reference systems. Therefore, the notation $\boldsymbol{\omega}_{b/a}$ should be more properly read as the "angular velocity vector of the whole reference system associated to frame $\langle b \rangle$, with respect to the one associated to frame $\langle a \rangle$". This is because the observer is not only as associated with frame $\langle a \rangle$, but actually with the whole reference system established by $\langle a \rangle$ itself.

2.2.3 Time Derivative of Vectors

Within a given, generally time varying universe, consider a geometric vector $\boldsymbol{s}(t)$. Let us recall how such a geometric vector represents the *entire class* of its projected vectors within the given universe considered at time t. Furthermore, remember that a vector is considered "moving" inside the universe if it changes projection coordinates with respect to at least one of its frames.

Accordingly with this consideration, the way $\boldsymbol{s}(t)$ changes in time its coordinates depends on the projection frame. Therefore, let us start by considering the "time derivative of a *projected-vector* on a frame", i.e. the time derivative of the algebraic vector representing its projection on a given frame. To this aim, let us consider a frame $\langle a \rangle$, with an observer associated to it. For its point of view, the observer will see the time changes occurring on the projected vector ${}^{a}\boldsymbol{s}(t)$ as

$$ {}^{a}\dot{\boldsymbol{s}}(t) \triangleq \frac{d}{dt}{}^{a}\boldsymbol{s}(t) \triangleq D^{a}\boldsymbol{s}(t), \tag{2.57} $$

where the shorter notation ${}^{a}\dot{\boldsymbol{s}}(t)$ has to be intended under the implicit understanding of always performing *differentiation after projection*. Furthermore, accordingly with the way it is constructed (i.e. by differentiating each component of ${}^{a}\boldsymbol{s}(t)$), ${}^{a}\dot{\boldsymbol{s}}(t)$ consequently results into an algebraic vector directly defined on frame $\langle a \rangle$.

The previous equation (2.57) implies that ${}^{a}\boldsymbol{s}(t)$ can be reconstructed via the simple time-integration

$$ {}^{a}\boldsymbol{s}(t) = \int_{0}^{t} {}^{a}\dot{\boldsymbol{s}}(t)dt + {}^{a}\boldsymbol{s}(0) \tag{2.58} $$

$$ {}^{a}\boldsymbol{s}(t+dt) = {}^{a}\boldsymbol{s}(t) + {}^{a}\dot{\boldsymbol{s}}(t)dt + O(dt) \tag{2.59} $$

2.2.3.1 Change of Differentiation Frame

Another observer, located on frame $\langle b \rangle$, will see the projected vector ${}^{b}\boldsymbol{s}(t)$ vary accordingly with ${}^{b}\dot{\boldsymbol{s}}(t)$. We are interested in establishing a link between these two point of views. In general

$$ {}^{b}\dot{\boldsymbol{s}}(t) \neq {}^{b}_{a}\boldsymbol{R}\,{}^{a}\dot{\boldsymbol{s}}(t). \tag{2.60} $$

Indeed, applying the rotation matrix ${}^{b}_{a}\boldsymbol{R}$ to the time derivative ${}^{a}\dot{\boldsymbol{s}}(t)$ means only the projection of the time derivative ${}^{a}\dot{\boldsymbol{s}}(t)$ on another frame, i.e.

$$ {}^{b}({}^{a}\dot{\boldsymbol{s}}(t)) = {}^{b}_{a}\boldsymbol{R}\,{}^{a}\dot{\boldsymbol{s}}(t), \tag{2.61} $$

where the notation on the left hand side must be preserved due to the fact that, in general, ${}^{b}({}^{a}\dot{\boldsymbol{s}}(t)) \neq {}^{b}\dot{\boldsymbol{s}}(t)$.

Let us now formally derive the correct relationship between the two time derivatives. Let us consider the time derivative of the projected vector ${}^{a}\boldsymbol{s}(t)$, but this time let us express it as the result of the projection on frame $\langle a \rangle$ of ${}^{b}\boldsymbol{s}(t)$, that is:

$$ {}^{a}\dot{\boldsymbol{s}}(t) = D^{a}\boldsymbol{s}(t) = D\left({}^{a}_{b}\boldsymbol{R}\,{}^{b}\boldsymbol{s}(t)\right) = {}^{a}_{b}\boldsymbol{R}\left(D^{b}\boldsymbol{s}(t)\right) + {}^{a}_{b}\dot{\boldsymbol{R}}\,{}^{b}\boldsymbol{s}(t) = {}^{a}_{b}\boldsymbol{R}\,{}^{b}\dot{\boldsymbol{s}}(t) + {}^{a}_{b}\dot{\boldsymbol{R}}\,{}^{b}\boldsymbol{s}(t), \tag{2.62} $$

which, using the strap-down equation (2.50), leads to the following two equivalent expressions

$$ {}^{a}\dot{\boldsymbol{s}}(t) = \begin{cases} {}^{a}_{b}\boldsymbol{R}\,{}^{b}\dot{\boldsymbol{s}}(t) + {}^{a}_{b}\boldsymbol{R}\left[{}^{b}\boldsymbol{\omega}_{b/a}\wedge\right]{}^{b}\boldsymbol{s}(t) \\ {}^{a}_{b}\boldsymbol{R}\,{}^{b}\dot{\boldsymbol{s}}(t) + \left[{}^{a}\boldsymbol{\omega}_{b/a}\wedge\right]{}^{a}_{b}\boldsymbol{R}\,{}^{b}\boldsymbol{s}(t) \end{cases}. \tag{2.63} $$

These equations show how ${}^{a}\dot{\boldsymbol{s}}(t)$ generally differs from the mere projection on $\langle a \rangle$ of the time derivative ${}^{b}\dot{\boldsymbol{s}}(t)$ evaluated by $\langle b \rangle$ unless $\boldsymbol{\omega}_{b/a} \wedge \boldsymbol{s} = 0$. This would be the case when frames $\langle a \rangle$ and $\langle b \rangle$ exhibit *no-rotational motion* one with respect to the other at the current instant (i.e. $\boldsymbol{\omega}_{b/a} = 0$) or when $\boldsymbol{\omega}_{b/a}$ is completely aligned with $\boldsymbol{s}$.

The above two equivalent relationships can be expressed in a more compact notational way, once a free-from-coordinate notation is used for representing the arguments of the above appearing projected cross-product:

$$ {}^{a}\dot{\boldsymbol{s}} = {}^{a}\left({}^{b}\dot{\boldsymbol{s}}\right) + {}^{a}\left(\boldsymbol{\omega}_{b/a} \wedge \boldsymbol{s}\right) = {}^{a}\left({}^{b}\dot{\boldsymbol{s}}\right) - {}^{a}\left(\boldsymbol{s} \wedge \boldsymbol{\omega}_{b/a}\right) \tag{2.64} $$

which is called *law of change of differentiation frame in projected form*.

Finally, the above law can be expressed using only free from coordinate notations, via the definition of the following geometric vector

$$D_a \boldsymbol{s} \triangleq \frac{d_a}{dt}\boldsymbol{s} \triangleq \boldsymbol{i}_a \dot{x}_a + \boldsymbol{j}_a \dot{y}_a + \boldsymbol{k}_a \dot{z}_a, \tag{2.65}$$

which is termed as *time-derivative of a geometric vector with respect to a frame*. By simple inspection, we have that

$${}^a D_a \boldsymbol{s} = D^a \boldsymbol{s} = {}^a \dot{\boldsymbol{s}}. \tag{2.66}$$

Consequently, we can write the *law of change of differentiation frame in geometric form*

$$D_a \boldsymbol{s} = D_b \boldsymbol{s} + \boldsymbol{\omega}_{b/a} \wedge \boldsymbol{s} = D_b \boldsymbol{s} - \boldsymbol{s} \wedge \boldsymbol{\omega}_{b/a} \tag{2.67}$$

for which the same considerations previously made for its projected form can obviously be repeated, without any change.

As a concluding remark, the time derivative $D_a \boldsymbol{s}$ of a geometric vector, with respect to a frame $\langle a \rangle$, is the same one of those evaluated with respect to any frame of its reference system. Therefore, the notation $D_a \boldsymbol{s}$ should be read as the "time derivative of geometric vector $\boldsymbol{s}$ with respect to the reference system associated to frame $\langle a \rangle$".

2.2.4 Time Derivative of Constant Module Vectors

Let us consider the particular case of a vector $\boldsymbol{s}$ exhibiting a constant non-zero module, that is such that $|\boldsymbol{s}| = \sigma > 0$ and $\dot{\sigma} = 0$, to be then differentiated with respect to a given frame $\langle a \rangle$. To this aim, let us instrumentally consider any frame $\langle h \rangle$ constraited to admit $\boldsymbol{s}$ as one of its fixed vector, i.e. ${}^h \dot{\boldsymbol{s}} = 0$.

For the law of change of differentiation frame we have

$$D_a \boldsymbol{s} = \boldsymbol{w}_{h/a} \wedge \boldsymbol{s}; \;\; \forall \langle h \rangle \; s.t. \; D_h \boldsymbol{s} = 0 \tag{2.68}$$

where with $\boldsymbol{w}_{h/a}$ we have denoted the angular velocity vector of *anyone* of the frames obeying to the indicated condition. Let us note how the time derivative of a non-zero constant-module vector is necessarily *orthogonal* to the vector itself. From the equation above, we can immediately see that class of angular velocity vectors $\boldsymbol{w}_{h/a}$ associated to a non-zero constant-module vector is represented by the following linear manifold

$$\boldsymbol{w}_{h/a} \triangleq \boldsymbol{\omega}_{s/a} + \boldsymbol{n} z \; \forall z \in \mathbb{R}, \tag{2.69}$$

where $\boldsymbol{n} \triangleq \boldsymbol{s}/\sigma$ is the unit vector of $\boldsymbol{s}$ and where $\boldsymbol{\omega}_{s/a}$ corresponds to the unique angular velocity vector that, within the class, exhibits the minimum module. Such a minimum-module vector denotes the *angular-velocity-vector of a non-zero constant-module vector* $\boldsymbol{s}$ with respect to frame $\langle a \rangle$. It is used to write the minimal representation of the time derivative of a non-zero constant-module vector with respect to frame $\langle a \rangle$:

$$D_a \boldsymbol{s} = \boldsymbol{\omega}_{s/a} \wedge \boldsymbol{s} \tag{2.70}$$

2.2.5 Time Derivative of Generic Vectors

Let us now consider a geometric vector $\boldsymbol{s}$, again assumed to be non-zero, but not necessarily with constant module:

$$\boldsymbol{s} \triangleq \boldsymbol{n}\sigma; \;\; |\boldsymbol{n}| = 1; \;\; \sigma > 0 \tag{2.71}$$

Then the differentiation of $\boldsymbol{s}$ with respect to a generic frame $\langle a \rangle$ yields

$$D_a \boldsymbol{s} = D_a \left(\boldsymbol{n}\sigma \right) = \boldsymbol{n}\dot{\sigma} + \sigma D_a \boldsymbol{n} \tag{2.72}$$

where the first term is proportional to $\boldsymbol{n}$, hence is aligned to $\boldsymbol{s}$, and the second term is proportional to the time-derivative of the non-zero constant module unit-vector $\boldsymbol{n}$, hence is orthogonal to $\boldsymbol{s}$. For these reasons, let us term (2.72) as the *aligned/orthogonal representation (AO) of the first type* of the time derivative of a vector.

A representation equivalent to the above can also be obtained by starting directly from $D_a(\boldsymbol{s})$ and by decomposing it in the following way

$$D_a \boldsymbol{s} = \boldsymbol{n}(\boldsymbol{n}\bullet) D_a \boldsymbol{s} + [\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)] D_a \boldsymbol{s}, \tag{2.73}$$

which is termed as *AO representation of the second type* of the time derivative of a vector. By direct comparison between (2.72) and (2.73) we get

$$\begin{aligned} \dot{\sigma} &= \boldsymbol{n} \bullet D_a(\boldsymbol{s}), \\ \sigma D_a \boldsymbol{n} &= [\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)] D_a \boldsymbol{s}. \end{aligned} \tag{2.74}$$

Recalling that $\boldsymbol{n}$ is a unit vector (hence constant module) and the formula (2.70) for the derivative of a constant module vector, its derivative is also equal to

$$D_a \boldsymbol{n} = \boldsymbol{\omega}_{n/a} \wedge \boldsymbol{n}. \tag{2.75}$$

Therefore, expanding (2.72) with the latest result we have the so called *the AO representation of the third type*:

$$D_a \boldsymbol{s} = \boldsymbol{n}\dot{\sigma} + \sigma(\boldsymbol{\omega}_{s/a} \wedge \boldsymbol{n}) = \boldsymbol{n}\dot{\sigma} + \boldsymbol{\omega}_{s/a} \wedge \boldsymbol{s}, \tag{2.76}$$

where $\boldsymbol{\omega}_{s/a} \triangleq \boldsymbol{\omega}_{n/a}$ is the angular velocity vector exhibited by $\boldsymbol{s}$ (and consequently by all vectors aligned with it, including $\boldsymbol{n}$) whenever considered with invariant module equal to the one at current time.

2.2.6 Time Derivative of Rotation Vectors

Within this subsection, the expressions of the time derivative of rotation vectors between couples of frames and between couples of unit vectors will be discussed. The final results will be formerly stated and commented, before providing their proof.

2.2.6.1 Time Derivative of Rotation Vectors for Couples of Frames

Consider two frames $\langle a \rangle$, $\langle b \rangle$ exhibiting the rotation matrix ${}^a_b\boldsymbol{R}$, and associated geometric (free from coordinate notation) rotation vector $\boldsymbol{\rho} \triangleq \boldsymbol{\rho}_{b/a}$. Let us assume frame $\langle b \rangle$ continuously changing its orientation with respect to $\langle a \rangle$ under the action of the relative angular velocity vector $\boldsymbol{\omega}_{b/a}$. We have already discussed the time derivative of ${}^a_b\boldsymbol{R}$ (see the strap down equations (2.50)). We are now interested in the time derivative of $\boldsymbol{\rho}$.

First of all, let us consider

$$\boldsymbol{\rho} = \boldsymbol{n}\theta;\ \boldsymbol{\rho} \neq 0;\ \theta \triangleq |\boldsymbol{\rho}|;\ \boldsymbol{n} \triangleq \boldsymbol{\rho}/\theta. \tag{2.77}$$

Then, following (2.72) let us write

$$D_a \boldsymbol{\rho} = \boldsymbol{n}\dot{\theta} + \theta D_a \boldsymbol{n}. \tag{2.78}$$

Using the fact that

$${}^a D_a \boldsymbol{\rho} = {}^b D_b \boldsymbol{\rho} \tag{2.79}$$

since by definition of rotation vector ${}^a\boldsymbol{\rho} = {}^b\boldsymbol{\rho}$, and hence it is consequently clear how two different observers respectively located in $\langle a \rangle$ and $\langle b \rangle$ will see such a projected common vector necessarily varying with the same modalities.

Then, it results

$$D_a \boldsymbol{\rho} = \boldsymbol{n}(\boldsymbol{n}\bullet)\boldsymbol{\omega}_{b/a} + \frac{\theta}{2}\left[\frac{\boldsymbol{I}_{3\times 3}}{\tan(\theta/2)} - (\boldsymbol{n}\wedge)\right][\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)]\,\boldsymbol{\omega}_{b/a}. \tag{2.80}$$

If we group $\boldsymbol{\omega}_{b/a}$ we have

$$D_a \boldsymbol{\rho} = [\boldsymbol{n}(\boldsymbol{n}\bullet) + \boldsymbol{N}_a(\theta)]\,\boldsymbol{\omega}_{b/a};\ \boldsymbol{N}_a(\theta) \triangleq \frac{\theta}{2}\left[\frac{\boldsymbol{I}_{3\times 3}}{\tan(\theta/2)} - (\boldsymbol{n}\wedge)\right][\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)], \tag{2.81}$$

where the linear operator $\boldsymbol{N}_a(\theta)$ transforms $\boldsymbol{\omega}_{b/a}$ into the sum of the indicated two vectors, both of them orthogonal to $\boldsymbol{\rho}$.

2.2.6.2 Time Derivative of Rotation Vectors for Couples of Vectors

Let us consider two unit vectors a, b and the rotation vector $\boldsymbol{\rho} = \boldsymbol{n}\theta$ between the two vectors. Our goal is to compute the time derivative of $\boldsymbol{\rho}$ with respect to an observer on a generic frame $\langle\alpha\rangle$, that is

$$D_\alpha \boldsymbol{\rho} = \boldsymbol{n}\dot{\theta} + \theta D_\alpha \boldsymbol{n}. \tag{2.82}$$

To this aim, let us first compute $\dot{\theta}$ recalling that

$$\boldsymbol{a} \wedge \boldsymbol{b} = \boldsymbol{n} \sin\theta \tag{2.83}$$
$$\boldsymbol{a} \bullet \boldsymbol{b} = \cos\theta \tag{2.84}$$

The, by differentiating the second equality (observer independent since it is a scalar quantity) one obtains

$$\boldsymbol{a} \bullet \left(\boldsymbol{w}_{b/\alpha} \wedge \boldsymbol{b}\right) + \boldsymbol{b} \bullet \left(\boldsymbol{w}_{a/\alpha} \wedge \boldsymbol{a}\right) = -\sin\theta\dot{\theta} \tag{2.85}$$

where we have maintained the point of view of an observer in $\langle\alpha\rangle$ and where $\boldsymbol{w}_{a/\alpha}, \boldsymbol{w}_{b/\alpha}$ are each one an element of the class of angular velocity vectors associated to $\boldsymbol{a}$ and $\boldsymbol{b}$ respectively (see (2.69)). By applying the cyclic property of the mixed products among vectors, one obtains

$$(\boldsymbol{a} \wedge \boldsymbol{b}) \bullet \left(\boldsymbol{w}_{b/\alpha} - \boldsymbol{w}_{a/\alpha}\right) \triangleq (\boldsymbol{a} \wedge \boldsymbol{b}) \bullet \boldsymbol{w}_{b/a} = \sin\theta\dot{\theta} \tag{2.86}$$

where the angular velocity difference $\boldsymbol{w}_{a/\alpha} - \boldsymbol{w}_{b/\alpha}$ results into the relative angular velocity $\boldsymbol{w}_{b/a}$. Substituting (2.83) in the above equation, we get the searched relationship

$$\dot{\theta} = \boldsymbol{n} \bullet \boldsymbol{w}_{b/a} \tag{2.87}$$

Let us now focus on the evaluation of $D_\alpha \boldsymbol{n}$:

$$\begin{aligned} D_\alpha \boldsymbol{n} &= D_\alpha \left(\frac{1}{\sin\theta} \boldsymbol{a} \wedge \boldsymbol{b}\right) \\ &= D_\alpha \left(\frac{1}{\sin\theta}\right) (\boldsymbol{a} \wedge \boldsymbol{b}) + \frac{1}{\sin\theta} D_\alpha (\boldsymbol{a} \wedge \boldsymbol{b}) \\ &= -\frac{\cos\theta\dot{\theta}}{\sin^2\theta} (\boldsymbol{a} \wedge \boldsymbol{b}) + \frac{1}{\sin\theta} \left(D_\alpha(\boldsymbol{a}) \wedge \boldsymbol{b} + \boldsymbol{a} \wedge D_\alpha(\boldsymbol{b})\right). \end{aligned} \tag{2.88}$$

To evaluate the rightmost term in the above equation, let us consider the derivative of the unit vectors $\boldsymbol{a}$ and $\boldsymbol{b}$

$$D_\alpha \boldsymbol{a} \triangleq \boldsymbol{w}_{a/\alpha} \wedge \boldsymbol{a} = \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right) \boldsymbol{w}_{a/\alpha}\right] \wedge \boldsymbol{a} + \left(\boldsymbol{n}(\boldsymbol{n}\bullet)\boldsymbol{w}_{a/\alpha}\right) \wedge \boldsymbol{a} \tag{2.89}$$
$$D_\alpha \boldsymbol{b} \triangleq \boldsymbol{w}_{b/\alpha} \wedge \boldsymbol{b} = \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right) \boldsymbol{w}_{b/\alpha}\right] \wedge \boldsymbol{b} + \left(\boldsymbol{n}(\boldsymbol{n}\bullet)\boldsymbol{w}_{b/\alpha}\right) \wedge \boldsymbol{b} \tag{2.90}$$

and let us substitute these expressions in the evaluation of $\frac{1}{\sin\theta}\left(D_\alpha(\boldsymbol{a}) \wedge \boldsymbol{b} + \boldsymbol{a} \wedge D_\alpha(\boldsymbol{b})\right)$ obtaining:

$$\frac{1}{\sin\theta} \left(D_\alpha(\boldsymbol{a}) \wedge \boldsymbol{b} + \boldsymbol{a} \wedge D_\alpha(\boldsymbol{b})\right) \triangleq \boldsymbol{t}_1 + \boldsymbol{t}_2 \tag{2.91}$$

where

$$\begin{aligned} \boldsymbol{t}_1 &\triangleq \frac{1}{\sin\theta} \left(\left\{\left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right) \boldsymbol{w}_{a/\alpha}\right] \wedge \boldsymbol{a}\right\} \wedge \boldsymbol{b} + \boldsymbol{a} \wedge \left\{\left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right) \boldsymbol{w}_{b/\alpha}\right] \wedge \boldsymbol{b}\right\}\right) \\ &= \frac{1}{\sin\theta} \left(\boldsymbol{b} \wedge \left\{\boldsymbol{a} \wedge \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right) \boldsymbol{w}_{a/\alpha}\right]\right\} - \boldsymbol{a} \wedge \left\{\boldsymbol{b} \wedge \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right) \boldsymbol{w}_{b/\alpha}\right]\right\}\right) \\ &= -\frac{1}{\sin\theta} \left(\boldsymbol{a} \wedge \left\{\boldsymbol{b} \wedge \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right)\right]\right\} \boldsymbol{w}_{b/\alpha} - \boldsymbol{b} \wedge \left\{\boldsymbol{a} \wedge \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right)\right]\right\} \boldsymbol{w}_{a/\alpha}\right) \\ &= -\frac{1}{\sin\theta} \left(\boldsymbol{a} \wedge \left\{\boldsymbol{b} \wedge \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right)\right]\right\} \boldsymbol{w}_{b/\alpha} - \boldsymbol{b} \wedge \left\{\boldsymbol{a} \wedge \left[\left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet)\right)\right]\right\} \boldsymbol{w}_{a/\alpha}\right) \\ &= \boldsymbol{N}(\theta)\boldsymbol{w}_{b/\alpha} - \boldsymbol{M}(\theta)\boldsymbol{w}_{a/\alpha} \end{aligned} \tag{2.92}$$

with the obvious definitions of the operators $\boldsymbol{N}(\theta)$ and $\boldsymbol{M}(\theta)$. Concerning the second term we have

$$\begin{aligned}
\boldsymbol{t}_2 &\triangleq \frac{1}{\sin\theta}\left\{\left[\left(\boldsymbol{n}(\boldsymbol{n}\bullet)\boldsymbol{w}_{a/\alpha}\right)\wedge\boldsymbol{a}\right]\wedge\boldsymbol{b}+\boldsymbol{a}\wedge\left[\left(\boldsymbol{n}(\boldsymbol{n}\bullet)\boldsymbol{w}_{b/\alpha}\right)\wedge\boldsymbol{b}\right]\right\}\\
&= \frac{1}{\sin\theta}\left\{(\boldsymbol{n}\bullet)\boldsymbol{w}_{a/\alpha}\left[(\boldsymbol{n}\wedge\boldsymbol{a})\wedge\boldsymbol{b}\right]+(\boldsymbol{n}\bullet)\boldsymbol{w}_{b/\alpha}\left[\boldsymbol{a}\wedge(\boldsymbol{n}\wedge\boldsymbol{b})\right]\right\}\\
&= \frac{1}{\sin^2\theta}\left\{(\boldsymbol{n}\bullet)\boldsymbol{w}_{a/\alpha}\left[((\boldsymbol{a}\wedge\boldsymbol{b})\wedge\boldsymbol{a})\wedge\boldsymbol{b}\right]+(\boldsymbol{n}\bullet)\boldsymbol{w}_{b/\alpha}\left[\boldsymbol{a}\wedge((\boldsymbol{a}\wedge\boldsymbol{b})\wedge\boldsymbol{b})\right]\right\}\\
&= \frac{1}{\sin^2\theta}\left\{-(\boldsymbol{n}\bullet)\boldsymbol{w}_{a/\alpha}(\boldsymbol{a}\bullet\boldsymbol{b})\left[\boldsymbol{a}\wedge\boldsymbol{b}\right]+(\boldsymbol{n}\bullet)\boldsymbol{w}_{b/\alpha}(\boldsymbol{a}\bullet\boldsymbol{b})\left[\boldsymbol{a}\wedge\boldsymbol{b}\right]\right\}\\
&= \frac{\cos\theta\left(\boldsymbol{n}\bullet(\boldsymbol{w}_{b/\alpha}-\boldsymbol{w}_{a/\alpha})\right)}{\sin^2\theta}(\boldsymbol{a}\wedge\boldsymbol{b})
\end{aligned} \tag{2.93}$$

Recalling the result of (2.87), it is evident that $\boldsymbol{t}_2$ cancels out with the term $-\frac{\cos\theta\dot{\theta}}{\sin^2\theta}(\boldsymbol{a}\wedge\boldsymbol{b})$ that was present in (2.88). This could not be otherwise, as $\boldsymbol{n}$ is a constant unit vector and its overall derivative cannot have terms directed along $\boldsymbol{n}$ itself. Hence, we have

$$D_\alpha\boldsymbol{n} = \boldsymbol{N}(\theta)\boldsymbol{w}_{b/\alpha}-\boldsymbol{M}(\theta)\boldsymbol{w}_{a/\alpha} \tag{2.94}$$

Hence

$$D_\alpha\boldsymbol{\rho} = \boldsymbol{n}\dot{\theta}+\theta D_\alpha\boldsymbol{n} = \boldsymbol{n}\boldsymbol{n}\bullet\boldsymbol{w}_{b/a}+\theta\boldsymbol{N}(\theta)\boldsymbol{w}_{b/\alpha}-\theta\boldsymbol{M}(\theta)\boldsymbol{w}_{a/\alpha}. \tag{2.95}$$

Note that, if we consider our observer on one of the two frames $\langle a\rangle$ or $\langle b\rangle$ the equation simplifies. For example, considering the derivative with respect to frame $\langle a\rangle$:

$$D_a\boldsymbol{\rho} = \boldsymbol{n}\boldsymbol{n}\bullet\boldsymbol{w}_{b/a}+\theta\boldsymbol{N}(\theta)\boldsymbol{w}_{b/a}. \tag{2.96}$$

In this particular case, we notice that the derivative of $\boldsymbol{\rho}$ is affected along $\boldsymbol{n}$ only by the projection of $\boldsymbol{w}_{b/a}$ along it, while in the plane orthogonal to $\boldsymbol{n}$ only by the orthogonal components of $\boldsymbol{w}_{b/a}$ to $\boldsymbol{n}$.

2.2.7 Time Derivative of Points

Within a given, generally time varying universe, now consider a geometric point $P(t)$. Let us remember how such a geometric point represents the entire class of its positions, or projections, on the frames of the given universe considered at time t and it is considered as "moving" if it changes its projection coordinates at least with respect to one of its frames. Consequently, here it does not make sense to speak about the time-derivative of a geometric point in an absolute sense, but only after having specified the frame with respect to which the time derivative has to be intended.

The time derivative of a geometric point $P(t)$ with respect to a frame $\langle a\rangle$ is defined as the following geometric vector

$$\boldsymbol{v}_{P/a}\triangleq D_a\boldsymbol{r}_{P,a} = \frac{d_a}{dt}\boldsymbol{r}_{P,a};\ \boldsymbol{r}_{P,a}\triangleq(P-O_a) \tag{2.97}$$

traditionally termed as the *linear velocity vector* of the point with respect to a frame.

First of all note, by inspection, that, once projected on the *same* reference/differentiation frame, vector ${}^a\boldsymbol{v}_{P/a}$ yields

$${}^a\boldsymbol{v}_{P/a} = {}^a\dot{P} \tag{2.98}$$

which is the time derivative of the projected point ${}^aP$ that can be used to reconstruct the evolution of ${}^aP(t)$ via time integration, i.e.

$$\begin{aligned}
{}^aP(t) &= \int_0^t {}^a\dot{P}dt+{}^aP(0)\\
{}^aP(t+dt) &= {}^aP(t)+{}^a\dot{P}dt+O(dt)
\end{aligned} \tag{2.99}$$

2.2.7.1 Composition of Linear Velocity Vectors

Consider again a geometric point $P(t)$, where now two different observers are respectively located on the two frames $\langle a\rangle$, $\langle b\rangle$. Each observer will associated to the same point P the velocity vectors $\boldsymbol{v}_{P/a}$ and $\boldsymbol{v}_{P/b}$. Now, let us write $\boldsymbol{r}_{P,a}$ equivalently as

$$\boldsymbol{r}_{P,a} = \boldsymbol{r}_{P,b}+\boldsymbol{r}_{b,a} \tag{2.100}$$

then by differentiating we obtain

$$\boldsymbol{v}_{P/a} = D_a(\boldsymbol{r}_{b,a} + \boldsymbol{r}_{P,b}) = D_a\boldsymbol{r}_{b,a} + D_a\boldsymbol{r}_{P,b}. \tag{2.101}$$

The first term is the velocity of the origin of frame $\langle b\rangle$ with respect to $\langle a\rangle$, i.e. $\boldsymbol{v}_{b/a}$. Furthermore, let us apply the law of change of differentiation frame to $D_a\boldsymbol{r}_{P,b}$ obtaining

$$\boldsymbol{v}_{P/a} = \boldsymbol{v}_{b/a} + D_b\boldsymbol{r}_{P,b} + \boldsymbol{\omega}_{b/a} \wedge \boldsymbol{r}_{P,b}, \tag{2.102}$$

in which we recognize that $D_b\boldsymbol{r}_{P,b} \triangleq \boldsymbol{v}_{P/b}$. Therefore, the *law of composition of linear velocity vectors* immediately follows as

$$\boldsymbol{v}_{P/a} = \boldsymbol{v}_{P/b} + \boldsymbol{v}_{b/a} + \boldsymbol{\omega}_{b/a} \wedge \boldsymbol{r}_{P,b}. \tag{2.103}$$

2.2.7.2 Points Attached to Rigid Spaces

If the point P belongs to the rigid space of $\langle b\rangle$ then $\boldsymbol{v}_{P/b} \equiv 0$, therefore (2.103) becomes

$$\boldsymbol{v}_{P/a} = \boldsymbol{v}_{b/a} + \boldsymbol{\omega}_{b/a} \wedge \boldsymbol{r}_{P,b}, \tag{2.104}$$

which is termed as the *law of composition of linear velocity vectors for points on a rigid space*, which is probably well know to the reader.

2.2.8 Time Derivative of Distance Vectors

Having defined the time derivative of points with respect to a frame (i.e. their linear velocity with respect to a frame) let us now tackle the problem of the time-derivative of a distance vector, i.e. of vectors which are really descending from a point difference. Therefore, let us consider the vector

$$\boldsymbol{s} = (P - Q). \tag{2.105}$$

Its time derivative with respect to a generic frame $\langle a\rangle$ is

$$D_a\boldsymbol{s} \triangleq \boldsymbol{v}_{s/a} = D_aP - D_aQ = \boldsymbol{v}_{P/a} - \boldsymbol{v}_{Q/a}. \tag{2.106}$$

Chapter 3

Single Agent Control: a Task Priority Approach

3.1 Case Study: Underwater Floating Manipulator Systems

For the development of the task priority based approach to the control of robotic systems, as well as the cooperation between multiple agents, we shall use the underwater floating manipulator systems (UVMS) as a case study. We will start from a single floating manipulator, which is the simplest case, and increase the complexity by considering a dual arm floating manipulator and then cooperative agents successively.

3.1.1 Notation

Vectors and matrices are expressed with a bold face character, such as $\boldsymbol{M}$, whereas scalar values are represented with a normal font such as γ. Given a matrix $\boldsymbol{M}$ and a vector $\boldsymbol{v}$:

- $M_{(i,j)}$ indicates the element of $\boldsymbol{M}$ at the i-th row and j-th column;
- $v_{(k)}$ refers to the k-th element of $\boldsymbol{v}$;
- $\boldsymbol{M}^{\#}$ is the exact generalized pseudo-inverse (see [**?**] for a review on pseudo-inverses and their properties), i.e. the pseudo inverse of $\boldsymbol{M}$ performed without any regularizations.

Further, less used, notation will be introduced as needed.

3.1.2 Degrees of Freedom of an UVMS

An UVMS is a robotic system composed of

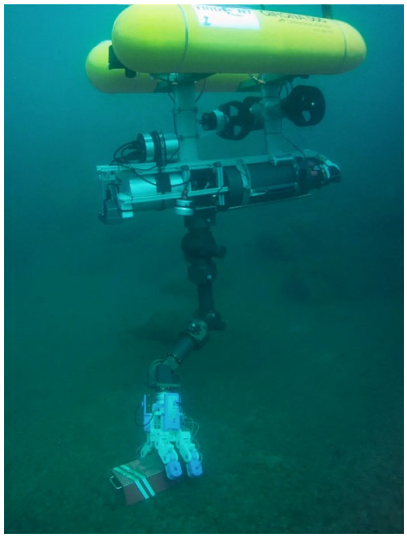
- a vehicle (6 DOFs);
- a manipulator (usually 6 or 7 DOFs)

A couple of examples of UVMS from the robotic literature are shown in Fig. 3.1

With DOF (degree of freedom) we mean the number of independent parameters that define the configuration of a mechanical system. Examples:

- an elevator has 1 DOF (distance along the track);
- a train still has 1 DOF (if cars are assumed rigidly connected);
- a car has 3 DOFs (if rigid suspensions are assumed), x, y and orientation;
- a rigid body in space is defined by 6 DOFs (3 for translational and 3 rotational).

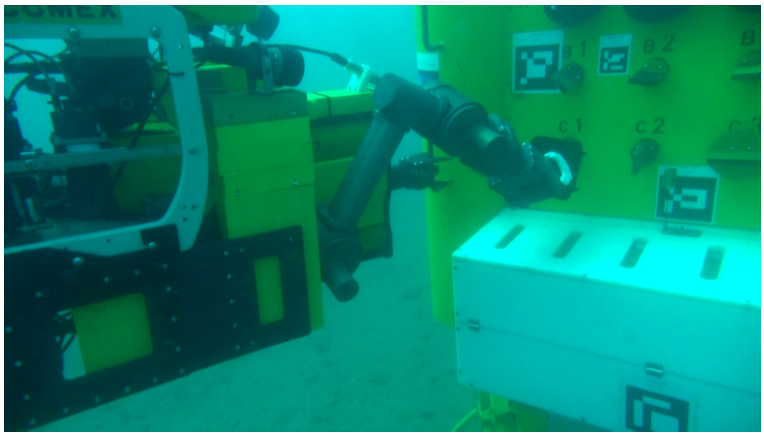
Not all the DOFs of a system are necessarily actuated. For example, most underwater vehicles do not have roll and pitch actuated, but rather they are designed to be passively stable in those DOFs (i.e. they tend to maintain a nominal position).



(a)



(b)



(c)

Figure 3.1: (a) TRIDENT UVMS (b) MARIS UVMS (c) DexROV UVMS.

3.1.3 Reference Frames

For the rest of these notes, we will use the following reference frames as depicted in Fig. 3.2.

3.1.4 Configuration of the System

Let us introduce a few definition. The configuration of the system is the vector of its DOF parameters. In the UVMS example we can define it as

$$\boldsymbol{c} \triangleq \begin{bmatrix} \boldsymbol{q} \\ \boldsymbol{\eta} \end{bmatrix}, \tag{3.1}$$

where

$$\boldsymbol{q} \triangleq \begin{bmatrix} q_1 \\ \vdots \\ q_n \end{bmatrix} \tag{3.2}$$

is the arm joint vector $\in \mathbb{R}^l$ and

$$\boldsymbol{\eta} \triangleq \begin{bmatrix} \boldsymbol{\eta}_1 \\ \boldsymbol{\eta}_2 \end{bmatrix} \in \mathbb{R}^6 \tag{3.3}$$

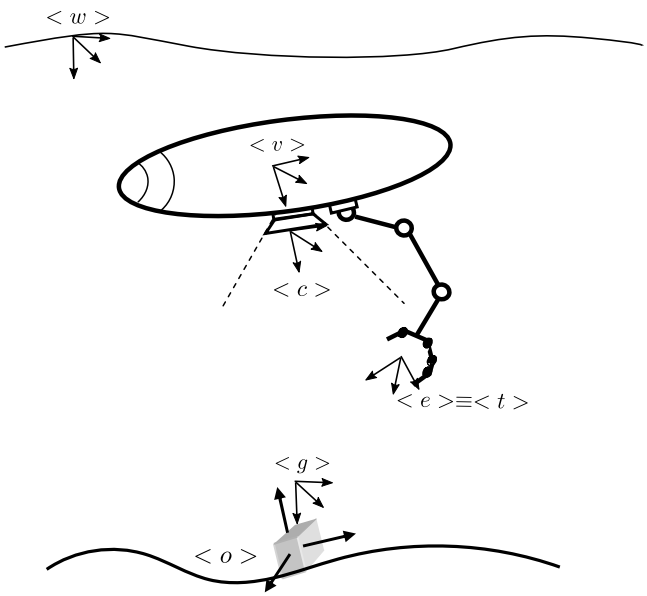


Figure 3.2: Relevant frames for an UVMS: $\langle w \rangle$ world inertial frame, $\langle v \rangle$ vehicle frame, $\langle c \rangle$ camera frame, $\langle o \rangle$ object frame, $\langle e \rangle$ end-effector frame, $\langle t \rangle$ tool frame, $\langle g \rangle$ goal frame.

is the position and orientation of the vehicle. In particular

$$\boldsymbol{\eta}_1 \triangleq \begin{bmatrix} x \\ y \\ z \end{bmatrix}, \ \boldsymbol{\eta}_2 \triangleq \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}, \tag{3.4}$$

where $\boldsymbol{\eta}_1$ is the vehicle frame position w.r.t. the world frame and $\boldsymbol{\eta}_2$ is the vehicle frame orientation expressed through its roll-pitch-yaw (RPY) representation which allows to define the rotation matrix ${}^{w}_{v}R$. The order of application of RPY angles is the following one: yaw (z) $\rightarrow$ pitch (y) $\rightarrow$ roll (x) (see Section 2.1.6.1).

3.1.5 Sensory Equipment of an UVMS

How does the robot knows is configuration vector?

3.1.5.1 Robot Position Estimation

$\boldsymbol{\eta}_1$ (robot position) is usually estimated through GPS for aerial and ground vehicles. However, for underwater vehicles this cannot be done, since GPS signal is absorbed by the water. A similar principle (triangulation) can be employed, but instead of electromagnetic signals, acoustic ones are used. Ultra short baseline (USBL) or long baseline (LBL) systems are usually employed for estimating the robot's position, and an example of how they work is shown in Fig. 3.3.

The problem is that since the acoustic waves propagate with approximately 1500 m/s in the water column, the update rate is very slow. This fact increase the difficulties of working in an underwater environment. Furthermore, note that:

- in LBL systems we need either clock synchronization or two-way ranging to estimate the time of flight (and thus the distance from the buoy); furthermore, the robot needs to receive periodically the buoy's position, unless it is moored;
- in USBL systems, the boat tracks the position of the underwater robot. Thus, the computed position must still be transmitted to the robot through an acoustic modem.

3.1.5.2 Robot Orientation Estimation

$\boldsymbol{\eta}_2$ (robot orientation) can be estimated through the fusion of different sensory information:

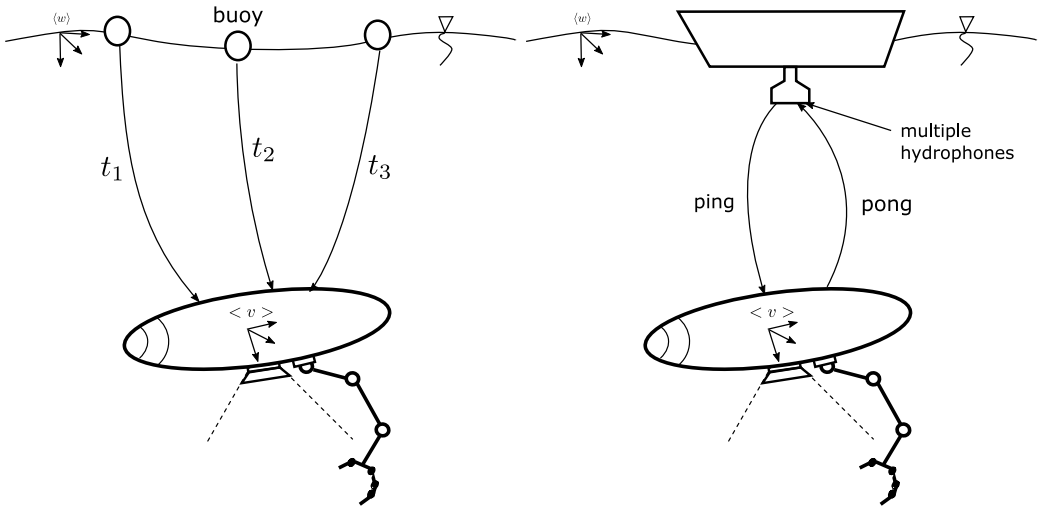


Figure 3.3: (a) LBL localization scheme: the vehicle triangulates its position on the basis of the time of flight of pings sent from the surface buoys. (b) USBL localization scheme: the USBL head on the boat emits a ping, the vehicle responds as soon as it receives the signal. The different hydrophones mounted on the USBL head will receive the pong signal with a slight delay depending on their mounting position w.r.t. the UVMS position. Through the analysis of the phase lag between these signals, the device computes the two angles (azimuth and elevation) to estimate the direction of the signal. The distance is estimated through the time of flight computation.

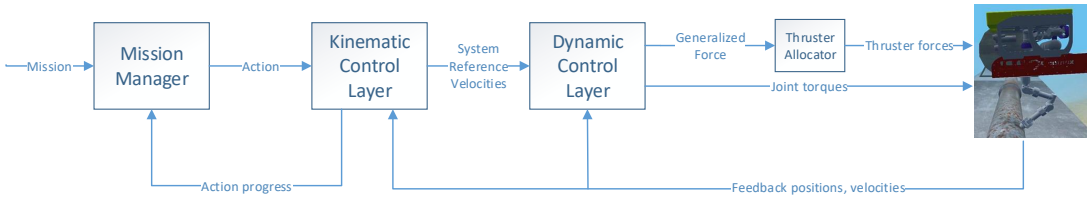


Figure 3.4: The overall architecture: the Kinematic Control Layer is the one implementing the proposed task priority approach, executing the current action scheduled by a Mission Manager; the output of the Kinematic Control Layer are the system velocities, tracked by the underlying Dynamic Control Layer.

- rate-gyroscope provides the angular velocity of the body;
- inclinometers or accelerometers provide the roll and pitch measurements;
- magnetometer provides the heading (yaw).

Together, these sensors form the INS (Inertial Navigation System).

3.2 General Control Framework Architecture

Let us consider as a explicative scenario the recovery of an object from the seafloor. The mission could be divided in different sequential phases:

1. Do a survey of the seafloor, to create a georeferenced photomosaic to allow the operator to individuate the object;
2. Once the operator has found the object, go the the approximate location;

3. Recover the object;

In each of the different phases, the UVMS has a different primary goal. For example, during the survey it is important to do path following, to ensure full coverage of the area. During the second phase, the UVMS should reach a waypoint. Finally in the last phase, the end-effector should reach the object's position allowing the recovery of the object.

We introduce the concept of *action*, which will be more formally explained later. For now, we can consider as action as a self-standing capability of the system. Therefore, navigation to a waypoint, survey execution, and end-effector positioning are examples of actions. Through their sequencing, the UVMS can be instructed to do a more complex mission. A more formal definition of action will be given in 3.5

The control framework discussed in these notes is based on the cascade of blocks shown in Fig. 3.4. In particular, the architecture is constituted by three main blocks:

1. The Mission Manager is in charge of supervising the execution of the current *mission*, and generates the corresponding *actions* to be executed by the Kinematic Control Layer.

2. The Kinematic Control Layer (KCL) implements the task priority control framework, and is in charge of reactively accomplishing the *control objectives* that make up the current action to be executed, generating the desired system velocity vector.

3. The Dynamic Control Layer (DCL) tracks the desired system velocity vector by generating appropriate force/torques references for the vehicle and the manipulator.

These notes will focus on the Kinematic Control Layer, since it is the one implementing the task priority approach. The interfaces with the higher level (Mission Manager) and the lower one (DCL) are highlighted whenever relevant.

3.3 Control Objectives

What does the robot need to accomplish? Suppose the robot's main mission is to recover an object from the seafloor. It turns out that the robot must take care of many concurrent objectives not strictly related to the mission. We shall see in the following how they can be defined, how can they be grouped and a few real-life examples.

3.3.1 Objective Definition

How are the control objectives defined?

Consider a configuration dependant scalar variable $x(\boldsymbol{c})$. We have an *equality control objective* when it is eventually required to satisfy

$$x(\boldsymbol{c}) = x_0, \tag{3.5}$$

or an *inequality control objective* when it is required to satisfy

$$x(\boldsymbol{c}) \geq x_m \tag{3.6}$$

or

$$x(\boldsymbol{c}) \leq x_M, \tag{3.7}$$

where the subscripts m and M indicate a minimum and maximum value respectively. The case where a variable needs to stay within an interval can be represented by two separate objectives.

Remark 1: We can also consider as $x(\boldsymbol{c})$ the modulus of a certain vector $\boldsymbol{p}$. This allows to control its norm to a particular value (for instance if we want to nullify some error vector), to be below a given threshold (if we just need to keep a reasonable error bound), or to be above a threshold (for instance if such a vector is a distance between two objects to avoid collisions).

Remark 2: We can also consider $x(\boldsymbol{c})$ to be the $i-$th component of a vector $\boldsymbol{p} \in \mathbb{R}^m$. Then if we consider m different variables $x_i(\boldsymbol{c})$, it is possible to control the vector $\boldsymbol{p}$ to any desired value.

3.3.2 Categories of Objectives

With the definition of the *action* concept given in 3.2, control objectives can be immediately divided in two groups: action defining objectives, and all the others. For example, the path following objective and the vehicle position control (to reach a waypoint) are those defining the respective actions. However, while executing both actions, it is important to stay away from the seafloor as a safety measure. There will be a control objective to ensure a minimum altitude from the seafloor.

In general, non-action defining objectives can be further divided in different categories. In the end, the control objectives of the UVMS can be divided in five broad categories:

- objectives related to physical constraints, i.e. tasks that deal with the interaction with the environment;
- objectives related to the safety of the system, e.g. avoiding joint limits or obstacles;
- objectives that are a prerequisite for the execution of the current *action*, e.g. maintaining the object to be manipulated in the camera vision system;
- *action* oriented objectives, i.e. what the system really needs to accomplish at this stage;
- optimization objectives, i.e. objectives that do not influence the mission, but allow to choose between multiple solutions, in case multiple solution exist.

We will see later why this division is very convenient. A more precise definition of what an *action* is will also be given later.

3.3.3 Examples of Control Objectives for an UVMS

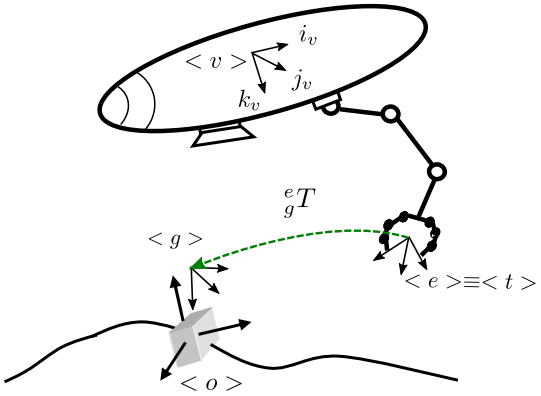


Figure 3.5: Relevant vectors and frames for the end-effector position control objective.

1. End-effector/Tool-frame Position Control (equality, action defining): this objective requires that the tool frame $\langle t \rangle$, rigidly attached to the end-effector space, converges to a given goal frame $\langle g \rangle$. In other words, the following two equality objectives must be eventually satisfied

$$r_{(i)} = 0,\ i = 1, 2, 3\,; \quad \vartheta_{(i)} = 0,\ i = 1, 2, 3\,; \tag{3.8}$$

   where $\boldsymbol{r}$ is the position error and $\boldsymbol{\vartheta}$ the orientation error between the tool and goal frames. Note that we have chosen a component by component zeroing for achieving a straight convergence to the target (especially important for grasping tasks).

   This is the easiest objective to be identified, since it is related to the mission (i.e. grasping an object from the seafloor). But what are the other objectives?

2. Joint Limits Objective (inequality, safety): the manipulator must operate within its joint limits, which means having the following inequality control objectives fulfilled

$$\begin{cases} q_i \geq q_{i,m} \\ q_i \leq q_{i,M} \end{cases} \quad i = 1, \ldots, l, \tag{3.9}$$

where q_i is the i-th joint variable, $q_{i,m}$ is the lower bound and $q_{i,M}$ is the higher one for the joint i, and where l is the total number of joints of the manipulator.

3. Dexterity Objective (inequality, operational prerequisite): Not all the arm configurations are the same from the dexterity point of view. In certain configurations, the control effort to achieve a desired end-effector velocity is greater than in others. For these reasons, the arm must maintain a certain minimum level of dexterity, to be able to perform the manipulation tasks.

   How do we measure the dexterity of a configuration? One possible index is the so called manipulability measure

$$\mu \triangleq \sqrt{\det(\boldsymbol{J}\boldsymbol{J}^\top)}. \tag{3.10}$$

   If μ is small we are close to a kinematic singularity, i.e. a configuration where the determinant is zero. In its vicinities, the control action $\dot{\boldsymbol{q}}$ tends to infinity for certain $\dot{\boldsymbol{x}}$.

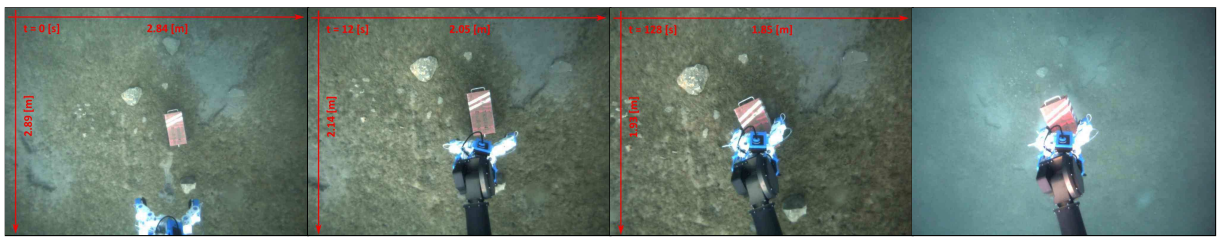
   The objective thus can be stated in the following terms

$$\mu > \mu_m. \tag{3.11}$$

4. Camera Centering Objective (inequality, operational prerequisite): Since in the underwater environment the availability of an absolute positioning system is not always guaranteed, the navigation toward a particular target and especially the grasping of an object is often carried out by means of the stereo vision of the target itself. Thus, it becomes important that the visual contact between the system and the target is maintained as the vehicle approaches it. This can be achieved by requiring that the target is roughly centered within the vision system. Toward that end, the control must ensure that the norm of the misalignment vector $\boldsymbol{\xi}$ between the vector joining the object $\langle o \rangle$ and the camera frame $\langle c \rangle$ with the z axis of the camera frame itself (supposed going outwards the camera image plane, as in Fig. 3.2) is within a maximum threshold, i.e.

$$\|\boldsymbol{\xi}\| \leq \xi_M. \tag{3.12}$$

5. Camera Occlusion Objective (inequality, operational prerequisite): a further inequality objective that might be required for the manipulator is to keep its links away from the camera system's cone of vision (assumed fixed to the vehicle), to avoid unnecessary occlusions of the target object frame $\langle o \rangle$. This task depends on the specific configuration of the arm: unavoidable occlusions might occur when the end-effector is in close proximity to the target, but in other cases other types of avoidable occlusions may happen, for instance when the elbow of the arm interferes with the camera well before the final grasping phases. In this case, a further inequality control objective can be introduced to force the elbow to stay outside the vision system cone.



(a) (b) (c) (d)

Figure 3.6: Sequence of snapshots taken from the onboard camera during the TRIDENT final experiments.

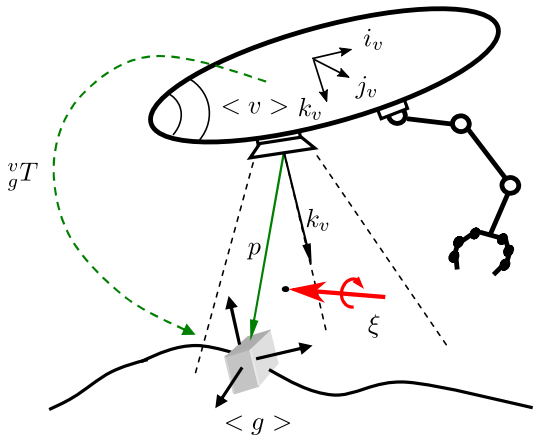


Figure 3.7: Relevant vectors and frames for the camera centering objective.

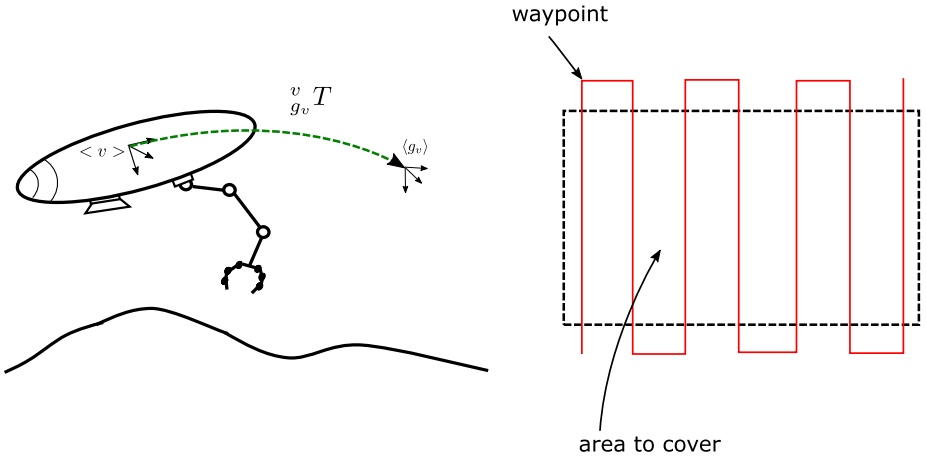


Figure 3.8: In the vehicle position control objective the goal is to get the vehicle frame $\langle v \rangle$ on top of the goal frame $\langle g_v \rangle$. This is usually exploited to perform a scan of a give area as depicted on the right side, where $\langle g_v \rangle$ is set to the end point of each leg of the survey, and updated once the vehicle has reached the waypoint 2.

6. Horizontal Attitude Objective (inequality, operational prerequisite): the vehicle attitude should be maintained within reasonable bounds, to avoid situations where the vehicle is upside-down. However, especially for vehicles and arms with comparable masses, such thresholds should be big enough to avoid excessive energy consumption to keep the vehicle perfectly horizontal whenever the arm moves its joints away from the center of buoyancy and thus tilts the vehicle. The above outlined control objective requires the achievement of the following inequality

$$\|\boldsymbol{\varphi}\| \leq \varphi_M, \tag{3.13}$$

where $\boldsymbol{\varphi}$ represents the misalignment vector that the absolute world frame z axis forms with respect to the vehicle z axis one. Note that most of the ROVs or AUVs are passively stable in roll and pitch and these d.o.f. are not controllable. For this reason, this objective should be considered only for fully actuated vehicles.

7. Vehicle Position Control (equality, action defining): another control objective is to have the vehicle frame $\langle v \rangle$ aligned with a particular goal frame $\langle g_v \rangle$. This could be required in order to bring the vehicle close to the area where the manipulation activity needs to be carried out, or to perform a scan of a given area (survey), as depicted in Fig. 3.8.

   This case is similar to the end-effector position control. This goal requires the achievement of

the following equality conditions

$$\|\boldsymbol{r}_v\| = 0 \;\; ; \;\; \|\boldsymbol{\vartheta}_v\| = 0, \tag{3.14}$$

where $\boldsymbol{r}_v$ is the position error and $\boldsymbol{\vartheta}_v$ the orientation error.

Note that, in certain applications, it could be enough to reach the waypoint within a sphere of certain radius. In such a case, the goal would require the achievement of the following *inequality* conditions

$$\|\boldsymbol{r}_v\| \leq r_{v,M} \;\; ; \;\; \|\boldsymbol{\vartheta}_v\| \leq \vartheta_{v,M}. \tag{3.15}$$

8. Vehicle Alignment (inequality, operational prerequisite): Another objective could be aligning the vehicle's longitudinal axis to a particular direction. For example, this was needed in the ESA rover control, during the extraction of the solar array. However, an alignment with the object to be manipulated could be very convenient to avoid having the arm assume weird configurations, and also to avoid moving too much the center of mass. If we denote with $\boldsymbol{\zeta}$ the misalignment vector between the vehicle's longitudinal axis and the desired direction, then the objective is to have
$$\|\boldsymbol{\zeta}\| \leq \zeta_M. \tag{3.16}$$

9. Vehicle Altitude (inequality, safety): the vehicle must maintain a certain distance from the seafloor to avoid collision. This objective can be simply stated as maintaining the altitude h above a certain threshold
$$h \geq h_m. \tag{3.17}$$

10. Vehicle Distance from Obstacles (inequality, safety): the UVMS must maintain a certain distance from known obstacles. This objective can be simply stated as maintaining the norm of the vector $\boldsymbol{d}$ joining two vehicle frame $\langle v \rangle$ with each obstacle frame $\langle o_i \rangle$ above a certain threshold value, i.e.
$$\|\boldsymbol{d}_i\| \geq d_m. \tag{3.18}$$

11. Arm Motion Minimality (equality, optimization): once an object has been grasped, it might be better if the arm minimizes its movements, leaving the bulk of the work to move the end-effector to the vehicle body. This allows to use the arm only to compensate the vehicle controller errors in tracking its reference velocity, as will be explained in Section 3.8.7;

12. Vehicle Motion Minimality (equality, optimization): during the grasping and manipulation phases it might be preferred to have the vehicle roughly stationary, unless its movement is strictly needed. This is a consequence of the fact that, usually, the control performances of the vehicle are fairly worse than those of the arms.

13. Arm Preferred Shape (inequality, optimization): another objective that might be useful is to have the arm maintaining a predefined desired shape. This can help maintaing a good dexterity (of course only if the predefined shape is chosen wisely!) if we define
$$\boldsymbol{e}_q \triangleq \begin{bmatrix} \bar{q}_i - q_i \\ \vdots \end{bmatrix} \tag{3.19}$$
and then require the following objective to be satisfied
$$\|\boldsymbol{e}_q\| \leq e_{q\,M}. \tag{3.20}$$

3.4 Control Tasks

3.4.1 Closed Loop Feedback Reference Rates

So far we have considered the set of objectives of our robot and divided them into groups. But how does a certain variable x reach its desired value (or region)?

We need to consider its time derivative $\dot{x}$. We introduce the reference rate:

$$\dot{\bar{x}} \triangleq \lambda(\bar{x} - x), \;\; \lambda > 0 \tag{3.21}$$

which represents the desired rate of change of the variable x. How does this law behaves?

- if $x > \bar{x}$, then $\dot{x} < 0$;
- if $x < \bar{x}$, then $\dot{x} > 0$;
- if $x = \bar{x}$, then $\dot{x} = 0$;

How do we choose $\bar{x}$?

- for equality objectives (see (3.5)) we have $\bar{x} = x_0$;
- for inequality objectives, $\bar{x}$ can be set equal to any point within the validity region of (3.6) or (3.7).

Graphically for our objectives:

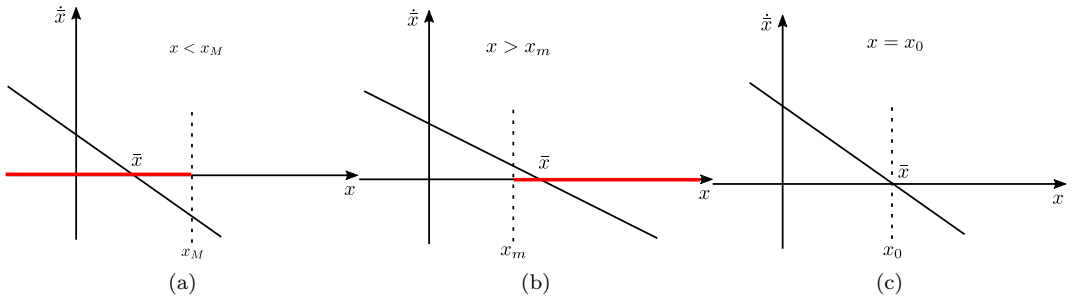


Figure 3.9: reference rate for objectives of type: (a) $x < x_M$ (b) $x > x_m$ (c) $x = x_0$

We note that a desired behaviour is imposed even when x is within its inequality range. This is not desirable, as it over constrains the system. For example, imagine the joint limit task, which has the highest priority. It would not allow any movement of the arm at all!

3.4.2 Tasks and Their Relationship with Control Variables

Now that we have defined a desired behavior for our variables of interest (with the caveat of not over-constraining when not necessary) how do we make our robot follow them? Clearly, the derivative of the variables of interest is not under our direct control. Instead, we can influence them by acting on some other variables. These are called control variables, and at the kinematic level are represented by the vehicle linear and angular velocities and the arm joint velocities:

$$\dot{\boldsymbol{y}} = \begin{bmatrix} \dot{\boldsymbol{q}} \\ \boldsymbol{\nu} \end{bmatrix} \qquad \boldsymbol{\nu} = \begin{bmatrix} \boldsymbol{\nu}_1 \\ \boldsymbol{\nu}_2 \end{bmatrix} \qquad \begin{matrix} \boldsymbol{\nu}_1 = {}^v\boldsymbol{v} \\ \boldsymbol{\nu}_2 = {}^v\boldsymbol{w} \end{matrix} \tag{3.22}$$

Note that $\dot{\boldsymbol{y}} \neq \dot{\boldsymbol{c}}$ because $\boldsymbol{\nu}_2$ is not the derivative of the RPY angles. However, there exists a Jacobian relationship between the angular velocity and the Euler angle derivatives, so $\dot{\boldsymbol{y}}$ is still a function of $\dot{\boldsymbol{c}}$:

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \tan(\theta)\sin(\phi) & \tan(\theta)\cos(\phi) \\ 0 & \cos(\phi) & -\sin(\phi) \\ 0 & \frac{\sin(\phi)}{\cos(\theta)} & \frac{\cos(\phi)}{\cos(\theta)} \end{bmatrix} \boldsymbol{\nu}_2$$

Since the variables we are considering are $x(\boldsymbol{c})$, a Jacobian relationship exists between them and $\dot{\boldsymbol{y}}$:

$$\dot{x} = \boldsymbol{g}^\top(\boldsymbol{c})\dot{\boldsymbol{y}} \tag{3.23}$$

3.4.3 Reactive and non-reactive Control Task Definitions

We define as *reactive control task* the need of tracking at best a suitable reference rate $\dot{\bar{x}}$ (as defined above), capable of driving the associated variable x toward the corresponding objective. Thus, for instance, a task is tracking at best a velocity reference rate generated to bring the arm's end-effector in the required Cartesian position.

Sometimes, a control task is defined directly in a certain task velocity space, without having an associated control objective. As an example, consider the case where a human operator wants to control the end-effector by generating velocity references through a joystick. In such a case, the reference rate $\dot{\tilde{x}}$ is generated by the user, rather than being the output of a feedback control loop as in (3.21). In this case, tracking the desired $\dot{\tilde{x}}$ is termed a *non-reactive control task*.

3.5 Actions

In the terminology of these notes, an *action* is a prioritized list of control tasks (and, for reactive ones, of their associated control objectives) to be *concurrently* managed. The meaning is that lower priority tasks should not influence the achievement of higher priority ones. If two or more scalar control tasks are assigned to the same priority level, then they are grouped into a (multidimensional) control task. For example, for a robotic manipulator, a manipulation action $\mathcal{A}_m$ could be represented by following list of prioritized control objectives

1. Arm joint limits;
2. Arm manipulability;
3. End-effector position control;
4. End-effector attitude control;
5. Arm preferred shape;

where it is natural to see *safety* objectives such as joint limits at the highest priority. Further remarks on how a list composing an action can be constructed will be given in section 3.8.5 and section 3.10. For the time being, let us assume that a list of prioritized control tasks has been established.

3.6 Task Activation Functions

One of the innovative points of this control architecture is the ability of handling inequality control objectives without overconstraining the system. Consider the aforementioned action $\mathcal{A}_m$, in which the joint limits avoidance is the highest priority control objective. Clearly, to maintain a joint away from its end of race, the associated reactive control task needs to be activated only in the vicinity of the limit itself. On the contrary, reactive control tasks corresponding to equality control objectives, like for instance driving the end-effector toward a precise position and attitude, must be always maintained active.

Motivated by the above considerations, for every control objective (and its reactive control task) there is always an associated *activation function* to be considered. The role of the activation function, as the name implies, is to specify whenever the reactive control task associated to a control objective is relevant for the system. Therefore, for each scalar inequality control objective, let us define an *activation function*

$$a^i(x) \in [0, 1] \tag{3.24}$$

as a continuous sigmoidal function of the objective variable $x(\boldsymbol{c})$, which assumes zero values within the validity region of the associated inequality objective. For example, assuming an objective of the class $x(\boldsymbol{c}) > x_{min}$:

$$a^i(x) \triangleq \begin{cases} 1, & x(\boldsymbol{c}) < x_{min} \\ s(x), & x_{min} \leq x(\boldsymbol{c}) \leq x_{min} + \Delta \\ 0, & x(\boldsymbol{c}) > x_{min} + \Delta \end{cases} \tag{3.25}$$

where $s(x)$ is any smooth, strictly decreasing function joining the two extrema, and Δ is a value that creates a buffer zone, where the inequality is already satisfied, but the activation value is still greater than zero, preventing chattering problems around the inequality control objective threshold x_{min}. An example of activation function (3.25) is reported in Fig. 3.10. A similar function can be defined for objectives of the class $x(\boldsymbol{c}) < x_{max}$.

It is important to note that, once the activation function is introduced with a given buffer Δ, the reference rate must be made pointing to the smaller subregion where the activation function is zero.

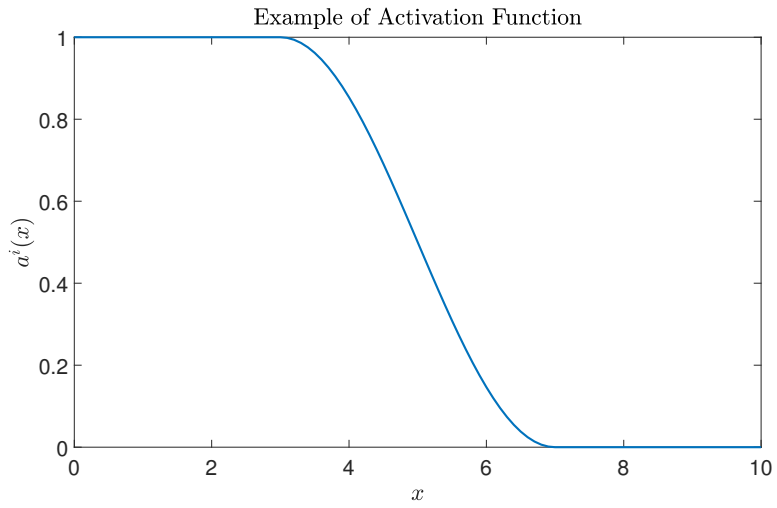


Figure 3.10: An example of activation function $a^i(x)$ for a control objectives that requires $x > 3$, where $\Delta = 4$.

For the example in Fig. 3.10, it means any point $x > 7$. The reason for this requirement is that, if a desired value $\bar{x}$ is chosen in the region $x_{min} < \bar{x} < x_{min} + \Delta$, then the task will always try to stay within its transition zone, possibly overconstraining the system. Indeed, in the case of $x > \bar{x}$, even if the contribution of lower priority tasks were in the direction of increasing x, the task itself would impose a negative behaviour back to $\bar{x}$. A good choice is thus choosing $\bar{x}$ as the first point where the activation is zero, i.e. $\bar{x} = x_{min} + \Delta$.

Finally, for both reactive control task associated to an equality control objective and for non-reactive control tasks, the activation function is always the unitary one, i.e $a^i(x) \equiv 1$. From now on, the distinction between reactive and non-reactive control tasks will be dropped, and the generic term control task will be used, unless otherwise specified.

3.7 Priorities between Control Tasks

Once all the control tasks have been individuated, their *priority* must be established. The task priority control in fact allows to establish a strict priority order between the control tasks and their associated objectives. The meaning is that lower priority tasks cannot influence the achievement of higher priority ones. If two or more scalar control objectives are assigned to the same priority level, then they are grouped into a (multidimensional) control objective.

After the above procedure has been followed, for each priority level k the following matrices and vectors are defined:

- $\dot{\bar{\boldsymbol{x}}}_k \triangleq \begin{bmatrix} \dot{\bar{x}}_{1,k} & \cdots & \dot{\bar{x}}_{m,k} \end{bmatrix}$ is the stacked vector of all the reference rates;
- $\boldsymbol{J}_k$ is the Jacobian relationship expressing the current rate-of-change of the k-th task vector $\begin{bmatrix} \dot{x}_{1,k} & \cdots & \dot{x}_{m,k} \end{bmatrix}$ w.r.t. the system velocity vector $\dot{\boldsymbol{y}}$, simply resulting from the stacking of the row vector ones in (3.23);
- $\boldsymbol{A}_k \triangleq \text{diag}(a_{1,k}, \cdots, a_{m,k})$ is the diagonal matrix of all the activation functions described in (3.24).

3.8 Task Priority Inverse Kinematics

3.8.1 Preliminaries on Least Squares Problems

Before introducing the core of these notes, it is useful to recall the fundamentals of pseudo inverses and the regularization mechanism. Toward that end, let us consider the following Jacobian relationship

$$\dot{\boldsymbol{x}} = \boldsymbol{J}\dot{\boldsymbol{y}}, \tag{3.26}$$

with $\boldsymbol{J} \in \mathbb{R}^{m\times n}$, $\dot{\boldsymbol{y}} \in \mathbb{R}^n$ and $\dot{\boldsymbol{x}} \in \mathbb{R}^m$.

Given a reference velocity vector $\dot{\bar{\boldsymbol{x}}}$, it is well known that the minimum norm velocity vector $\dot{\boldsymbol{y}}$ that realizes $\dot{\bar{\boldsymbol{x}}}$ at best, in a least-squares sense, can be found as the minimum norm solution of the following minimization problem

$$\min_{\dot{\boldsymbol{y}}} \left\|\dot{\bar{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}}\right\|^2. \tag{3.27}$$

Such a solution can be found computing the square

$$\left\|\dot{\bar{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}}\right\|^2 = (\dot{\bar{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}})^\top(\dot{\bar{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}}) = \dot{\bar{\boldsymbol{x}}}^\top\dot{\bar{\boldsymbol{x}}} + \dot{\boldsymbol{y}}^\top\boldsymbol{J}^\top\boldsymbol{J}\dot{\boldsymbol{y}} - 2\dot{\boldsymbol{y}}^\top\boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}} \tag{3.28}$$

and taking the derivative equal to zero

$$\frac{\partial}{\partial\dot{\boldsymbol{y}}}(\cdot) = \cancel{\dot{\bar{\boldsymbol{x}}}^\top\dot{\bar{\boldsymbol{x}}}} + 2\boldsymbol{J}^\top\boldsymbol{J}\dot{\boldsymbol{y}} - 2\boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}} = 0. \tag{3.29}$$

This leads to the condition

$$(\boldsymbol{J}^\top\boldsymbol{J})\dot{\boldsymbol{y}} = \boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}}, \tag{3.30}$$

which can be solved using the pseudo inverse as

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}^\top\boldsymbol{J})^\#\boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}}. \tag{3.31}$$

For more details on pseudo inverses, see the book [**?**] or briefly the Appendix A.1.

Since for the rest of the work we will focus on exploiting the residual arbitrariness in the solution, it is worth to consider the actual manifold of all solutions, by also including the null space, that is the manifold:

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}^\top\boldsymbol{J})^\#\boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}} + (\boldsymbol{I} - (\boldsymbol{J}^\top\boldsymbol{J})^\#\boldsymbol{J}^\top\boldsymbol{J})\dot{\boldsymbol{z}},\ \forall\dot{\boldsymbol{z}}. \tag{3.32}$$

Then, by exploiting the identities involving pseudo inverses, we note that (3.32) can be equivalently rewritten in the following, more usual form

$$\dot{\boldsymbol{y}} = \boldsymbol{J}^\#\dot{\bar{\boldsymbol{x}}} + (\boldsymbol{I} - \boldsymbol{J}^\#\boldsymbol{J})\dot{\boldsymbol{z}},\ \forall\dot{\boldsymbol{z}}. \tag{3.33}$$

Performing a regularization means changing the original minimization problem (3.27) by adding an additional regularization cost, as in the following

$$\min_{\dot{\boldsymbol{y}}} \left\|\dot{\bar{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}}\right\|^2 + \|\dot{\boldsymbol{y}}\|^2_{\boldsymbol{R}}\ ;\ \boldsymbol{R} \geq \boldsymbol{0}, \tag{3.34}$$

where the notation $\|\dot{\boldsymbol{y}}\|^2_{\boldsymbol{R}}$ indicates the weighted norm, i.e. $\dot{\boldsymbol{y}}^\top\boldsymbol{R}\dot{\boldsymbol{y}}$.

This is usually done to deal with the ill-definition of the solution near a singularity of the matrix $\boldsymbol{J}$. The solution of the regularized problem is found along the same steps performed for the original one, as the minimum norm solution of the following equation

$$(\boldsymbol{J}^\top\boldsymbol{J} + \boldsymbol{R})\dot{\boldsymbol{y}} = \boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}} \tag{3.35}$$

that is

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}^\top\boldsymbol{J} + \boldsymbol{R})^\#\boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}}, \tag{3.36}$$

while the manifold of all solutions is

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}^\top\boldsymbol{J} + \boldsymbol{R})^\#\boldsymbol{J}^\top\dot{\bar{\boldsymbol{x}}} + (\boldsymbol{I} - (\boldsymbol{J}^\top\boldsymbol{J} + \boldsymbol{R})^\#(\boldsymbol{J}^\top\boldsymbol{J} + \boldsymbol{R}))\dot{\boldsymbol{z}},\ \forall\dot{\boldsymbol{z}}. \tag{3.37}$$

However it is important to note that the above projection matrix $(\boldsymbol{I}-(\boldsymbol{J}^\top\boldsymbol{J}+\boldsymbol{R})^\#(\boldsymbol{J}^\top\boldsymbol{J}+\boldsymbol{R}))$ preserves the overall minimum of problem (3.34), including that of the regularization cost $\|\dot{\boldsymbol{y}}\|^2_{\boldsymbol{R}}$. This means that if $\boldsymbol{R} > \boldsymbol{0}$ then the projection matrix becomes equal to $\boldsymbol{0}$ (e.g. in the case of damped least squares, see the next subsection) and no arbitrariness is available.

For the above reason, the following substitution is instead typically performed

$$\boldsymbol{J}^\# \rightarrow (\boldsymbol{J}^\top\boldsymbol{J}+\boldsymbol{R})^\#\boldsymbol{J}^\top \tag{3.38}$$

and a manifold of solutions with $\boldsymbol{J}$ regularized by $\boldsymbol{R}$ is consequently constructed by analogy with (3.33) as

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}^\top\boldsymbol{J}+\boldsymbol{R})^\#\boldsymbol{J}^\top\dot{\hat{\boldsymbol{x}}} + (\boldsymbol{I}-(\boldsymbol{J}^\top\boldsymbol{J}+\boldsymbol{R})^\#\boldsymbol{J}^\top\boldsymbol{J})\dot{\boldsymbol{z}},\ \forall\dot{\boldsymbol{z}}. \tag{3.39}$$

It is noteworthy to see that the projection matrix $(\boldsymbol{I}-(\boldsymbol{J}^\top\boldsymbol{J}+\boldsymbol{R})^\#\boldsymbol{J}^\top\boldsymbol{J})$ is not anymore an orthogonal projector on the null space of $\boldsymbol{J}$ whenever $\boldsymbol{R} \neq \boldsymbol{0}$.

3.8.1.1 Damped Least Squares

Among the different regularization we can cite the damped least squares (DLS)

$$\min_{\dot{\boldsymbol{y}}} \left\|\dot{\hat{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}}\right\|^2 + \gamma^2\left\|\dot{\boldsymbol{y}}\right\|^2, \tag{3.40}$$

where thus $\boldsymbol{R} = \gamma^2\boldsymbol{I}$. In this case, finding the solution to the original problem (3.27) is balanced with the requirement of also maintaining all the components of the velocity vector $\dot{\boldsymbol{y}}$ limited. The scalar value γ balances this trade-off: smaller values of γ will favor accomplishing the original minimization, while greater values will maintain $\dot{\boldsymbol{y}}$ much more limited. A more detailed discussion on DLS can be found in the book [**?**] and in [**?**].

3.8.1.2 Singular Value Oriented Regularization

Another popular regularization is the SVD-based one (see the book [**?**] or the Appendix A.2 for a deeper investigation). In this case, the Jacobian matrix is first decomposed along its singular values:

$$\boldsymbol{J} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top, \tag{3.41}$$

with $\boldsymbol{U} \in \mathbb{R}^{m\times m}$, $\boldsymbol{\Sigma} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{V} \in \mathbb{R}^{n\times n}$. $\boldsymbol{\Sigma}$ is a rectangular diagonal matrix, whose diagonal elements are the singular values. $\boldsymbol{U}$ and $\boldsymbol{V}$ are orthogonal matrices, where the m columns of $\boldsymbol{U}$ and the n columns of $\boldsymbol{V}$ are called the left-singular vectors and right-singular vectors of $\boldsymbol{J}$ and are respectively the eigenvectors of $\boldsymbol{J}\boldsymbol{J}^\top$ and $\boldsymbol{J}^\top\boldsymbol{J}$.

It is easy to see that, if a singular value is small, there is a need for a big effort along corresponding control direction in $\boldsymbol{V}$ to accomplish a given task along the corresponding task direction of $\boldsymbol{U}$. To prevent the control from growing unbounded as the singular value approaches zero, a diagonal singular value oriented regularization matrix $\boldsymbol{P} \in \mathbb{R}^{n\times n}$ can be computed as follows: each diagonal elements $p_{(i,i)}$ of $\boldsymbol{P}$ is a bell-shaped, finite support function of the corresponding singular value, or it is zero if $i > m$ (i.e. the corresponding singular value does not exist).

With this definition, the singular value oriented (SVO) regularization problem becomes that of minimizing

$$\min_{\dot{\boldsymbol{y}}} \left\|\dot{\hat{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{y}}\right\|^2 + \left\|\boldsymbol{V}^\top\dot{\boldsymbol{y}}\right\|^2_{\boldsymbol{P}}, \tag{3.42}$$

where thus $\boldsymbol{R} = \boldsymbol{V}\boldsymbol{P}\boldsymbol{V}^\top$. In this case it is possible to see that the regularized solution is

$$\dot{\boldsymbol{y}} = \boldsymbol{V}(\boldsymbol{\Sigma}^\top\boldsymbol{\Sigma}+\boldsymbol{P})^\#\boldsymbol{\Sigma}^\top\boldsymbol{U}^\top\dot{\hat{\boldsymbol{x}}}, \tag{3.43}$$

and that the projection matrix becomes

$$\boldsymbol{V}(\boldsymbol{I}-(\boldsymbol{\Sigma}^\top\boldsymbol{\Sigma}+\boldsymbol{P})^\#\boldsymbol{\Sigma}^\top\boldsymbol{\Sigma})\boldsymbol{V}^\top. \tag{3.44}$$

Given the definition of $\boldsymbol{P}$ it is simple to see that only the directions with a small singular values, such that $p_{(i,i)} \neq 0$, are affected by the regularization. Finally, it is evident how with the use of the SVO regularization the matrix $(\boldsymbol{\Sigma}^\top\boldsymbol{\Sigma}+\boldsymbol{P})$ is always of constant rank m. An application of such a regularization method can be found in [**?**].

3.8.2 Classic Task Priority Algorithm

In this section we shall outline how the basic mechanism of any task priority algorithm works, using the original version proposed by [?] and generalized to any number of tasks in [?]. Let us consider the highest priority task. The solutions $\dot{\boldsymbol{y}}$ which satisfy at best, in a minimum norm fashion the need of tracking $\dot{\tilde{\boldsymbol{x}}}_1$ can be found by minimizing the following problem

$$S_1 \triangleq \left\{ \arg\min_{\dot{\boldsymbol{y}}} \left\| (\dot{\tilde{\boldsymbol{x}}}_1 - \boldsymbol{J}_1 \dot{\boldsymbol{y}}) \right\|^2 \right\}, \tag{3.45}$$

whose corresponding manifold S_1 of non-regularized solutions is

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_1 + (\boldsymbol{I} - (\boldsymbol{J}_1)^{\#} \boldsymbol{J}_1) \dot{\boldsymbol{z}}, \ \forall \dot{\boldsymbol{z}}. \tag{3.46}$$

The key mechanism in the task priority algorithm is properly exploiting any residual arbitrariness that is allowed by the manifold 3.46. Indeed, if the matrix $(\boldsymbol{I} - (\boldsymbol{J}_1)^{\#} \boldsymbol{J}_1)$ has a rank greater than 0, then it spans a subspace of $\mathbb{R}^n$ which can be further exploited by subsequent, lower priority tasks. Let us now consider a second task, for which the minimization problem can be stated as

$$S_2 \triangleq \left\{ \arg\min_{\dot{\boldsymbol{y}} \in S_1} \left\| (\dot{\tilde{\boldsymbol{x}}}_2 - \boldsymbol{J}_2 \dot{\boldsymbol{y}}) \right\|^2 \right\}, \tag{3.47}$$

where the key difference is that the manifold of solutions S_2 must now be found inside the manifold S_1 of the solutions of the first, higher priority task. To do so, let us substitute the expression of the manifold (3.46) within (3.47):

$$S_2 \triangleq \left\{ \arg\min_{\dot{\boldsymbol{z}}} \left\| (\dot{\tilde{\boldsymbol{x}}}_2 - \boldsymbol{J}_2 (\boldsymbol{J}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_1 - \boldsymbol{J}_2 (\boldsymbol{I} - (\boldsymbol{J}_1)^{\#} \boldsymbol{J}_1) \dot{\boldsymbol{z}}) \right\|^2 \right\}, \tag{3.48}$$

where now the minimization is done with respect to the arbitrary vector $\dot{\boldsymbol{z}}$. Let us now highlight a few interesting properties. First, let us defined the modified reference rate as

$$\dot{\tilde{\boldsymbol{x}}}_2 \triangleq \dot{\tilde{\boldsymbol{x}}}_2 - \boldsymbol{J}_2 (\boldsymbol{J}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_1. \tag{3.49}$$

We can notice how the reference rate for the secondary task must naturally take into account the contribution imposed by the higher priority task. Indeed $\boldsymbol{J}_2 (\boldsymbol{J}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_1$ is the effect, in the secondary task space, of the minimum norm solution $(\boldsymbol{J}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_1$ of the first task. In other terms, the second problem needs to minimize "what remains to be done" after a part of the solution has been set by the highest priority task. The second remark concerns the form of the Jacobian $\boldsymbol{J}_2 (\boldsymbol{I} - (\boldsymbol{J}_1)^{\#} \boldsymbol{J}_1)$, which represents the projection of the Jacobian $\boldsymbol{J}_2$ in the kernel of $\boldsymbol{J}_1$, hence it holds

$$\boldsymbol{J}_1 \left(\boldsymbol{J}_2 (\boldsymbol{I} - (\boldsymbol{J}_1)^{\#} \boldsymbol{J}_1) \right)^{\#} = \boldsymbol{0}. \tag{3.50}$$

Therefore, whatever the choice of $\dot{\boldsymbol{z}}$, it will not have any influence on the resulting $\dot{\boldsymbol{x}}_1$, enforcing the concept of "priority" between tasks.

Let us now proceed and let us find the expression of the manifold of solutions S_2. First, let us compute

$$\dot{\boldsymbol{z}} = (\boldsymbol{J}_2 \boldsymbol{Q}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_2 + \left(\boldsymbol{I} - (\boldsymbol{J}_2 \boldsymbol{Q}_1)^{\#} \boldsymbol{J}_2 \boldsymbol{Q}_1 \right) \dot{\boldsymbol{w}}, \ \forall \dot{\boldsymbol{w}}, \tag{3.51}$$

where, for notation compactness, we have introduced the definition $\boldsymbol{Q}_1 \triangleq (\boldsymbol{I} - (\boldsymbol{J}_1)^{\#} \boldsymbol{J}_1)$. The final expression of the manifold can be found by substituting $\dot{\boldsymbol{z}}$ within the manifold of the first task (3.46):

$$\dot{\boldsymbol{y}} = (\boldsymbol{J}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_1 + \boldsymbol{Q}_1 (\boldsymbol{J}_2 \boldsymbol{Q}_1)^{\#} \dot{\tilde{\boldsymbol{x}}}_2 + \boldsymbol{Q}_1 \left(\boldsymbol{I} - (\boldsymbol{J}_2 \boldsymbol{Q}_1)^{\#} \boldsymbol{J}_2 \boldsymbol{Q}_1 \right) \dot{\boldsymbol{w}}, \ \forall \dot{\boldsymbol{w}}, \tag{3.52}$$

The above process can be repeated for a third task, exploiting the new residual arbitrariness expressed by the vector $\dot{\boldsymbol{w}}$, and so forth. While at first glance the expression seems to get more and more complicated, luckily it can be expressed in a compact iterative form. Let us start the algorithm with the initialization

$$\boldsymbol{\rho}_0 = \boldsymbol{0}, \quad \boldsymbol{Q}_0 = \boldsymbol{I}, \tag{3.53}$$

then for $k = 1, \ldots, p$, where p is the total number of priority levels:

$$\begin{aligned} \boldsymbol{Q}_k &= \boldsymbol{Q}_{k-1} (\boldsymbol{I} - (\boldsymbol{J}_k \boldsymbol{Q}_{k-1})^{\#} \boldsymbol{J}_k \boldsymbol{Q}_{k-1}) \\ \boldsymbol{\rho}_k &= \boldsymbol{\rho}_{k-1} + \boldsymbol{Q}_{k-1} (\boldsymbol{J}_k \boldsymbol{Q}_{k-1})^{\#} \left(\dot{\tilde{\boldsymbol{x}}}_k - \boldsymbol{J}_k \boldsymbol{\rho}_{k-1} \right) \end{aligned} \tag{3.54}$$

where the final solution is $\dot{\boldsymbol{y}} = \boldsymbol{\rho}_p$.

3.8.3 Modern Task Priority Algorithms

The shortcomings of the classic task priority implementation lie mainly in the discontinuities that can be generated by a change of rank of any of the Jacobians involved. Indeed, it is intuitive to see that, as the rank of $\boldsymbol{J}_1$ diminishes by one, and its corresponding kernel increases by one, then the subsequent tasks find themselves operating in a bigger subspace, and hence the solution is likely to abruptly change. While this problem can be somehow limited by the introduction of the regularized pseudo inverse, the traditional regularization methods such as DLS or SVO do not allow for the proper introduction of a task activation/deactivation mechanism. For this reason, the original task priority was seen as a "redundancy optimization" technique: the highest priority task was considered the movement of the end-effector, and subsequent tasks were *attempted* in its null space, including safety ones. Clearly, no guarantee that safety tasks could be accomplished can be given with this priority ordering.

For the aforementioned reasons, the last decade has seen many efforts in developing novel task priority frameworks capable of properly handling inequality control objectives, allowing them to be put at a high priority level without generating discontinuities or chattering phenomena. The interested reader can see, for example, the works [**?**, **?**, **?**].

In these notes, we will exploit the approach presented in [**?**, **?**], which is based on the definition of a new regularized pseudo-inversion operator $(\boldsymbol{X})^{\#,\boldsymbol{A},\boldsymbol{Q}}$ as in the following:

$$\boldsymbol{X}^{\#,\boldsymbol{A},\boldsymbol{Q}} \triangleq \left(\boldsymbol{X}^\top \boldsymbol{A}\boldsymbol{X} + \eta(\boldsymbol{I}-\boldsymbol{Q})^\top(\boldsymbol{I}-\boldsymbol{Q}) + \boldsymbol{V}^\top \boldsymbol{P}\boldsymbol{V}\right)^{\#} \boldsymbol{X}^\top \boldsymbol{A}\boldsymbol{A} \tag{3.55}$$

where $\boldsymbol{V}$ is the right orthonormal matrix of the SVD decomposition of $\boldsymbol{X}^\top \boldsymbol{A}\boldsymbol{X} + \eta(\boldsymbol{I}-\boldsymbol{Q})^\top(\boldsymbol{I}-\boldsymbol{Q})$.

The final algorithm for the TPIK is very similar, in shape, to the original one. It is initialized with

$$\boldsymbol{\rho}_0 = \boldsymbol{0}, \quad \boldsymbol{Q}_0 = \boldsymbol{I}, \tag{3.56}$$

then for $k = 1, \ldots, p$, where p is the total number of priority levels:

$$\begin{aligned} \boldsymbol{W}_k &= \boldsymbol{J}_k \boldsymbol{Q}_{k-1} (\boldsymbol{J}_k \boldsymbol{Q}_{k-1})^{\#,\boldsymbol{A}_k,\boldsymbol{Q}_{k-1}} \\ \boldsymbol{Q}_k &= \boldsymbol{Q}_{k-1} (\boldsymbol{I} - (\boldsymbol{J}_k \boldsymbol{Q}_{k-1})^{\#,\boldsymbol{A}_k,\boldsymbol{I}} \boldsymbol{J}_k \boldsymbol{Q}_{k-1}) \\ \boldsymbol{\rho}_k &= \boldsymbol{\rho}_{k-1} + \boldsymbol{Q}_{k-1} (\boldsymbol{J}_k \boldsymbol{Q}_{k-1})^{\#,\boldsymbol{A}_k,\boldsymbol{I}} \boldsymbol{W}_k \left(\dot{\tilde{\boldsymbol{x}}}_k - \boldsymbol{J}_k \boldsymbol{\rho}_{k-1}\right) \end{aligned} \tag{3.57}$$

thus ending up with the final control law

$$\dot{\boldsymbol{y}} = \boldsymbol{\rho}_p. \tag{3.58}$$

The details of the procedure are briefly recalled in the Appendix C, while the complete development can be found in [**?**].

3.8.4 Building a Mission through Action Sequencing

Let us now highlight a few aspects of how actions can be sequenced and how they can be therefore used by a Mission Control Module:

- Usually the main difference between actions will be the objective(s) needed to achieve its particular ultimate goal and possibly some other prerequisite ones, while the safety tasks will be the same.

- Actions can be temporally sequenced accordingly with a *mission* plan represented by a decisional action graph, having the actions as nodes and the arcs as logic decision alternatives, with just a few parameters relevant to the action ultimate-goal only.

- The aforementioned two points highlight how the interface of the KCL with the Mission Control Module should therefore result simplified, allowing an easier implementation of the latter.

- Moreover, transitions from an action to another one located at the end of a selected arc, should be smoothly activated.

Table 3.1: Switching between Actions

Control Task	Previous Action $\mathcal{A}_1$	Current Action $\mathcal{A}_2$	$a^p(\boldsymbol{p})$
A	active	active	1
D_1	inactive	active	$s_{inc}(t, t_{max})$
B	active	inactive	$s_{dec}(t, t_{max})$
C	active	active	1
D_2	active	inactive	$s_{dec}(t, t_{max})$
E	inactive	active	$s_{inc}(t, t_{max})$

To the aim of satisfying the last point, consider the following two actions $\mathcal{A}_1$, $\mathcal{A}_2$, composed by objectives abstractly labelled with alphabetic letters, where $A \prec B$ denotes that A has higher priority than B and where B, C means that B has equal priority to C:

$$\begin{aligned} \mathcal{A}_1: &\quad A \prec B, C, D \\ \mathcal{A}_2: &\quad A \prec D \prec C, E \end{aligned}$$

where A, C, D are in common, but with D at a different priority ordering w.r.t. C. Now consider the following merged list:

$$\mathcal{U}: \quad A \prec D_1 \prec B, C, D_2, E;$$

where D_1 and D_2 represent the same objective D, but with a different priority. It is clear that, through insertion/deletion of some of the entries, the two original lists can be reconstructed. For example, $\mathcal{A}_1$ can be obtained by removing D_1 and E from the unified list. Conversely, $\mathcal{A}_2$ can be obtained by removing the control objectives D_2 and E.

To enable such transitions, the activation function (3.24) is modified to become

$$a(x, \boldsymbol{p}) = a^i(x) a^p(\boldsymbol{p}), \tag{3.59}$$

where $a^p(\boldsymbol{p}) \in [0, 1]$ is a continuous sigmoidal function of a vector of parameters $\boldsymbol{p}$ external to the control task itself. In particular, the $a^p(\boldsymbol{p})$ can be conveniently parametrized by the previous and current action being executed, and the time elapsed in the current action, to obtain the desired activation/deactivation smooth transition. Of course, this new activation function $a^p(\boldsymbol{p})$ applies to both to reactive and non-reactive control tasks since it does not depend on the variable $x(\boldsymbol{c})$. An example of transition from action $\mathcal{A}_1$ to $\mathcal{A}_2$ is reported in Table 3.1, where $s_{dec}(t, t_{max})$ represents a sigmoidal function similar to Fig. 3.10 whose values goes from 1 to 0 as the time t within the current action goes from 0 to the threshold time t_{max}, after which the transition between $\mathcal{A}_1$ and $\mathcal{A}_2$ is completed. Similarly, $s_{inc}(t, t_{max})$ goes from 0 to 1.

Once all the actions that the system can execute have been defined, the unified list can be constructed. Many safety tasks will be common to all actions, and they will be at the same (high) priority level in each of them, therefore for such tasks it will result that $a^p(\boldsymbol{p}) \equiv 1$. Conversely, $a^p(\boldsymbol{p})$ will be mostly exploited to activate/deactivate action-defining objectives and prerequisite ones.

3.8.5 Remarks on Conflicting Objectives

Until now, no remarks on the feasibility of a hierarchy of control objectives and corresponding tasks was made. An action is concluded successfully when its *action defining* objectives are satisfied. For example, a grasping action $\mathcal{A}_g$ is concluded when the end-effector is in the desired final position and the gripper has been closed. However, the action-defining objectives are usually at a low priority in the hierarchy, as objectives that deal with safety considerations or that represent necessary preconditions (e.g. centering the camera on the object to be grasped) need to have higher priority.

Therefore, a necessary condition to guarantee that an action can be terminated is the existence of a *non-void* set of configurations, to be eventually reached by the system, where all high priority and action-defining objectives are *contemporaneously* satisfied. Only control objectives that are lower priority than the action-defining one(s) can remain unsatisfied (e.g. maintaining a preferred shape), since they usually express a *preference*.

The presence of such a *non-conflicting zone* is, however, not a sufficient condition, as it does not imply its reachability in correspondence of all system initial configurations starting outside it. A classical example regards navigation in presence of obstacles, where it is well known that reactive control strategies might get stuck in local minima. Avoidance of the obstacle is guaranteed, but convergence to the final desired position is not. Such a reachability problem affects all the task priority strategies, and in general all reactive control approaches.

In the proposed architecture, this problem needs to be addressed at a higher level, within the Mission Control Module block. The creation of different control actions helps in defining a set of well structured blocks, that can be sequenced by the Mission Control Module. Furthermore, the KCL should continuously monitor its objectives, and eventually report to the higher level the inability of completing the required action (e.g. since it is stuck in a local minima). At that point, the Mission Control Module job would be to devise an alternative solution, e.g. a different sequencing of actions or different action parameters (i.e. a new path to follow). A detailed discussion of how the Mission Control Module could be implemented falls outside the scope of the current work.

3.8.6 Control in Presence of Vehicle Underactuations

Until now the vehicle was assumed fully actuated in all its degrees of freedom. In any case, the proposed architecture can also be used for managing underactuated vehicles at the best extent possible. Consider an example where the vehicle does not have thrusters allowing to control the angular velocities along the vehicle x and y axes (roll and pitch), which is often the case for small size vehicles. Then, consider the following *non-reactive* control task where:

- Its reference rate $\dot{\tilde{\boldsymbol{x}}}$ is composed by the vector of the *measured* vehicle velocities (all vehicle velocities, including the non actuated ones, are assumed measurable). Therefore, for such a task $\dot{\tilde{\boldsymbol{x}}} = \begin{bmatrix} \boldsymbol{v}_1^\top & \omega_x & \omega_y & \omega_z \end{bmatrix}^\top$, where $\boldsymbol{v}_2$ has been explicitly written in terms of its components;
- Its Jacobian is simply $\begin{bmatrix} \boldsymbol{0}_{6\times l} & \boldsymbol{I}_{6\times 6} \end{bmatrix}$, where the first block of zeroes regards the arm joint velocities that are kinematically non-influencing the vehicle ones;
- Its activation function, in the considered roll and pitch underactuation example, is the constant diagonal matrix $\mathrm{diag}(0,0,0,1,1,0)$ selecting the non-actuated velocities only.

Then, provided such a task is located at the *top of the hierarchy*, the following facts hold true:

- The vehicle angular velocity components of $\boldsymbol{\rho}_1$, formally evaluated as if all vehicle velocities were actuated, will therefore be returned, by the procedure itself, just coinciding with the measured ones, i.e. $\boldsymbol{\rho}_1 = \begin{bmatrix} \boldsymbol{0}_{1\times l} & \boldsymbol{0}_{1\times 3} & \omega_x & \omega_y & 0 \end{bmatrix}^\top$.
- The matrix $\boldsymbol{Q}_1$, which for a highest priority task with constant activation matrix, such as the proposed one, reduces to be the orthogonal projector $(\boldsymbol{I} - \boldsymbol{J}_1^{\#}\boldsymbol{J}_1)$, will prevent all the lower priority tasks from interfering with the highest priority one, effectively inhibiting them from changing the values of $\boldsymbol{\rho}_k$ in the x and y angular velocities components.
- Each lower priority task will take into account the current angular velocities ω_x and ω_y of the vehicle, during the computation of their task references $\left(\dot{\tilde{\boldsymbol{x}}}_k - \boldsymbol{J}_k\boldsymbol{\rho}_{k-1}\right)$. Therefore, the final control vector is necessarily the best that the procedure can provide, given that some vehicle velocities are non actuated, but measured.

At the end of the procedure, the values of $\dot{\tilde{\boldsymbol{y}}}$ corresponding to the non-actuated degrees of freedom are then trivially discarded, while the remaining (actuated) ones are instead sent to the DCL to be tracked. In general, with a proper choice of the activation function, the proposed architecture can actually cope with any kind of vehicle underactuation; thus, possibly, even underactuation caused by actuators failures, provided they are detected and the vehicle velocities remain all measurable.

Naturally, the remarks made in section 3.8.5 are more demanding if the vehicle is underactuated. For example, centering a vehicle-fixed camera on the target might be more difficult as arm movements would move the center of mass and therefore create a different passively stable point in roll and pitch, with possibly negative consequences on target camera centering. Also, the assumed underactuated

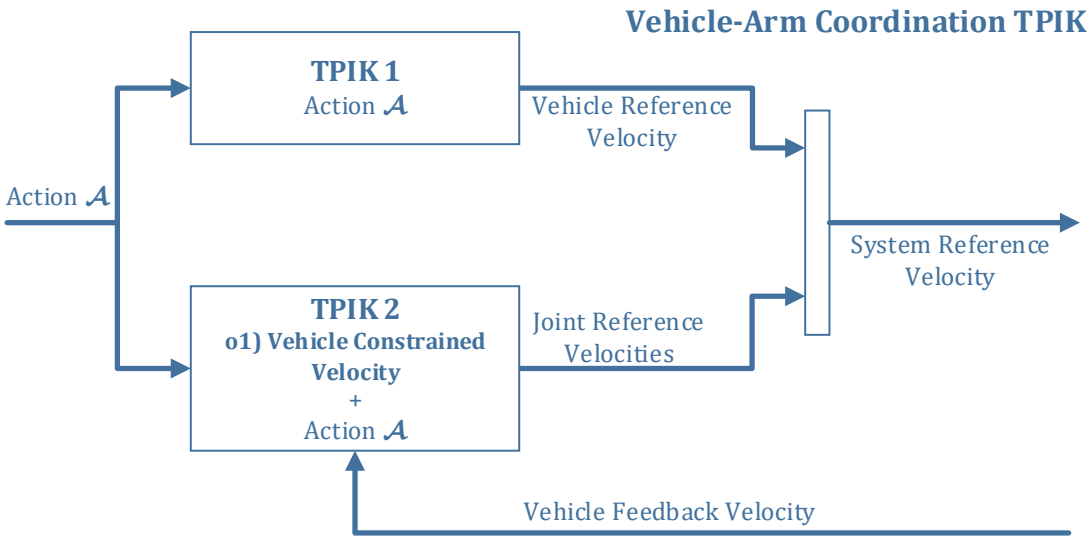


Figure 3.11: A detailed view of the Kinematic Control Layer, with the two Task Priority Inverse Kinematics blocks highlighted, performing the vehicle velocity tracking compensation scheme.

pitch velocity could obviously create risks of having the pitch angle getting close to its upper limit of 90 degrees.

In any case, for reducing the risks of the above inconveniences, nothing prevents the addition of a further objective aimed at maintaining the overall system center of mass substantially aligned under the center of buoyancy, along the z-axis *of the vehicle frame*.

Summing up, the above discussion has briefly outlined how the proposed control architecture is able to manage, at the best extent possible, the existence of vehicle underactuations, even when caused by occasional actuators failures, via a suitable structuring of its objectives and tasks.

3.8.7 Arm-Vehicle Coordination Scheme

An important feature of the proposed architecture, relevant when the UVMS needs to perform *floating* manipulative operations, is that the effects of the vehicle velocity tracking inaccuracy are taken into account. As it is well known, due to the prominent nonlinear properties of thrusters [**?**, **?**, **?**], the velocity control of the vehicle results worse than the one for the manipulator's joints. Furthermore, the lower ratios of force/mass for the vehicle than for the manipulator exacerbates this issue.

If manipulator and vehicle motions are not kinematically decoupled, then disturbances of the floating base would propagate through the coupled kinematics immediately to the end effector of the manipulator. The idea of decoupling the two subsystems is well known in ground mobile manipulation, where delayed or imprecise wheel actuation has similar effects to those of the thrusters, and some strategies were proposed in [**?**] and successively in [**?**]. However, these approaches are not framed within a task priority approach, and therefore the mobile base does not support the manipulator in achieving all its control objectives such as joint limits or obstacle avoidance.

To cope with this problem, the idea is to have two optimizations (C.16) running in parallel as depicted in Fig. 3.11. Given an action $\mathcal{A}$ to be executed, the parallel scheme which coordinates the vehicle and arm movements works as follows:

1. The first optimization (TPIK 1) considers the vehicle together with the manipulator, as a fully controllable system. The hierarchy of tasks corresponding to action $\mathcal{A}$ is solved as outlined in section 3.8.3, and of the whole result $\dot{\boldsymbol{y}}$, only the vehicle reference velocity is used, while the manipulator part is discarded. If the vehicle is underactuated, the additional non-reactive control task presented in section 3.8.6 is placed at the top of the task hierarchy;

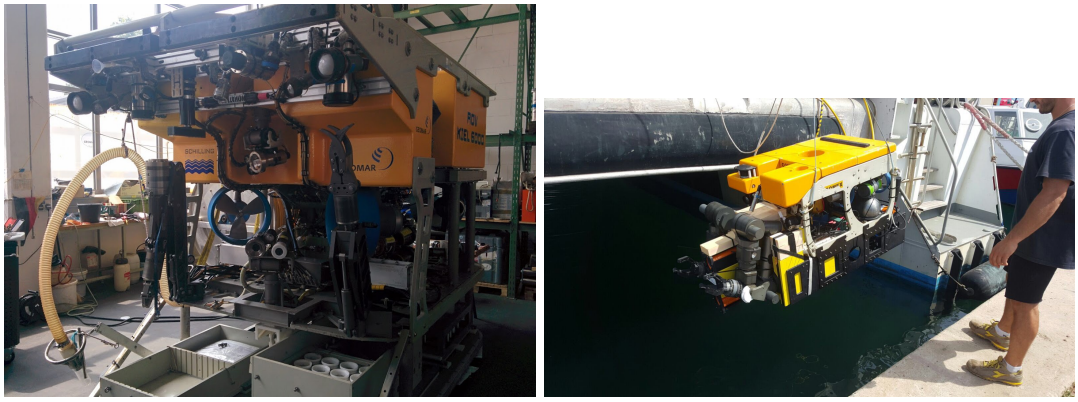


Figure 3.12: (a) An example of dual arm ROV (photo courtesy of GEOMAR) (b) dual arm hybrid ROV-AUV used in the DexROV project

2. The second optimization (TPIK 2) considers the vehicle as *totally non-controllable*. Therefore, the non-reactive control task presented in 3.8.6 is always used and all the vehicle degrees of freedom are initialized with the corresponding *measured* velocities. Therefore, the vehicle constrained velocity task is placed on top of the task hierarchy of action $\mathcal{A}$, as depicted in the TPIK 2 block of Fig. 3.11.

   In such an optimization, de-facto only the manipulator variables are subject to be optimized. In fact, the outputs of this procedure are the *optimal* joint velocities in correspondence of the *measured* vehicle velocity.

The two outputs are then forwarded to the corresponding parts of the DCL to be tracked.

Therefore, thanks to the TPIK 2 optimization, the arm joint velocities are always the optimal ones based on the current vehicle velocities, independently of any vehicle inaccuracy in tracking the desired ones generated by TPIK 1. As a remark, note that if the vehicle were tracking its velocities perfectly, the joint velocities output of TPIK 2 would be exactly the same as the one discarded in TPIK 1.

3.8.8 Arm-Vehicle Multi-rate Control

The parallel strategy for arm-vehicle coordinated control is also suitable for implementing multi-rate control of the two subsystems. For example, let us consider the control rates of 10 Hz for the vehicle and 100 Hz for the manipulator:

1. At time $t = 0s$, both TPIK 1 and 2 will be executed, and the corresponding vehicle and joint velocities will be generated and sent to the DCLs.
2. At $t = 0.01s$, TPIK 2 will run and generate new reference velocities for the manipulator, taking into account any new feedback (joint positions, vehicle velocity, etc.), while the reference velocity for the vehicle will be kept constant. (i.e. the one generated in point 1 of this procedure).
3. The above step is repeated until the period for the vehicle control has elapsed (i.e. 100 ms).
4. At $t = 0.1s$ both TPIKs will be run and everything will proceed from point 1 again.

3.9 Multi Arm UVMS Control

Let us consider a multi arm system, such as the one represented in Fig. 3.13, where for simplicity of discussion we are limiting the number of arms to two. The extension of the task priority approach seen until now to this case is quite straightforward.

First of all, let us re-define the main variables

$$\boldsymbol{c} \triangleq \begin{bmatrix} \boldsymbol{q}_a \\ \boldsymbol{q}_b \\ \boldsymbol{\eta} \end{bmatrix}, \tag{3.60}$$

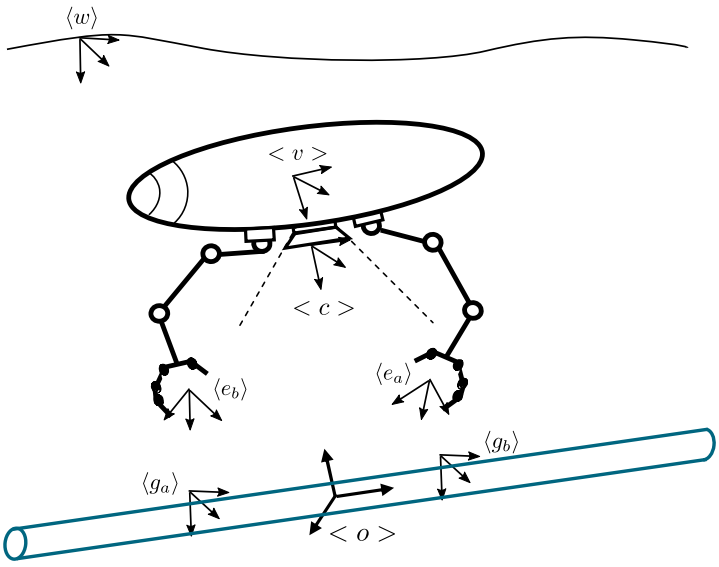


Figure 3.13: Relevant frames for a dual arm UVMS: $\langle w \rangle$ world inertial frame, $\langle v \rangle$ vehicle frame, $\langle c \rangle$ camera frame, $\langle o \rangle$ object frame, $\langle e_a \rangle$ end-effector frame for arm a, $\langle e_b \rangle$ end-effector frame for arm b, $\langle g_a \rangle$ goal frame for arm a, $\langle g_b \rangle$ goal for arm b

where $\boldsymbol{q}_a \in \mathbb{R}^{l_a}$ represents the DOFs of arm a, while $\boldsymbol{q}_b \in \mathbb{R}^{l_b}$ does the same for arm b. Similarly, the control vector becomes

$$\dot{\boldsymbol{y}} = \begin{bmatrix} \dot{\boldsymbol{q}}_a \\ \dot{\boldsymbol{q}}_b \\ \boldsymbol{\nu} \end{bmatrix}. \tag{3.61}$$

Now, some tasks like the manipulability one are defined both for manipulator a and b. Since there is no reason to have one at an higher priority w.r.t. the other one, they can be simply stacked, resulting in the following multidimensional task:

$$\begin{bmatrix} \dot{\tilde{x}}_{\mu_a} \\ \dot{\tilde{x}}_{\mu_b} \end{bmatrix} = \begin{bmatrix} \boldsymbol{J}_{\mu_a,a} & 0 & 0 \\ 0 & \boldsymbol{J}_{\mu_b,b} & 0 \end{bmatrix} \dot{\boldsymbol{y}}, \tag{3.62}$$

where the notation $\boldsymbol{J}_{x,y}$ indicates the sole contribution of the subsystem y to the Jacobian of task x.

The same procedure can be applied to other tasks such as joint limits, camera occlusions, etc. For example, for the end-effector approach we have:

$$\begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_{ee_a} \\ \dot{\tilde{\boldsymbol{x}}}_{ee_b} \end{bmatrix} = \begin{bmatrix} \boldsymbol{J}_{ee_a,a} & 0 & \boldsymbol{J}_{ee_a,v} \\ 0 & \boldsymbol{J}_{ee_b,b} & \boldsymbol{J}_{ee_b,v} \end{bmatrix} \dot{\boldsymbol{y}}. \tag{3.63}$$

In the above equation, the vehicle provides different rigid body contributions to the two end-effectors, as two different frames are considered.

The introduction of a second arm however creates the possibility of manipulating the *same* object. Let us consider the case where the two arms have firmly grasped the object. Furthermore, let us now consider the control point of each end-effector prolonged to the object frame $\langle o \rangle$. Then, the following equation holds

$$\boldsymbol{J}_{o,a}\dot{\boldsymbol{q}}_a + \cancel{\boldsymbol{J}_{o,v}\boldsymbol{\nu}} = \boldsymbol{J}_{o,b}\dot{\boldsymbol{q}}_b + \cancel{\boldsymbol{J}_{o,v}\boldsymbol{\nu}}, \tag{3.64}$$

where the rigid-body contribution of the vehicle is of course the same since a single frame is now considered. The equation can be rewritten as

$$\begin{bmatrix} \boldsymbol{J}_{o,a} & -\boldsymbol{J}_{o,b} & \boldsymbol{0} \end{bmatrix} \dot{\boldsymbol{y}} = 0, \tag{3.65}$$

which is the *kinematic constraint* imposed by the object, assuming a firm grasp by both end-effectors. Since the velocity of the rigid body (object) is uniquely defined, the constraint simply state that the

velocity of the two end-effectors, once considered rigidly translated to the same point (this is embedded in the definition of the Jacobians), must be equal. By defining

$$\boldsymbol{J}_{okc} \triangleq \begin{bmatrix} \boldsymbol{J}_{o,a} & -\boldsymbol{J}_{o,b} & \mathbf{0} \end{bmatrix}, \tag{3.66}$$

then the object kinematic constraint requires that $\dot{\boldsymbol{y}} \in N(\boldsymbol{J}_{okc})$. Since this constraint is enforced by the firm grasp assumption, there is no reason for the control to generate a reference velocity that does not lie in the null space of $\boldsymbol{J}_{okc}$. Following this reasoning, a new control task arises. The object kinematic constraint task can be simply defined as the requirement of having

$$\mathbf{0} = \boldsymbol{J}_{okc}\dot{\boldsymbol{y}} \tag{3.67}$$

and is an example of the *physical constraint* type of task. For this reason, it should be placed at the top of the hierarchy. It is easy to see that this task can be always guaranteed when it is the highest priority one.

3.10 Exploiting the Task Priority Approach: Actions and Possible Applications

This sections highlights different operative scenarios where the control of the UVMS can take advantage of the proposed architecture, showing its flexibility in being adapted to many applications. In the following, the actions to carry out the specific operations will be presented using the following compact notation:

- [R/NR, I/E, C/S/P/AD/O] Name of the task/objective;

where

- R/NR specifies if the task is reactive or non-reactive;
- I/E specifies if the task is of inequality or equality type;
- C/S/P/AD/O specifies if the category of the task, i.e. constraint, safety, prerequisite, action-defining, optimization.

Finally, all the actions that will be presented in the following sections can be thought as having the underactuation constraint task, presented in section 3.8.6, at the top of the hierarchy, i.e.

- [NR, E, C] Vehicle underactuation constraint.

3.10.1 Safe Waypoint Navigation

Let us consider the requirement of having the UVMS reaching a new position, expressed for example in global coordinates through an ultra-short or long baseline system. In such a scenario, other than having the vehicle in the required position, there are a number of other objectives that the system needs to satisfy, which can be summarized in the following list (highest to lowest priority objective):

1. [R, I, S] Vehicle minimum altitude;
2. [R, I, S] Vehicle obstacle avoidance;
3. [R, I, S] Vehicle horizontal attitude;
4. [R, I, P] Vehicle auto-heading;
5. [R, I, AD] Vehicle position (x, y and depth);

where all the objectives are of inequality type, since for floating vehicles attaining a precise positioning would require excessive energy consumption. The altitude task and obstacle avoidance one are written separately just because they usually work with different sensors, i.e. an altimeter and a forward looking sonar.

The above defined navigation to a waypoint action allows us to make a few important remarks:

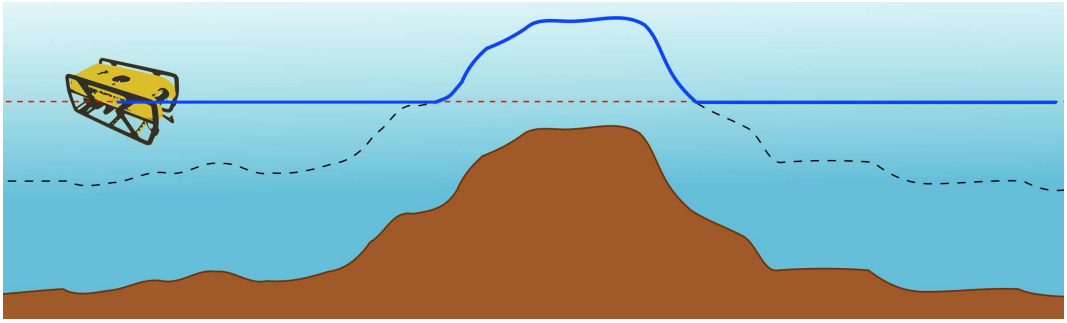


Figure 3.14: Desired behaviour of the safe waypoint navigation action: (brown solid) sea-floor depth, (black-dashed) minimum altitude, (red-dashed) depth desired value, (blue) resulting behaviour of the task priority approach, considering the higher priority altitude objective and the depth one.

- There is a natural conflict between certain objectives. In the above example, the minimum altitude might conflict with the auto-depth objective, which is implicitly considered in the vehicle position one. However, the altitude objective is activated *only* when really necessary.
- The specified higher priority of the altitude task allows to ensure that the altitude is maintained at the cost of the depth setpoint *whenever necessary*. The resulting task-priority behaviour is depicted in Fig. 3.14.
- Thanks to the possibility of activating and deactivating tasks, the priority of the objectives can follow a natural order without over-constraining the system: safety objectives first, then operational enabling objectives (auto-heading), then actual action-defining objectives (vehicle position).

3.10.2 Assisted Teleoperation of UVMS Degrees of Freedom

The flexibility of the proposed control framework also allows to implement an assisted teleoperation of the UVMS, in a sort of advanced ROV operational mode. Typically, the ROV operator controls the vehicle through body frame velocities, i.e. using the surge, sway, heave and angular velocity commands, while he/she sets the desired joint velocities for the manipulator. This means that the system velocity vector $\dot{\boldsymbol{y}}$ is exactly the variable that contains the desired inputs for the operator. Therefore, a non-reactive control task for the manual control can be easily defined:

- The Jacobian relationship is simply the identity matrix;
- The activation function is set to one in correspondence of the degrees of freedom that need to be manually controlled, zero otherwise;
- The reference rate vector $\dot{\tilde{\boldsymbol{x}}}$ is initialized with the values generated by the operator's console.

A control task so defined, placed at the top of the control task hierarchy, will directly set the corresponding control vector to the desired one provided by the operator. It is important to note that if only some of the degrees of freedom are activated for manual control, all the others can be used by the control system for the achievement of the remaining control objectives. Furthermore, the task priority approach will automatically take into account the contribution of the manually controlled degrees of freedom to the remaining control tasks.

To clarify, let us consider the following example action of safe navigation with manual/assisted teleoperation, characterized by the following list of tasks:

1. [NR, E, AD] Manual control;
2. [R, I, S] Vehicle minimum altitude;
3. [R, I, S] Vehicle obstacle avoidance;

4. [R, I, S] Vehicle horizontal attitude;
5. [R, I, P] Vehicle auto-heading;
6. [R, I, AD] Vehicle position (x, y and depth);

which is the same one as the previous section, with the addition of the non-reactive "Manual control" task.

Let us suppose that the operator configures the manual control task, through a proper choice of the activation function, to take control of surge, sway and angular velocity along the z axis, leaving the degrees of freedom of heave, and the rotation along the x and y axes to the control system. In such a configuration, if the operator moves the vehicle with a certain surge velocity, and the pitch of the vehicle points slightly down, the depth will increase and altitude decrease. However, the control system, through the vehicle position depth objective, will automatically regulate the heave motion to compensate for the possible increase in depth. Finally, the altitude control objective will prevent the operator for getting too close to the seafloor, at the expenses of the depth regulation task, accordingly with the desired priority and safety requirements.

3.10.3 Grasping

Another action that can be defined with the proposed architecture regards the grasping of an object. In this case, the action involves also the manipulator, therefore the number of control objectives is naturally higher than in the previous examples. The following hierarchy could be used to define such an action:

1. [R, I, S] Vehicle minimum altitude;
2. [R, I, S] Vehicle obstacle avoidance;
3. [R, I, S] Arm joint limits;
4. [R, I, P] Arm manipulability;
5. [R, I, P] Camera centering;
6. [R, I, P] Camera-arm occlusions;
7. [R, E, AD] End-effector linear position control;
8. [R, E, AD] End-effector angular position control;
9. [R, I, O] Arm preferred shape;
10. [NR, E, O] Vehicle motion minimization;

where the need of avoiding occlusions between the manipulator and the camera system providing the feedback as well as the need of maintaining the object in the camera frame are clearly prerequisites for guiding the end-effector on top of the object, therefore they have higher priority. Finally, it might be desirable to move the vehicle as little as possible, for the same considerations made in section 3.8.7, hence the final motion minimization task.

3.10.4 Assisted End-Effector Teleoperation

A further advanced assisted ROV application consists in considering a teleoperation of the end-effector of the manipulator, rather than a set of degrees of freedom of the UVMS, as done in section 3.10.2.

In this scenario, the operator generates reference velocities for the end-effector, and the control system assists him in a set of prerequisite and safety objectives. This can lead to the creation of an action similar to the one presented in section 3.10.3:

1. [R, I, S] Vehicle minimum altitude;
2. [R, I, S] Vehicle obstacle avoidance;

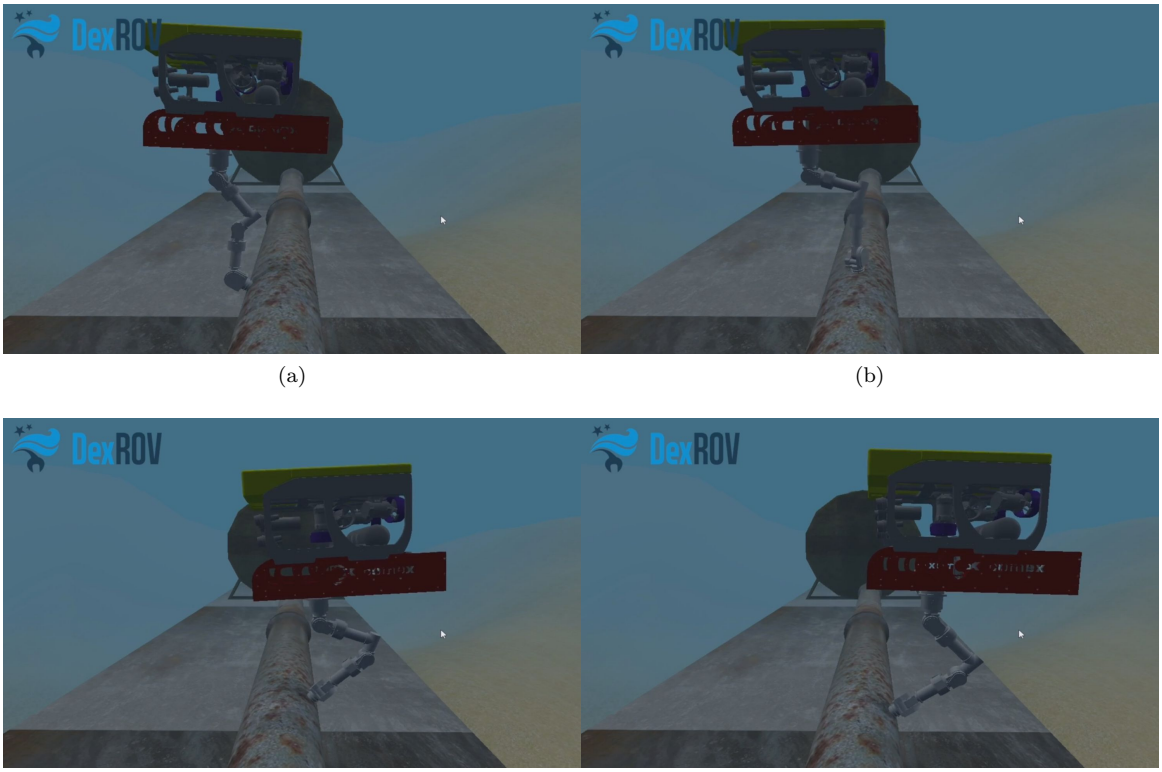


Figure 3.15: Screenshots of the UVMS as it performs the pipe welding inspection task.

3. [R, I, S] Arm joint limits;
4. [R, I, P] Arm manipulability;
5. [R, I, P] Camera centering (if visual feedback is available);
6. [R, I, P] Camera-arm occlusions (if visual feedback is available);
7. [NR, E, AD] End-effector linear velocity teleoperation;
8. [NR, E, AD] End-effector angular velocity teleoperation;
9. [R, I, O] Arm preferred shape;
10. [NR, E, O] Vehicle motion minimization;

In this case, the camera objectives are included only if an acoustic/vision feedback is available, since it is not anymore stricly necessary as in the fully autonomous operation of section 3.10.3. Let us remark how the control loop to do the manipulative operation is now "closed" by the human operator, hence the end-effector tasks are of non-reactive type.

3.10.5 Inspection

Another interesting action that can be defined within the proposed architecture stems from one of the reference missions of the DexROV project and regards the inspection of a pipeline. An electromagnetic sensor placed at the end-effector needs to be put in contact with the pipeline's weld and follow it in order to discover any possible cracks or leaks.

This application is very interesting since it requires interaction between the UVMS and the environment, and allows us to show how the proposed task priority framework can enforce kinematic constraints, as well as, despite being at kinematic level, manage interaction tasks.

Let us consider a surface, characterized by its normal $\boldsymbol{n} \in \mathbb{R}^3$, and let us call $\boldsymbol{J}_{el} \in \mathbb{R}^{3\times n}$ the total linear Jacobian of the end-effector frame, comprehensive of both the vehicle and the manipulator contributions. For a moment, let us consider the end-effector to be bilaterally constrained on the surface, then the following holds

$$0 = \boldsymbol{n}^\top \boldsymbol{J}_{el} \dot{\boldsymbol{y}}. \tag{3.68}$$

If the control vector $\dot{\boldsymbol{y}}$ is such that (3.68) would be violated, a force is exerted on the surface, and the actual $\dot{\boldsymbol{y}}$ will not converge to $\dot{\bar{\boldsymbol{y}}}$ due to the constraint and consequently the tracking of the control task reference rates cannot be anymore guaranteed.

For the aforementioned reasons, the control algorithm should take the constraint into account at the highest priority. Therefore, a *reactive* control task of the following kind can be placed at the top of the hierarchy:

$$S_1 \triangleq \left\{ \arg\mathrm{R\text{-}}\min_{\dot{\boldsymbol{y}}} \left\| (0 - \boldsymbol{n}^\top \boldsymbol{J}_{el} \dot{\boldsymbol{y}}) \right\|^2 \right\}. \tag{3.69}$$

Introducing the above task will make the subsequent ones avoiding generating system velocities in the direction of the constraint, which would not be trackable by the DCL. In case of perfect tracking, no interaction with the environment will occur. Note that this is actually a reactive control task, since the control objective corresponds to maintaining the geometric constraint which is guaranteed by the assumption made at the start of the section, hence the reference is always zero.

Let us now relax the assumption, and consider the actual *unilateral* constraint. In this case, an interaction with the environment is desired to maintain contact with the surface through force regulation. To that aim, the idea is to impose a value $\dot{\bar{x}}_f$ rather than 0 as task reference of (3.69), obtaining

$$S_1 \triangleq \left\{ \arg\mathrm{R\text{-}}\min_{\dot{\boldsymbol{y}}} \left\| (\dot{\bar{x}}_f - \boldsymbol{n}^\top \boldsymbol{J}_{el} \dot{\boldsymbol{y}}) \right\|^2 \right\}, \tag{3.70}$$

where $\dot{\bar{x}}_f$ can be used to implement any force control scheme reported at velocity level, see for example [**?**, **?**, **?**].

Following the aforementioned requirement of having interaction tasks at the top of the task hierarchy, the pipeline inspection action results to be:

1. [R, E, C] Constraint complying and force regulation;
2. [R, I, S] Arm joint limits;
3. [R, I, S] Arm manipulability;
4. [R, I, S] Vehicle horizontal attitude;
5. [R, I, P] End-effector alignment with the surface's normal;
6. [R, E, AD] End-effector linear position tracking;
7. [R, E, AD] End-effector angular position tracking;
8. [R, I, O] Arm preferred shape.

To carry out the inspection of the weld, a reference path has been defined in the horizontal plane, in correspondence of the projection of the weld. Therefore, there is no knowledge of the actual shape or radius of the pipe. The expected result is that the end-effector follows the path projected on the pipe's surface, adapting to the unknown surface. A frictionless multi-point contact between the end-effector and the pipe surface was simulated. All the simulated forces and moments have been transferred to a unique point on the end-effector's rigid body space, where the presence of force/torque sensor has been assumed.

Screenshots of the UVMS performing the inspection task are reported in Fig. 3.15 The results are summarized in Fig. 3.16, where in particular Fig. 3.16a shows how the force is regulated close to the desired value of 10 N, while following the path on the pipe with a maximum deviation of approximately 3 cm, as shown in Fig. 3.16f. The above results are achieved notwithstanding the activation of different safety tasks such as joint limits (see Fig. 3.16d) and prerequisite ones such as manipulability (see Fig. 3.16e). Finally, the generated system velocity references take into account the saturations as it can be clearly appreciated in both Fig. 3.16b and Fig. 3.16c.

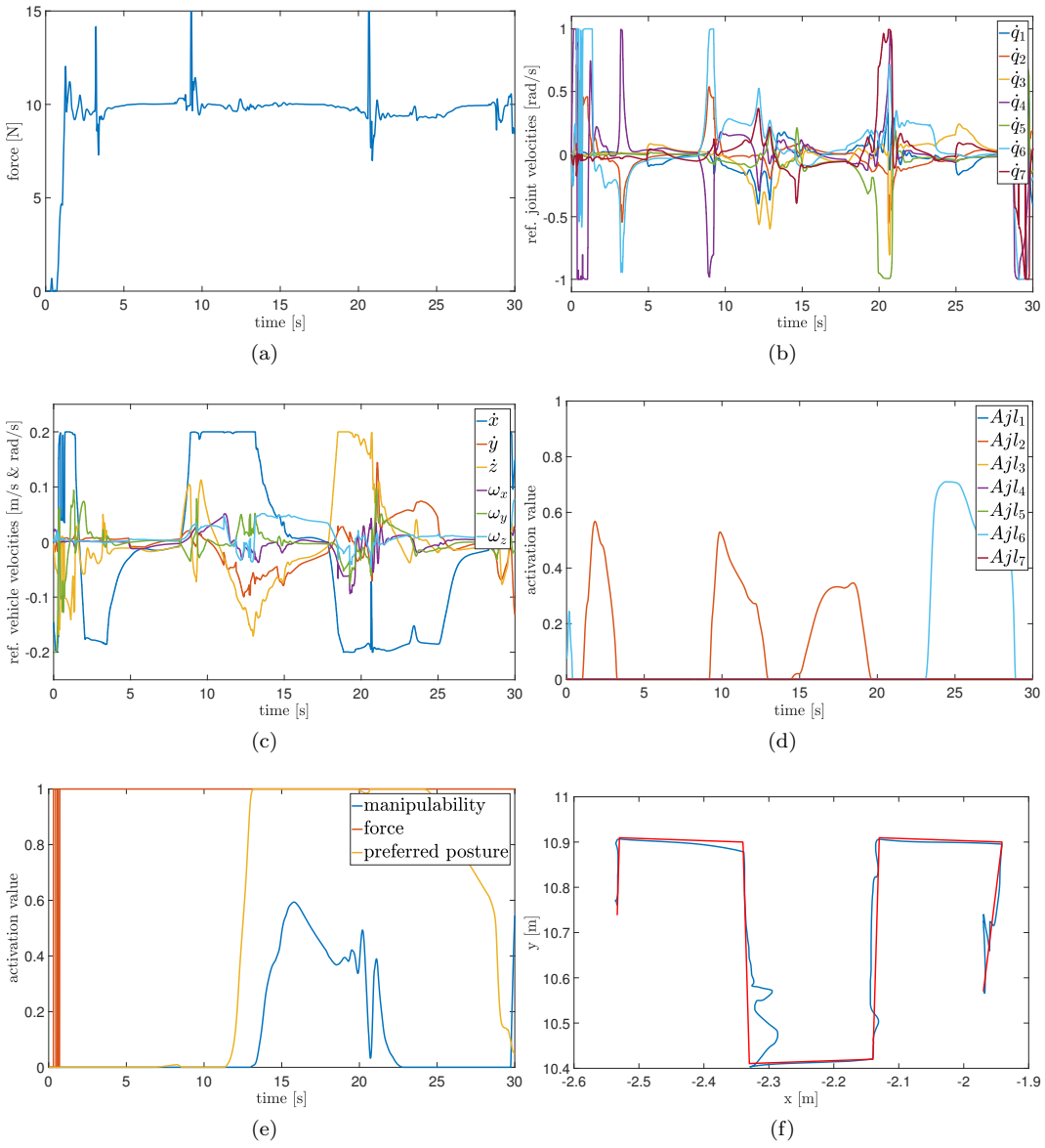


Figure 3.16: Pipeline inspection action simulation results: (a) the time behaviour of the norm of the force (reference value is 10 N), (b) the arm joint velocities (saturation value at 1 rad/s) (c) vehicle reference velocities (saturation value at 0.2 rad/s and 0.2 m/s) (d) the activation values of the joint limit tasks, (e) the activation values of the other tasks, (f) reference path (red) and actual trajectory (blue).

3.10.6 Landing

Finally, a landing action is considered. This action is the result of the initial investigations of the ROBUST project [**?**]. The AUV needs to land in front of a manganese nodule, to perform an in-situ measurement of the nodule itself and identify the presence of possible Rare Earth Elements deposits in the seafloor. The landing is achieved using the following hierarchy of objectives:

1. [R, I, S] Vehicle horizontal attitude;
2. [R, I, P] Vehicle longitudinal alignment to the nodule;
3. [R, I, P] Vehicle distance to the nodule;
4. [R, E, AD] Vehicle altitude.

After the successful landing, the in-situ measurement is executed through the arm manipulation action presented in section 3.5.

3.10.7 Case Study Examples

3.10.7.1 Aligning to a Target

Let us consider the following problem. We want to align the longitudinal axis of the vehicle (the x-axis $\boldsymbol{i}_v$) so that the vehicle faces a given target P. Let us assume that we can compute the distance vector

$$\boldsymbol{d} \triangleq P - O_v \tag{3.71}$$

between the origin of the vehicle frame $\langle v \rangle$ and the point P. The problem is thus solved if the x-axis of the vehicle is aligned to the unit vector $\boldsymbol{n}_d \triangleq \boldsymbol{d}/\|\boldsymbol{d}\|$ of $\boldsymbol{d}$. Hence, let us consider the misalignment vector (which can be computed with the reduced versor lemma)

$$\boldsymbol{\rho} = \theta \boldsymbol{n} = \frac{\theta}{\sin\theta} \boldsymbol{i}_v \wedge \boldsymbol{n}_d \tag{3.72}$$

between the unit vector $\boldsymbol{i}_v$ and the unit vector $\boldsymbol{n}_d$. To solve the problem we will need to create a task which is able to bring $\boldsymbol{\rho}$ to zero. Among many possible choices, one is require the derivative to be proportional to the misalignment vector itself, i.e.

$$\dot{\boldsymbol{\rho}} \triangleq -\lambda \boldsymbol{\rho} = -\lambda \theta \boldsymbol{n}, \lambda > 0. \tag{3.73}$$

If the true derivative $D\boldsymbol{\rho}$ follows the desired value, this would guarantee an asymptotic convergence of $\boldsymbol{\rho} \to 0$

To do so, we need to find out the relationship between how $\boldsymbol{\rho}$ evolves in time and our system velocity vector $\dot{\boldsymbol{y}}$ which is the collection of the variables which we can act upon. The derivative of the vector $\boldsymbol{\rho}$ with respect to an observer α is given by (2.95) (with $\boldsymbol{a} \triangleq \boldsymbol{i}_v$ and $\boldsymbol{b} \triangleq \boldsymbol{n}_d$)

$$D_\alpha \boldsymbol{\rho} = \boldsymbol{n}\dot{\theta} + \theta D_\alpha \boldsymbol{n} = \boldsymbol{n}\boldsymbol{n} \bullet \boldsymbol{w}_{n_d/i_v} + \theta \boldsymbol{N}(\theta) \boldsymbol{w}_{n_d/\alpha} - \theta \boldsymbol{M}(\theta) \boldsymbol{w}_{i_v/\alpha}.$$

Note that, if we consider our observer on one of the two vectors the equation simplifies. For example, considering the derivative with respect to frame $\langle v \rangle$, which is the same as an observer on vector $\boldsymbol{i}_v$, we obtain:

$$D_v \boldsymbol{\rho} = \boldsymbol{n}\boldsymbol{n} \bullet \boldsymbol{w}_{n_d/i_v} + \theta \boldsymbol{N}(\theta) \boldsymbol{w}_{n_d/i_v}. \tag{3.74}$$

We cannot stop here, as $\boldsymbol{w}_{n_d/i_v}$ needs to be expressed in terms of the actual control variable (within $\dot{\boldsymbol{y}}$). To this aim, let us express $\boldsymbol{w}_{n_d/i_v}$ as

$$\boldsymbol{w}_{n_d/i_v} = \boldsymbol{w}_{n_d/w} - \boldsymbol{w}_{i_v/w}. \tag{3.75}$$

The second term $\boldsymbol{w}_{i_v/w}$ is the angular velocity of the x-axis of the frame $\langle v \rangle$. Looking at the class of angular velocity vectors for a vector (2.69), it is clear that the angular velocity of the frame $\langle v \rangle$ belongs to them, hence we can use $\boldsymbol{w}_{i_v/w} = \boldsymbol{\omega}_{v/w}$, which is one of our control variables.

Concerning $\boldsymbol{w}_{n_d/w}$, we know that the tip of the vector $\boldsymbol{d}$ moves with a velocity which satisfies

$$\boldsymbol{v}_{d/w} = \boldsymbol{w}_{d/w} \wedge \boldsymbol{d} \tag{3.76}$$

We can solve the inverse cross product problem, finding out $\boldsymbol{w}_{d/w}$, which is a vector orthogonal to both $\boldsymbol{v}_{d/w}$ and $\boldsymbol{d}$, whose length is equal to $\|\boldsymbol{v}_{d/w}\|/\|\boldsymbol{d}\|$, hence

$$\boldsymbol{w}_{n_d/w} \triangleq \boldsymbol{w}_{d/w} = \left(\frac{\boldsymbol{d}}{\|\boldsymbol{d}\|} \wedge \frac{\boldsymbol{v}_{d/w}}{\|\boldsymbol{v}_{d/w}\|} \right) \frac{\|\boldsymbol{v}_{d/w}\|}{\|\boldsymbol{d}\|} = \left(\boldsymbol{d} \wedge \boldsymbol{v}_{d/w} \right) \frac{1}{\|\boldsymbol{d}\|^2} \tag{3.77}$$

The velocity $\boldsymbol{v}_{d/w}$ can be computed from the definition of $\boldsymbol{d}$

$$\boldsymbol{v}_{d/w} = \boldsymbol{v}_{P/w} - \boldsymbol{v}_{v/w} \tag{3.78}$$

where we recognize $\boldsymbol{v}_{v/w}$ as one of our control variables.

To simplify the analysis, let us consider the point P as fixed in the space, hence $\boldsymbol{v}_{P/w} = 0$. We have that

$$\boldsymbol{w}_{n_d/w} = \left(\boldsymbol{d} \wedge \boldsymbol{v}_{d/w} \right) \frac{1}{\|\boldsymbol{d}\|^2} = -\frac{1}{\|\boldsymbol{d}\|^2} \left(\boldsymbol{d} \wedge \boldsymbol{v}_{v/w} \right), \tag{3.79}$$

which gives us the relationship

$$\boldsymbol{w}_{n_d/i_v} = -\frac{1}{\|\boldsymbol{d}\|^2} \left(\boldsymbol{d} \wedge \boldsymbol{v}_{v/w} \right) - \boldsymbol{\omega}_{v/w}. \tag{3.80}$$

By recalling that

$$\theta \boldsymbol{N}(\theta) = -\frac{\theta}{\sin\theta} \boldsymbol{a} \wedge \{ \boldsymbol{b} \wedge [(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}(\boldsymbol{n}\bullet))] \} \, \boldsymbol{w}_{n_d/i_v} \tag{3.81}$$

then the final expression is

$$D_v \boldsymbol{\rho} = \left(\boldsymbol{n}\boldsymbol{n}^\top - \frac{\theta}{\sin\theta} [\boldsymbol{a}\wedge][\boldsymbol{b}\wedge] \left(\boldsymbol{I}_{3\times 3} - \boldsymbol{n}\boldsymbol{n}^\top \right) \right) \begin{bmatrix} \boldsymbol{0}_{3\times l} & -\frac{1}{\|\boldsymbol{d}\|^2} [\boldsymbol{d}\wedge] & -\boldsymbol{I}_{3\times 3} \end{bmatrix} \begin{bmatrix} \dot{\boldsymbol{q}} \\ \boldsymbol{v}_{v/w} \\ \boldsymbol{\omega}_{v/w} \end{bmatrix} \triangleq \boldsymbol{J}_{at} \dot{\boldsymbol{y}} \tag{3.82}$$

Notice that, if we are interested only in the derivative of θ, then we have

$$\dot{\theta} = \boldsymbol{n}^\top \begin{bmatrix} \boldsymbol{0}_{3\times l} & -\frac{1}{\|\boldsymbol{d}\|^2} [\boldsymbol{d}\wedge] & -\boldsymbol{I}_{3\times 3} \end{bmatrix} \begin{bmatrix} \dot{\boldsymbol{q}} \\ \boldsymbol{v}_{v/w} \\ \boldsymbol{\omega}_{v/w} \end{bmatrix} \triangleq \boldsymbol{J}_{at} \dot{\boldsymbol{y}} \tag{3.83}$$

Chapter 4

Cooperative Control of Multiple Agents

4.1 Cooperative Transportation and Manipulation

In this section we shall tackle the extension of the above outlined control framework to the case of cooperating UVMSs, when transporting and deploying a grasped, shared object. For the sake of simplicity, the discussion is limited to two UVMSs, even though it can be easily extended to more than two agents.

In this case, by assuming a firm object grasping by part of both agents, we have that the tool frames $\langle t_a \rangle$, $\langle t_b \rangle$ can be respectively assigned by the agents to coincide with the shared object fixed frame $\langle o \rangle$, that is $\langle t_a \rangle \equiv \langle t_b \rangle \equiv \langle o \rangle \triangleq \langle t \rangle$, as exemplified in Fig. 4.1.

4.1.1 Introduction

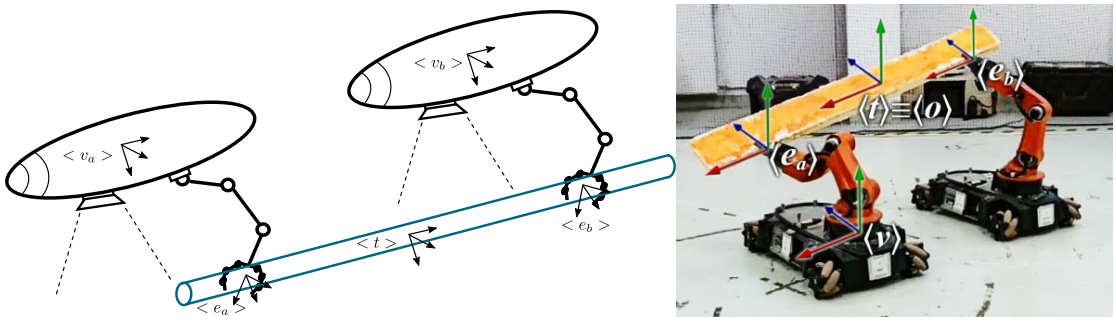
Assuming a firm object grasping by part of two agents a, b, the tool control points $\langle t_a \rangle$, $\langle t_b \rangle$ are assigned by the agents to coincide with the shared object fixed frame $\langle o \rangle$, that is $\langle t_a \rangle \equiv \langle t_b \rangle \equiv \langle o \rangle \triangleq \langle t \rangle$, as exemplified in Fig. 4.1. In the following we will only consider the tool-frame velocities, and whenever the term end-effector is used is to be intended prolonged at the tool control point $\langle t \rangle$.

In these conditions, the following differential constraints are imposed as consequence of the geometric ones

$$\dot{\boldsymbol{x}}_t = \boldsymbol{J}_{t,a}\dot{\boldsymbol{y}}_a = \boldsymbol{J}_{t,b}\dot{\boldsymbol{y}}_b, \tag{4.1}$$

with $\dot{\boldsymbol{x}}_t$ the object velocity with components on $\langle t \rangle$ and $\boldsymbol{J}_{t,a}$, $\boldsymbol{J}_{t,b}$ the system absolute Jacobians (i.e. w.r.t. an inertial observer) of frame $\langle t \rangle$, with output components on $\langle t \rangle$ itself, which is the sole frame common to both agents. Let us rewrite the second equation in (4.1) as

$$\begin{bmatrix} \boldsymbol{J}_{t,a} & -\boldsymbol{J}_{t,b} \end{bmatrix} \begin{bmatrix} \dot{\boldsymbol{y}}_a \\ \dot{\boldsymbol{y}}_b \end{bmatrix} \triangleq \boldsymbol{G}\dot{\boldsymbol{y}}_{ab} = \boldsymbol{0}, \iff \dot{\boldsymbol{y}}_{ab} \in \ker(\boldsymbol{G}), \tag{4.2}$$



(a) (b)

Figure 4.1: The frames involved in cooperative transportation: (a) for two UVMS and (b) for two youbot mobile manipulators

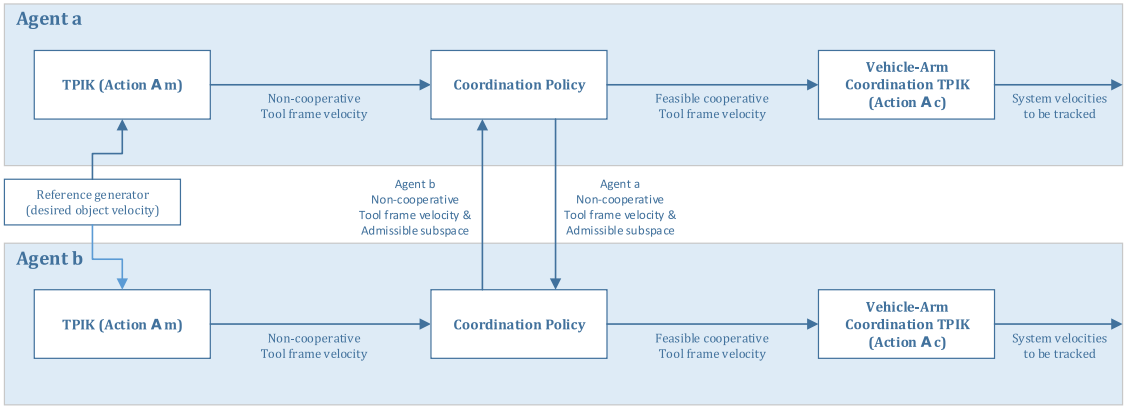


Figure 4.2: Block diagram of the cooperation. 1) A TPIK (action $\mathcal{A}_m$) is first solved, acting as if the agent was alone, using the desired object velocity $\dot{\tilde{\boldsymbol{x}}}_t$ generated by the reference generator. 2) The system velocity vector output of the TPIK is used to compute the *non-cooperative* tool-frame velocity. 3) The non-cooperative tool frame velocity, together with the admissible Cartesian subspace is exchanged with the other agent. Then, the *feasible cooperative* velocity is computed through (4.11) and (4.12). 4) Each agent now solves a coordinated TPIK (action $\mathcal{A}_c$) constrained to the feasible cooperative tool-frame velocity

which represents the subspace where $\dot{\boldsymbol{y}}_{ab}$ is constrained to lay as a consequence of the firm grasp assumption. The definition of the constrained space $\boldsymbol{G}$ of the overall system velocities of both mobile manipulators clearly couples the separate system velocities of the two agents, and it could be exploited to find a *centralized solution* satisfying the constraints at the kinematic level. However, such a possibility is discarded due to its required high rate of information exchange between the systems and the required context switch between separate controllers and a centralized one. Instead, the coordination policy proposed here is based on a reduced set of information that qualifies the evolution of the cooperation objectives, i.e. to comply with the grasp condition and to transfer the object to the desired location, and does not require changing the individual controllers.

Before proceeding with the coordination policy, let us compute the subspace of the combined end-effector velocities that are compatible with the grasp constraints. Let us first consider the motion space of a single agent, i.e.

$$\boldsymbol{H}_i \triangleq \boldsymbol{J}_{t,i}\boldsymbol{J}_{t,i}^{\#}. \tag{4.3}$$

Then, since $\dot{\boldsymbol{x}}_t$ belongs to the motion spaces of both agents, (4.1) can be written as follows

$$\dot{\boldsymbol{x}}_t = \boldsymbol{H}_a\dot{\boldsymbol{x}}_t = \boldsymbol{H}_b\dot{\boldsymbol{x}}_t, \tag{4.4}$$

which in matrix form becomes

$$\begin{bmatrix}\boldsymbol{H}_a & -\boldsymbol{H}_b\end{bmatrix}\begin{bmatrix}\dot{\boldsymbol{x}}_t \\ \dot{\boldsymbol{x}}_t\end{bmatrix} \triangleq \boldsymbol{C}\dot{\boldsymbol{x}}_{t,ab} = \boldsymbol{0}, \tag{4.5}$$

which is satisfied iff $\dot{\boldsymbol{x}}_{t,ab} \in \ker(\boldsymbol{C}) = \mathrm{Span}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C})$. During the cooperation, the actual end-effector velocities $\dot{\boldsymbol{x}}_{t,ab}$ necessarily lie in such a subspace, as result of the constraint. From a control point of view, if one is interested in avoiding interaction between the mobile manipulators and the object, the *desired* end-effector velocities must be forced to lie within this subspace. Hence, projecting the ideally desired, even possibly different, tool-frame velocities $\dot{\tilde{\boldsymbol{x}}}_{t,a}$,$\dot{\tilde{\boldsymbol{x}}}_{t,b}$ as follows

$$\dot{\tilde{\boldsymbol{x}}}_{t,ab} = (\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C})\begin{bmatrix}\dot{\tilde{\boldsymbol{x}}}_{t,a} \\ \dot{\tilde{\boldsymbol{x}}}_{t,b}\end{bmatrix} \tag{4.6}$$

forces $\dot{\tilde{\boldsymbol{x}}}_{t,ab} \in \ker(\boldsymbol{C})$, and hence makes it compliant with the constraints. Note that this does not imply that the first six components of the projected reference velocities $\dot{\tilde{\boldsymbol{x}}}_{t,ab}$ are equal to the second six ones. Indeed, there might be components outside of the motion spaces of each agent (i.e. each one

respectively belonging to the separate infeasible output spaces $\ker(\boldsymbol{J}_{a,t}^{\top})$ and $\ker(\boldsymbol{J}_{b,t}^{\top})$ of each agent) that are naturally compliant with the constraints, since they do not correspond to any tool frame $\langle t \rangle$ motion.

However, despite the possible differences, if these projected references velocities $\dot{\tilde{\boldsymbol{x}}}_{t,ab}$ are respectively given to the corresponding agent and provided each one has the non-reactive tool-frame velocity tracking task located at the *highest priority*, then the actual tool-frame velocities $\dot{\boldsymbol{x}}_{t,ab} \in \mathbb{R}^{12}$ result to be

$$\begin{aligned} \dot{\boldsymbol{x}}_{t,ab} = \begin{bmatrix} \dot{\boldsymbol{x}}_t \\ \dot{\boldsymbol{x}}_t \end{bmatrix} &= \begin{bmatrix} \boldsymbol{H}_a & \boldsymbol{0}_{6\times 6} \\ \boldsymbol{0}_{6\times 6} & \boldsymbol{H}_b \end{bmatrix} (\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}) \begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_{t,a} \\ \dot{\tilde{\boldsymbol{x}}}_{t,b} \end{bmatrix} \\ &\triangleq \boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}) \begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_{t,a} \\ \dot{\tilde{\boldsymbol{x}}}_{t,b} \end{bmatrix} \end{aligned} \tag{4.7}$$

which are consequently composed by identical six-dimensional partitions compliant with the grasp constraints.

From the above equation we can see that $\boldsymbol{H}_{ab}(\boldsymbol{I}-\boldsymbol{C}^{\#}\boldsymbol{C})$ maps from the full $\mathbb{R}^{12}$ space of tool-frame velocities to the subspace of tool-frame velocities feasible by *both* mobile manipulators *and* compliant with the object kinematic constraint. To better understand the role of the subspace $\mathrm{Span}(\boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}))$, let us remark that the whole $\mathbb{R}^{12}$ space can be divided in two orthogonal subspaces:

- $\mathrm{Span}(\boldsymbol{H}_{ab}) \subseteq \mathbb{R}^{12}$ represents the *end-effectors unconstrained motion subspace* and is the direct sum of the unconstrained motion spaces of each single agent when the object is not yet grasped.
- $\ker(\boldsymbol{H}_{ab}) = \mathrm{Span}(\boldsymbol{I} - \boldsymbol{H}_{ab}^{\#}\boldsymbol{H}_{ab}) \subseteq \mathbb{R}^{12}$ represents the *end-effectors infeasible motion subspace*, i.e. all the velocities that, even if separately commanded to both agents when the object is not yet grasped, are infeasible for both agents.

Clearly, to simultaneously control the object motion and its internal wrenches, it is useful to consider reference velocities belonging only to $\mathrm{Span}(\boldsymbol{H}_{ab})$, as those lying on $\ker(\boldsymbol{H}_{ab})$ do not generate neither motion nor interaction forces and can consequently be neglected. Once an object has been grasped by both agents, the original unconstrained subspace $\mathrm{Span}(\boldsymbol{H}_{ab})$ can be further subdivided in two orthogonal subspaces, namely:

- $\mathrm{Span}(\boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}))$ represents the subspace of tool-frame velocities which are compliant with the object kinematic constraints, and are feasible by both agents. Hence, it is termed the *end-effectors constrained motion subspace*. Since the tool frames are prolonged to the same point $\langle t \rangle$, this subspace is only composed by vectors $\dot{\boldsymbol{x}}_{t,ab} \in \mathbb{R}^{12}$ whose first six and second six components are equal and feasible by both agents.
- $\mathrm{Span}(\boldsymbol{H}_{ab}\boldsymbol{C}^{\#}\boldsymbol{C})$ instead represents the subspace of tool-frame velocities which violate the object kinematic constraints, and therefore would generate internal wrenches. Hence it is termed the *end-effectors internal-wrenches subspace*.

Therefore, to ensure a pure motion of the object (i.e., with no internal wrenches), $(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C})$ must be formerly used to project any non-compliant set of desired tool-frame velocities into compliant ones:

$$\begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_{t,a} \\ \dot{\tilde{\boldsymbol{x}}}_{t,b} \end{bmatrix} = (\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}) \begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_{t,a} \\ \dot{\tilde{\boldsymbol{x}}}_{t,b} \end{bmatrix}. \tag{4.8}$$

Of course, one will start with desired velocities which are equal, i.e. $\dot{\tilde{\boldsymbol{x}}}_t = \dot{\tilde{\boldsymbol{x}}}_{t,a} = \dot{\tilde{\boldsymbol{x}}}_{t,b}$, however the above projection process is anyway necessary to cope with possible defectiveness of each agent, while still satisfying the grasp constraints.

While the use of the sole projection matrix $(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C})$ is sufficient to ensure that the resulting desired velocities $\dot{\tilde{\boldsymbol{x}}}_{t,a}, \dot{\tilde{\boldsymbol{x}}}_{t,b}$ are compliant with the grasp constraints, they might still contain components belonging to the separate infeasible output spaces $\ker(\boldsymbol{J}_{a,t}^{\top})$ and $\ker(\boldsymbol{J}_{b,t}^{\top})$ of each agent. For clarity purposes, the successive projection in the feasible subspace is also performed, as follows:

$$\begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_t \\ \dot{\tilde{\boldsymbol{x}}}_t \end{bmatrix} = \boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}) \begin{bmatrix} \dot{\tilde{\boldsymbol{x}}}_{t,a} \\ \dot{\tilde{\boldsymbol{x}}}_{t,b} \end{bmatrix}. \tag{4.9}$$

In this way, the projected velocities are also the same ones, since they are now constituted only by feasible components.

Therefore, if the feasible tool-frame velocity $\dot{\tilde{\boldsymbol{x}}}_t$, lying on $\text{Span}(\boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C}))$, is separately commanded to both agents *and* the corresponding non-reactive tool-frame velocity tracking task is located at the *highest priority* in both agents task priority lists, then the object kinematic constraint is satisfied at the kinematic level.

At this point, accordingly with the above conclusion and the expressed general requirements, the following coordination policy development can be therefore proposed.

4.1.2 Coordination Policy

For the development of the coordination policy the following assumptions are made:

- A *reference generator* software computes the desired object velocity and broadcasts it to all the involved agents. This desired velocity could be generated to asymptotically transfer the object to a given goal location (e.g. the velocity is the output of a position loop), or be directly generated by a user in case of teleoperation. Note that this reference generator could be hosted by one of the involved agents, if required.
- The coordination policy is *distributed* between all the agents. In case of a perfect wireless network (no packet loss, infinite bandwidth, no delay), this corresponds to having a centralized coordinator software doing the computation and broadcasting the result. However, the first solution is clearly preferable from a practical implementation point of view.

Then, during each sampling interval, the following sequential steps are executed, as sketched in Fig. 4.2 and reported in Algorithm 1 (as seen from a generic agent i):

1. The latest velocity $\dot{\tilde{\boldsymbol{x}}}_t$ generated by the reference generator is acquired (line 1).
2. Each agent runs the TPIK procedure detailed in section 3.8.3 as if it were the sole one acting on the object, with its original task hierarchy $\mathcal{A}_m$ having safety and prerequisite tasks located at higher priority than the tool control point motion. The two TPIK procedures separately provide the vectors $\dot{\boldsymbol{y}}_a, \dot{\boldsymbol{y}}_b$ (line 2).
3. Each agent evaluates the Cartesian velocity that it would impose to the object if it were alone and it were applying $\dot{\boldsymbol{y}}_a$ or $\dot{\boldsymbol{y}}_b$ respectively, i.e. the following *non-cooperative* tool-frame velocities (line 3)
$$\dot{\boldsymbol{x}}_{t,i} = \boldsymbol{J}_{t,i}\dot{\boldsymbol{y}}_i, \quad i = a, b. \tag{4.10}$$
Note that the separate Cartesian velocities in (4.10) might not satisfy the kinematic constraint (4.1), i.e. $\dot{\boldsymbol{x}}_{t,a} \neq \dot{\boldsymbol{x}}_{t,b}$, due to higher priority tasks within each agent task hierarchy. Furthermore, each agent evaluates its corresponding matrix $\boldsymbol{H}_i$ (line 4) as defined in (4.3), which represents the admissible tool-frame velocity space of each agent whenever standalone acting.
4. Both agents transfer their computed quantities $\boldsymbol{H}_i$ and $\dot{\boldsymbol{x}}_{t,i}$ to the other involved agents (lines 5-6).
5. Each agent, in a *distributed way*, evaluates the *cooperative* tool-frame velocity vector (lines 7-9)
$$\dot{\tilde{\boldsymbol{x}}}_t = \frac{1}{\mu_a + \mu_b}\left(\mu_a \dot{\boldsymbol{x}}_{t,a} + \mu_b \dot{\boldsymbol{x}}_{t,b}\right), \quad \mu_a, \mu_b > 0, \tag{4.11}$$
which corresponds to a weighted compromise between the two output velocities $\dot{\boldsymbol{x}}_{t,a}, \dot{\boldsymbol{x}}_{t,b}$. The rationale and the details underlying such a weighting policy will be given at the end of the description of this procedure.

   If both agents have their safety objectives satisfied, and their current tool-frame Jacobians $\boldsymbol{J}_{t,i}$ are non-defective, i.e. $\boldsymbol{H}_i = \boldsymbol{I}$ for both $i = a, b$, then $\dot{\tilde{\boldsymbol{x}}}_t = \dot{\boldsymbol{x}}_{t,a} = \dot{\boldsymbol{x}}_{t,b}$ independently from the weights, and the kinematic constraints are clearly satisfied.

   Instead, in the general case, when $\dot{\boldsymbol{x}}_{t,a} \neq \dot{\boldsymbol{x}}_{t,b}$ their weighted sum (4.11) might not lay in the end-effectors constrained motion subspace, therefore it must be projected on such a subspace, as performed in the next two steps.
6. Each agent evaluates the Cartesian constraint matrix as defined in (4.5) (line 10).

Algorithm 1 CooperativeTPIK($\mathcal{A}_m$)

Require: The manipulation action $\mathcal{A}_m$, the constrained manipulation action $\mathcal{A}_c$
Ensure: The reference system velocity vector $\dot{\tilde{\boldsymbol{y}}}_i$

1: $\dot{\tilde{\boldsymbol{x}}}_t \leftarrow$ GetDesiredObjectVelocity()
2: $\dot{\boldsymbol{y}}_i \leftarrow$ TPIK($\mathcal{A}_m$, $\dot{\tilde{\boldsymbol{x}}}_t$)
3: $\dot{\boldsymbol{x}}_{t,i} \leftarrow \boldsymbol{J}_{t,i}\dot{\boldsymbol{y}}_i$
4: $\boldsymbol{H}_i \leftarrow \boldsymbol{J}_{t,i}\boldsymbol{J}_{t,i}^{\#}$
5: SendDataToAgent(j, $\dot{\boldsymbol{x}}_{t,i}$, $\boldsymbol{H}_i$)
6: $\dot{\boldsymbol{x}}_{t,j}, \boldsymbol{H}_j \leftarrow$ GetLatestDataFromAgent(j)
7: $\mu_i \leftarrow \mu_0 + \|\dot{\tilde{\boldsymbol{x}}}_t - \dot{\boldsymbol{x}}_{t,i}\|$
8: $\mu_j \leftarrow \mu_0 + \|\dot{\tilde{\boldsymbol{x}}}_t - \dot{\boldsymbol{x}}_{t,j}\|$
9: $\dot{\hat{\boldsymbol{x}}}_t \leftarrow \frac{1}{\mu_i+\mu_j}\left(\mu_i\dot{\boldsymbol{x}}_{t,i} + \mu_j\dot{\boldsymbol{x}}_{t,j}\right)$
10: $\boldsymbol{C} \leftarrow \begin{bmatrix}\boldsymbol{H}_i & -\boldsymbol{H}_j\end{bmatrix}$
11: $\dot{\tilde{\boldsymbol{x}}}_t \leftarrow \boldsymbol{H}_{ij}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C})\begin{bmatrix}\dot{\hat{\boldsymbol{x}}}_t^{\top} & \dot{\hat{\boldsymbol{x}}}_t^{\top}\end{bmatrix}^{\top}$
12: $\dot{\tilde{\boldsymbol{y}}}_i \leftarrow$ CoordinatedTPIK($\mathcal{A}_c$, $\dot{\tilde{\boldsymbol{x}}}_t$, $\boldsymbol{v}_i$)
13: **return** $\dot{\tilde{\boldsymbol{y}}}_i$

7. Each agent considers the vector $\dot{\hat{\boldsymbol{x}}}_t$ as its reference velocity, i.e. $\dot{\hat{\boldsymbol{x}}}_{t,i} = \dot{\hat{\boldsymbol{x}}}_t$ and projects it on the end-effectors constrained motion subspace Span($\boldsymbol{H}_{ab}(\boldsymbol{I}-\boldsymbol{C}^{\#}\boldsymbol{C})$), obtaining the so-called *feasible cooperative* velocity vector (line 11)

$$\begin{bmatrix}\dot{\tilde{\boldsymbol{x}}}_t \\ \dot{\hat{\boldsymbol{x}}}_t\end{bmatrix} \triangleq \boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^{\#}\boldsymbol{C})\begin{bmatrix}\dot{\hat{\boldsymbol{x}}}_t \\ \dot{\hat{\boldsymbol{x}}}_t\end{bmatrix}. \tag{4.12}$$

8. Both agents separately run a new TPIK procedure, implementing the vehicle arm coordination of section 3.8.7, with the original task-priority hierarchy now modified into the one having the non-reactive tool-frame velocity tracking of $\dot{\tilde{\boldsymbol{x}}}_t$ at the *highest priority* (line 12):

   o_1) constrained tool velocity tracking;
   o_2) arm joint limits;
   o_3) obstacle avoidance;
   o_4) arm manipulability;
   o_5) arm preferred pose.

   In this way, both systems independently optimize their safety tasks under the imposed constraint of the common tool frame velocity. Such a new hierarchy is termed action $\mathcal{A}_c$ in Fig. 4.2 and Algorithm 1.

9. Each agent actuates the output of the second TPIK procedure (line 13).

In order to explain the rationale supporting the proposed coordination procedure, first of all note that, as regard the proposed choice for the assignment of weights μ_a, μ_b within (4.11) the following choice is actually made

$$\begin{aligned}\mu_a &= \mu_0 + \|\dot{\tilde{\boldsymbol{x}}}_t - \dot{\boldsymbol{x}}_{t,a}\| \triangleq \mu_0 + \|\boldsymbol{e}_a\|, \\ \mu_b &= \mu_0 + \|\dot{\tilde{\boldsymbol{x}}}_t - \dot{\boldsymbol{x}}_{t,b}\| \triangleq \mu_0 + \|\boldsymbol{e}_b\|,\end{aligned} \tag{4.13}$$

with $\mu_0 > 0$ and where the norms $\|\boldsymbol{e}_i\|$ serve as a measure of the difficulties that agent i has in tracking the original object reference velocity, due to higher priority safety and prerequisite tasks. Thus, with the above weighting choice, the resulting cooperative velocity $\dot{\hat{\boldsymbol{x}}}_t$ will be closer to the one evaluated by the agent exhibiting the greatest difficulty in tracking the original desired object velocity, than to the one evaluated by the other agent.

For example, in case $\|\boldsymbol{e}_a\|/\|\boldsymbol{e}_b\| \to \infty$, then $\dot{\hat{\boldsymbol{x}}}_t \to \dot{\boldsymbol{x}}_{t,a}$, which implies that agent a progressively imposes its non-cooperative tool-frame velocity as the cooperative one. Consequently, agent b will progressively follow what is imposed by agent a at the Cartesian level. Such a situation, and its

opposite one, correspond to the extreme cases. In all the other situations, the weighting rule (4.13) provides an adequate compromise.

The above discussion is strongly based on the implicitly assumed property that nothing can change, for an agent, whenever an equality task is transferred to the highest priority, provided that its velocity is maintained equal to the result it provided when it was at a lower priority level. Such an assumed property, despite appearing as an evident one at a first glance, is proven in the Appendix.

Furthermore, it should be noted how the proposed coordination policy does not require an explicit, a priori, definition of any leader or follower roles (contribution C_2 highlighted in the introduction), since the tendency of an agent to become leader or follower within the team is governed by its comparative difficulty with the other agent in accomplishing the required object motion task. Such an emerging behaviour is very similar to how we operate in everyday life when transporting a big object with another human, as we naturally adapt to the other person if he/she is in difficulty, e.g. due to an obstacle or to a worse grasp of the object.

Finally, a remark should be made on the possibility of imposing desired internal wrenches. During step 7 (line 11 of Algorithm 1) of the coordination policy, the cooperative object velocity $\begin{bmatrix}\dot{\boldsymbol{x}}_t^\top & \dot{\boldsymbol{x}}_t^\top\end{bmatrix}^\top$ was projected using the projection matrix $\boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^\#\boldsymbol{C})$, thus guaranteeing that its result was feasible, i.e. compatible with object kinematic constraint, therefore ensuring that the motion of the agents would not generate *unwanted* internal object forces. However, (4.12) could be modified to take into account *desired* internal wrenches using the orthogonal space projection matrix $\boldsymbol{H}_{ab}\boldsymbol{C}^\#\boldsymbol{C}$:

$$\begin{bmatrix}\dot{\tilde{\boldsymbol{x}}}_{t,a}\\ \dot{\tilde{\boldsymbol{x}}}_{t,b}\end{bmatrix} \triangleq \boldsymbol{H}_{ab}(\boldsymbol{I} - \boldsymbol{C}^\#\boldsymbol{C}) \begin{bmatrix}\dot{\tilde{\boldsymbol{x}}}_t\\ \dot{\tilde{\boldsymbol{x}}}_t\end{bmatrix} + \boldsymbol{H}_{ab}(\boldsymbol{C}^\#\boldsymbol{C}) \begin{bmatrix}\dot{\tilde{\boldsymbol{x}}}_{w,a}\\ \dot{\tilde{\boldsymbol{x}}}_{w,b}\end{bmatrix}, \tag{4.14}$$

where $\dot{\tilde{\boldsymbol{x}}}_{w,i}$ are velocity requests that are projected to be against the constraint and therefore, after a small transient, will generate wrenches in the same direction. For example, one could generate both $\dot{\tilde{\boldsymbol{x}}}_{w,i}$ to point from the i-th end-effector grasping point to the object's centre, to keep it compressed. However, this possibility could not be tested and validated during the experiments since the YouBot mobile manipulators are not equipped with wrench sensors able to measure the interaction with the object, and therefore is left for future investigations.

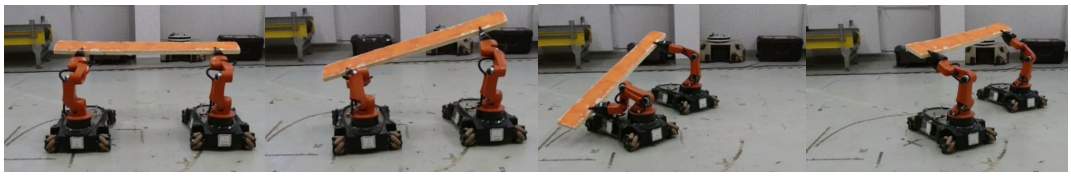
4.1.3 Experimental Results

In this section we present some experimental results of the proposed cooperative control architecture. The setup used to test the proposed algorithm is composed by two Kuka YouBot mobile manipulators, as already shown in Fig. 4.1. The YouBot robot is constituted by a platform actuated by 4 omnidirectional wheels and by a 5 DOF manipulator. Therefore, the quantities defined in Section 3 can now be narrowed to the following ones:

- $\boldsymbol{c} \triangleq \begin{bmatrix}\boldsymbol{q}^\top & \boldsymbol{\eta}^\top\end{bmatrix}^\top \in \mathbb{R}^8$ contains the robot DOF, i.e. the arm joint positions $\boldsymbol{q} \in \mathbb{R}^5$ and the vehicle position and orientation vector $\boldsymbol{\eta} = \begin{bmatrix}x & y & \psi\end{bmatrix}^\top \in \mathbb{R}^3$;
- $\dot{\boldsymbol{y}} \triangleq \begin{bmatrix}\dot{\boldsymbol{q}}^\top & \boldsymbol{v}^\top\end{bmatrix}^\top \in \mathbb{R}^8$, and represents the controls to actuate the robot, i.e. the joint velocities $\dot{\boldsymbol{q}} \in \mathbb{R}^5$ and vehicle linear and yaw angular velocities $\boldsymbol{v} \triangleq \begin{bmatrix}\dot{x} & \dot{y} & \dot{\psi}\end{bmatrix}^\top \in \mathbb{R}^3$.

The two agents operate in a motion capture area, which is used to assess their absolute position, and to compute the transformation matrices necessary for setting their tool-frames $\langle t_a\rangle \equiv \langle t_b\rangle \equiv \langle o\rangle \triangleq \langle t\rangle$. The use of the motion capture environment allows to simplify the perception problem and to focus this work on the kinematic control strategy. During each experiment, the software process running the KCL is executed at 100 Hz. During every loop, the controller sends the computed quantities $\boldsymbol{H}_i$ and $\dot{\boldsymbol{x}}_{t,i}$ to the other involved agents using non-reliable UDP (User Datagram Protocol) packets (line 5 of Algorithm 1), and it uses the latest information received in the same manner from the other agents (line 6). The choice of UDP is made to avoid the queuing messages when only the latest one is actually useful. If one of the agents does not receive the corresponding information for a time longer than a time-out threshold, the cooperation process is stopped. No time synchronization is needed between the agents.

Finally, let us recall how the DCLs of the YouBot arm and vehicle are predefined and could not be modified. Hence, the KCL performances were tuned, acting on the task gains and thresholds, to cope with the underlying DCL performances.



(a) (b) (c) (d)

Figure 4.3: Teleoperation experiment: snapshots of the two YouBots during the teleoperated manipulation.

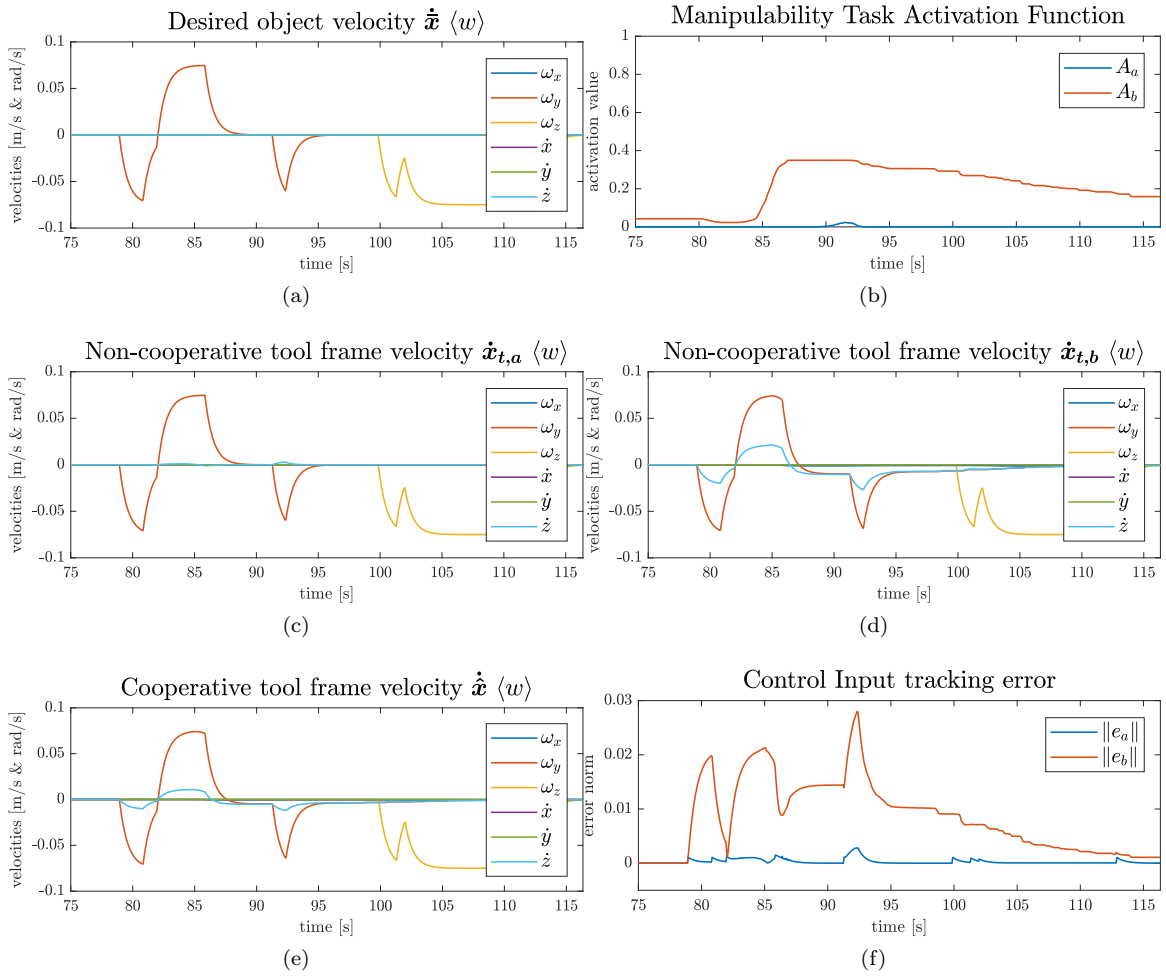


Figure 4.4: Teleoperation experiment results: (a) the desired object velocity, (b) manipulability task activation function, (c) non-cooperative tool-frame velocities for agent a (d) non-cooperative tool-frame velocities for agent b, (e) cooperative tool-frame velocities (tuned toward agent b), (f) norms of object velocity tracking errors, showing how agent b is in higher difficulty.

4.1.3.1 Teleoperation Experiment

In the first experiment, a user teleoperates the cooperating agents by generating desired object velocities. The outcome of the experiment is shown through some snapshots in Fig. 4.3, while the user-generated object reference velocity $\dot{\bar{x}}_t$ is depicted in Fig. 4.4a.

Figure 4.4 shows an excerpt of the experiment, which can be seen in the attached video. In particular, at the start of the graphs ($t = 75$ s) the two mobile manipulators are in a configuration similar to the one represented in Fig. 4.3c, where agent b is close to a singular configuration. This is



(a) (b)



(c) (d)

Figure 4.5: Obstacle avoidance experiment: snapshots of the two YouBots as they cooperatively avoid the obstacle and reach the desired final position.

shown by the fact that the manipulability task's activation function is greater than zero, as depicted in Fig. 4.4b. From $t = 80$ s to $t = 100$ s the operator specifically gives a desired object reference velocity which agent b cannot track exactly. This fact can be seen comparing Fig. 4.4a with Fig. 4.4c and Fig. 4.4d for agents a and b respectively, and it is highlighted by the error norms $\|\dot{\tilde{\boldsymbol{x}}}_t - \dot{\boldsymbol{x}}_{t,i}\|$ shown in Fig. 4.4f. As can be seen in Fig. 4.4e, the *feasible cooperative* tool frame velocity is tuned in favour of agent b. Indeed, components on the linear z-direction are now present, which instead were zero in the original reference velocity. Thanks to the proposed coordination policy, agent b manages to keep its manipulability above the required threshold, as confirmed by the fact that its activation value in Fig. 4.4b never reaches the value one, while both systems instantaneously generate system velocities complying with the firm grasp object kinematic constraint.

4.1.3.2 Obstacle Avoidance Experiment

In this experiment an obstacle has been placed within the motion capture area. As the YouBots are not endowed with perception sensors, its position is known a priori to both agents. Of course, in a real scenario, the position of the obstacle would be perceived in real-time through the use of dedicated sensors and detection algorithms. The desired object velocity is generated to reach a final goal position on the other side of the motion capture area, with respect to the initial position of the YouBots and the object.

The execution of this experiment is depicted in Fig. 4.5. Roughly at $t = 24$ s, agent b is getting close to the obstacle, and therefore the activation function of the obstacle avoidance task raises, as shown in Fig. 4.6b. For this reason, the error norm $\|\boldsymbol{e}_b\|$ of the end-effector position and attitude tasks increases, as shown in Fig. 4.6f. The non-cooperative Cartesian velocity of agent b, shown in Fig. 4.6d, mainly differs from the desired object velocity (Fig. 4.6a) in the $\dot{x}$ and $\dot{y}$ linear velocity components. This is due to the fact that the vehicle needs to avoid the obstacle moving in the (x, y) horizontal plane. Instead, agent a has no problem in tracking the desired object velocity, and indeed its non-cooperative Cartesian velocity shown in Fig. 4.6c is very close to the desired one. Thanks to the proposed coordination policy, the two agents agree on a cooperative tool-frame velocity (Fig. 4.6e) which is substantially the one of agent b, due to its higher tracking error. In this way, the two YouBots

coordinate themselves for avoiding the obstacle, as agent a adapts to the movement imposed by agent b for avoiding the obstacle.

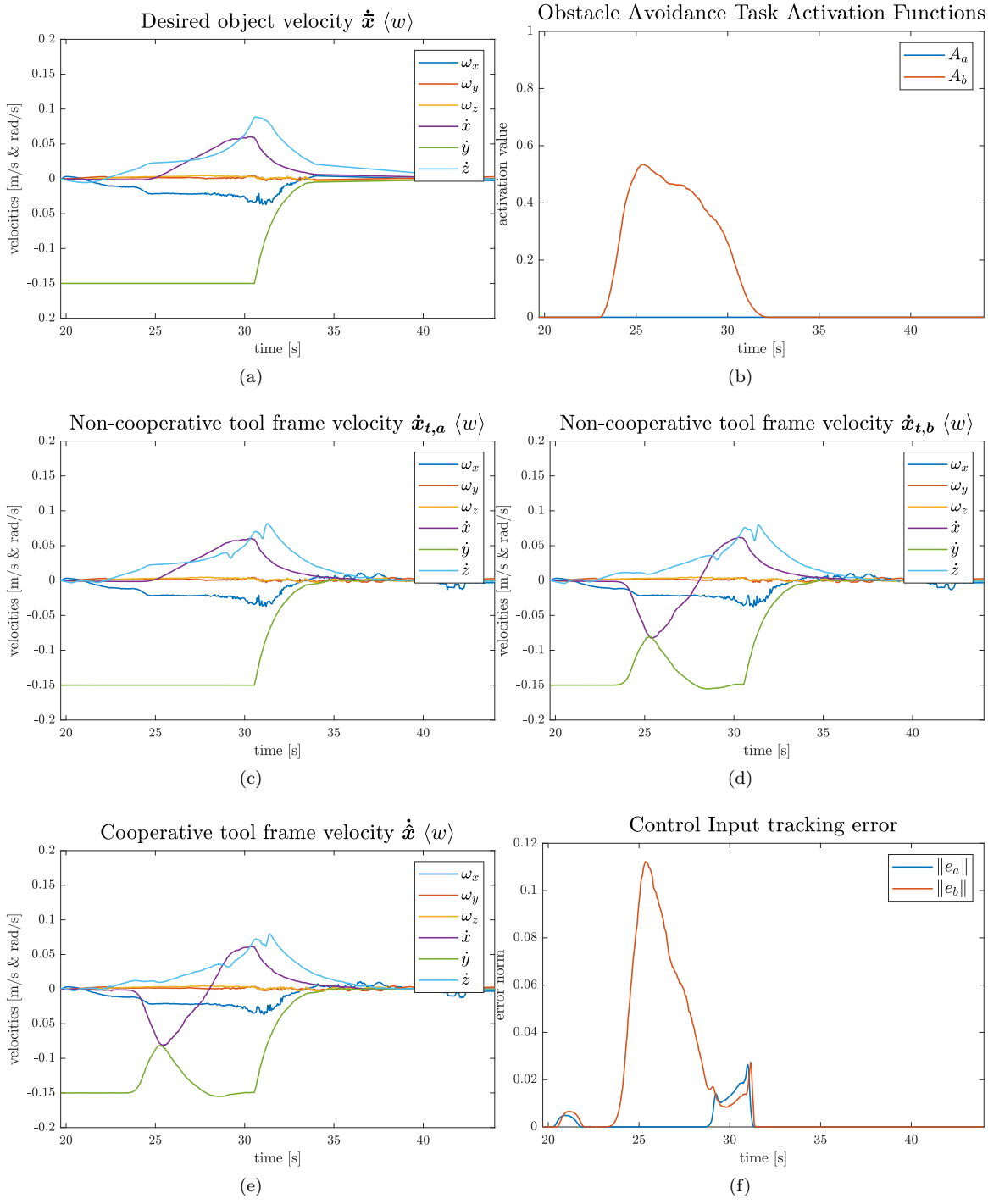


Figure 4.6: Obstacle avoidance experiment results: (a) the desired object velocity, (b) obstacle avoidance task activation function, (c) non-cooperative tool-frame velocities for agent a (d) non-cooperative tool-frame velocities for agent b, (e) cooperative tool-frame velocities (tuned toward agent b), (f) norms of object velocity tracking errors, showing how agent b is in higher difficulty.

Appendix A

Pseudo Inverses and SVD

A.1 Moore-Penrose Pseudo Inverse

A pseudoinverse $\boldsymbol{A}^{\#}$ of a matrix $\boldsymbol{A}$ is a generalization of the inverse matrix. The most widely known type of matrix pseudoinverse is the Moore–Penrose pseudoinverse

Let us consider a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$. A pseudoinverse of $\boldsymbol{A}$ is defined as a matrix $\boldsymbol{A}^{\#} \in \mathbb{R}^{n \times m}$ satisfying all of the following four criteria:

$$\boldsymbol{A}\boldsymbol{A}^{\#}\boldsymbol{A} = \boldsymbol{A} \tag{A.1}$$

$$\boldsymbol{A}^{\#}\boldsymbol{A}\boldsymbol{A}^{\#} = \boldsymbol{A}^{\#} \tag{A.2}$$

$$(\boldsymbol{A}\boldsymbol{A}^{\#})^{\top} = \boldsymbol{A}\boldsymbol{A}^{\#} \tag{A.3}$$

$$(\boldsymbol{A}^{\#}\boldsymbol{A})^{\top} = \boldsymbol{A}^{\#}\boldsymbol{A} \tag{A.4}$$

The pseudoinverse $\boldsymbol{A}^{\#}$ exists for any matrix. However, when $\boldsymbol{A}$ has full rank, then it can be expressed as a simple algebraic formula. In particular, when $\boldsymbol{A}$ has linearly independent columns (therefore $\boldsymbol{A}^{\top}\boldsymbol{A}$ is invertible), then

$$\boldsymbol{A}^{\#} = (\boldsymbol{A}^{\top}\boldsymbol{A})^{-1}\boldsymbol{A}^{\top} \tag{A.5}$$

The above pseudoinverse is also called a *left inverse*, since in this case $\boldsymbol{A}^{\#}\boldsymbol{A} = \boldsymbol{I}$.

When $\boldsymbol{A}$ has linearly independent rows (therefore $\boldsymbol{A}\boldsymbol{A}^{\top}$ is invertible), then

$$\boldsymbol{A}^{\#} = \boldsymbol{A}^{\top}(\boldsymbol{A}\boldsymbol{A}^{\top})^{-1} \tag{A.6}$$

The above pseudoinverse is also called a *right inverse*, since in this case $\boldsymbol{A}\boldsymbol{A}^{\#} = \boldsymbol{I}$.

Among others, the following identities hold for a Moore-Penrose pseudoinverse:

$$\boldsymbol{A}^{\#} = (\boldsymbol{A}^{\top}\boldsymbol{A})^{\#}\boldsymbol{A}^{\top}, \tag{A.7}$$

$$\boldsymbol{A}^{\#} = \boldsymbol{A}^{\top}(\boldsymbol{A}\boldsymbol{A}^{\top})^{\#}. \tag{A.8}$$

A.2 Singular Value Decomposition

Theorem
Given a matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, it can be always decomposed as

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top} \tag{A.9}$$

where

$$\begin{aligned} &\boldsymbol{U} \in \mathbb{R}^{m \times m} \quad \text{orthonormal} \quad \boldsymbol{U}^{-1} = \boldsymbol{U}^{\top} \\ &\boldsymbol{V} \in \mathbb{R}^{n \times n} \quad \text{orthonormal} \quad \boldsymbol{V}^{-1} = \boldsymbol{V}^{\top} \\ &\boldsymbol{\Sigma} \in \mathbb{R}^{m \times n} \end{aligned} \tag{A.10}$$

$$\boldsymbol{\Sigma} = \begin{cases} \begin{bmatrix} \bar{\boldsymbol{\Sigma}} & | & 0 \end{bmatrix} & \text{if } m \leq n \quad (\text{if } m = n \rightarrow \boldsymbol{\Sigma} = \bar{\boldsymbol{\Sigma}}) \\ \begin{bmatrix} \bar{\boldsymbol{\Sigma}} \\ - \\ 0 \end{bmatrix} & \text{if } m > n \end{cases} \tag{A.11}$$

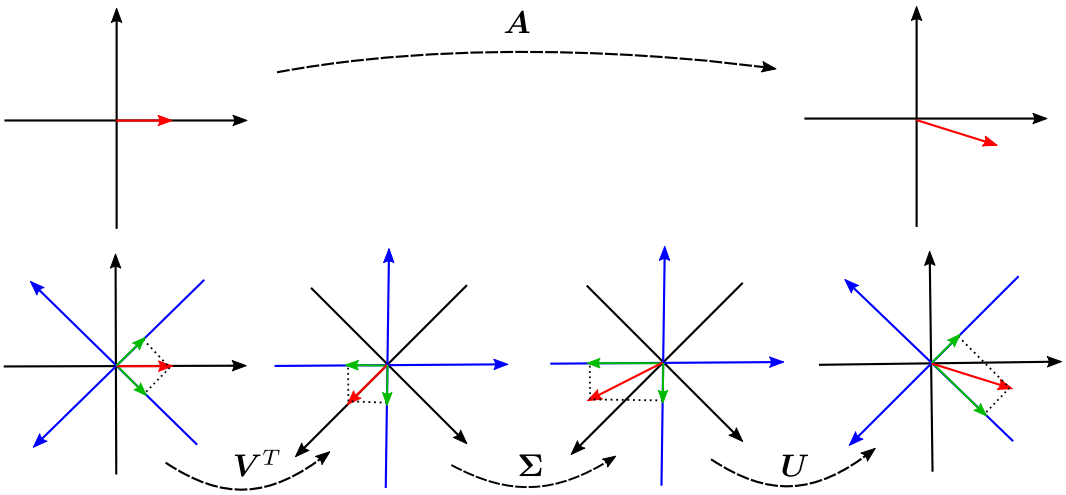


Figure A.1: Analysis of the geometric meaning of the SVD decomposition of a transformation $\boldsymbol{A}$. First the vector $\boldsymbol{x}$ (depicted in red) is decomposed along the axes of $\boldsymbol{V}^\top$ (depicted in blue). Along these new axes the transformation is independent, and each scalar factor is applied. Finally, the rotation $\boldsymbol{U}$ is applied to obtain the final decomposition, which yields the vector $\boldsymbol{y}$.

with

$$\bar{\boldsymbol{\Sigma}} \in \mathbb{R}^{k\times k} \qquad k = \min[m,n] \tag{A.12}$$

$$\bar{\boldsymbol{\Sigma}} = diag(\sigma_1, \sigma_2, \ldots, \sigma_k) \tag{A.13}$$

where

$$\sigma_i = \begin{cases} \sqrt{\lambda_i(\boldsymbol{A}\boldsymbol{A}^\top)} & m < n \\ \sqrt{\lambda_i(\boldsymbol{A}^\top\boldsymbol{A})} & m \geq n \end{cases} \tag{A.14}$$

and ordered within $\bar{\boldsymbol{\Sigma}}$ as $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{k-1} \geq \sigma_k$. λ is a generic eigenvalue.

What is the geometric meaning?

A.2.1 Geometric Interpretation of SVD

$$\begin{aligned} \boldsymbol{y} &= \boldsymbol{A}\boldsymbol{x} \\ \boldsymbol{y} &= (\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top)\boldsymbol{x} \\ \boldsymbol{U}^\top\boldsymbol{y} &= \boldsymbol{U}^\top\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top\boldsymbol{x} \\ \boldsymbol{U}^\top\boldsymbol{y} &= \boldsymbol{\Sigma}\boldsymbol{V}^\top\boldsymbol{x} \\ \boldsymbol{\xi} &= \boldsymbol{\Sigma}\boldsymbol{\theta} \end{aligned} \tag{A.15}$$

where

$$\begin{cases} \boldsymbol{\xi} \triangleq \boldsymbol{U}^\top\boldsymbol{y} \\ \boldsymbol{\theta} \triangleq \boldsymbol{V}^\top\boldsymbol{x} \end{cases} \tag{A.16}$$

Notice that the product $\boldsymbol{U}^\top\boldsymbol{U}$ in equation A.15 is equal to 1 because $\boldsymbol{U}$ is orthonormal ($\boldsymbol{U}^\top = \boldsymbol{U}^{-1}$). The geometric meaning is: SVD allows to change the frames of the input and the output so that the transformation matrix is Σ. $\boldsymbol{U}$ and $\boldsymbol{V}$ are partially unique, we have arbitrary on some columns if some σ are equal to zero.

Consider

$$\boldsymbol{A} = \begin{bmatrix} 1.4142 & 1.4142 \\ -0.7071 & 0.7071 \end{bmatrix} \tag{A.17}$$

then its SVD decomposition is

$$\boldsymbol{A} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} -0.7071 & -0.7071 \\ -0.7071 & 0.7071 \end{bmatrix}. \tag{A.18}$$

Then if we consider $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ and we choose $\boldsymbol{x} = \begin{bmatrix}1\\0\end{bmatrix}$ the result is $\boldsymbol{y} = \begin{bmatrix}1.4142\\-0.7071\end{bmatrix}$. Fig A.1 reports a step by step analysis of the transformation applied by $\boldsymbol{A}$ thanks to its SVD decomposition.

A.2.2 Pseudo Inversion through SVD

Let us consider the case $m < n$:

$$\begin{aligned}\boldsymbol{J}^{\#} &= \boldsymbol{J}^{\top}(\boldsymbol{J}\boldsymbol{J}^{\top})^{-1} = (\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top})\left[(\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top})(\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top})\right]^{-1}\\ &= (\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top})\left[\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top}\right]^{-1}\\ &= (\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top})(\boldsymbol{U}\bar{\boldsymbol{\Sigma}}^{-2}\boldsymbol{U}^{\top})\\ &= \boldsymbol{V}\boldsymbol{\Sigma}^{\top}\bar{\boldsymbol{\Sigma}}^{-2}\boldsymbol{U}^{\top}\end{aligned} \tag{A.19}$$

$$\boldsymbol{\Sigma}^{\top}\bar{\boldsymbol{\Sigma}}^{-2} = \begin{bmatrix}\bar{\boldsymbol{\Sigma}}\\0\end{bmatrix}\bar{\boldsymbol{\Sigma}}^{-2} = \begin{bmatrix}\bar{\boldsymbol{\Sigma}}^{-1}\\0\end{bmatrix} \tag{A.20}$$

$$\boldsymbol{J}^{\#} = \boldsymbol{V}\begin{bmatrix}\bar{\boldsymbol{\Sigma}}^{-1}\\0\end{bmatrix}\boldsymbol{U}^{\top} = \boldsymbol{V}\begin{bmatrix}\frac{1}{\sigma_1} & & \\ & \ddots & \\ & & \frac{1}{\sigma_m}\\ 0 & & \end{bmatrix}\boldsymbol{U}^{\top} \tag{A.21}$$

Looking at the transformed spaces we have

$$\boldsymbol{\theta} = \begin{bmatrix}\bar{\boldsymbol{\Sigma}}^{-1}\\0\end{bmatrix}\boldsymbol{\xi} = \begin{bmatrix}\frac{1}{\sigma_1} & & \\ & \ddots & \\ & & \frac{1}{\sigma_m}\\ 0 & & \end{bmatrix}\boldsymbol{\xi} \tag{A.22}$$

The last $n - m$ components are zero. However with the loss of rank $\frac{1}{\sigma_m} \rightarrow \infty$ and consequently $\theta_{(m)\rightarrow\infty}$.

Let us now focus on $m > n$

$$\begin{aligned}\boldsymbol{J}^{\#} &= (\boldsymbol{J}^{\top}\boldsymbol{J})^{-1}\boldsymbol{J}^{\top} = \left[(\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top})(\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\top})\right]^{-1}(\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top})\\ &= \left[\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{\Sigma}\boldsymbol{V}^{\top}\right]^{-1}\boldsymbol{V}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top}\\ &= \boldsymbol{V}(\boldsymbol{\Sigma}^{\top}\boldsymbol{\Sigma})^{-1}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top}\\ &= \boldsymbol{V}\bar{\boldsymbol{\Sigma}}^{-2}\boldsymbol{\Sigma}^{\top}\boldsymbol{U}^{\top}\end{aligned} \tag{A.23}$$

$$\bar{\boldsymbol{\Sigma}}^{-2}\boldsymbol{\Sigma}^{\top} = \bar{\boldsymbol{\Sigma}}^{-2}\left[\bar{\boldsymbol{\Sigma}} \quad 0\right] = \left[\bar{\boldsymbol{\Sigma}}^{-1} \quad 0\right] \tag{A.24}$$

Consequently:

$$\boldsymbol{J}^{\#} = \boldsymbol{V}\left[\bar{\boldsymbol{\Sigma}}^{-1} \quad 0\right]\boldsymbol{U}^{\top} = \boldsymbol{V}\begin{bmatrix}\frac{1}{\sigma_1} & & & \\ & \ddots & & 0\\ & & \frac{1}{\sigma_n} & \end{bmatrix}\boldsymbol{U}^{\top} \tag{A.25}$$

A.2.3 Singular Value Oriented Regularization

We consider

$$\boldsymbol{J}^{\#} = (\boldsymbol{J}^{\top}\boldsymbol{J} + \boldsymbol{K})^{-1}\boldsymbol{J}^{\top} \tag{A.26}$$

Now $\boldsymbol{K}$ is a matrix. How do we choose it?

Let us structure $\boldsymbol{K}$ in the following way

$$\boldsymbol{K} \triangleq \boldsymbol{V}\boldsymbol{\Gamma}\boldsymbol{V}^{\top} \tag{A.27}$$

with

$$\boldsymbol{\Gamma} \triangleq \begin{bmatrix} \gamma_1 & & \\ & \ddots & \\ & & \gamma_n \end{bmatrix} \in \mathbb{R}^{n \times n}, \tag{A.28}$$

and where

$$\gamma_i \triangleq \gamma_i(\sigma_i). \tag{A.29}$$

Then we have

$$\begin{aligned} \boldsymbol{J}^{\#} &= (\boldsymbol{J}^\top \boldsymbol{J} + \boldsymbol{K})^{-1} \boldsymbol{J}^\top = \left[(\boldsymbol{V}\boldsymbol{\Sigma}^\top \boldsymbol{U}^\top)(\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top + \boldsymbol{K}) \right]^{-1} (\boldsymbol{V}\boldsymbol{\Sigma}^\top \boldsymbol{U}^\top) \\ &= \left[\boldsymbol{V}\boldsymbol{\Sigma}^\top \boldsymbol{\Sigma}\boldsymbol{V}^\top + \boldsymbol{V}\boldsymbol{\Gamma}\boldsymbol{V}^\top \right]^{-1} \boldsymbol{V}\boldsymbol{\Sigma}^\top \boldsymbol{U}^\top \\ &= \left[\boldsymbol{V}(\boldsymbol{\Sigma}^\top \boldsymbol{\Sigma} + \boldsymbol{\Gamma})\boldsymbol{V}^\top \right]^{-1} \boldsymbol{V}\boldsymbol{\Sigma}^\top \boldsymbol{U}^\top \\ &= \boldsymbol{V}(\boldsymbol{\Sigma}^\top \boldsymbol{\Sigma} + \boldsymbol{\Gamma})^{-1} \boldsymbol{V}^\top \boldsymbol{V}\boldsymbol{\Sigma}^\top \boldsymbol{U}^\top \\ &= \boldsymbol{V}(\boldsymbol{\Sigma}^\top \boldsymbol{\Sigma} + \boldsymbol{\Gamma})^{-1} \boldsymbol{\Sigma}^\top \boldsymbol{U}^\top \end{aligned} \tag{A.30}$$

Now, if $m < n$

$$\begin{aligned} (\boldsymbol{\Sigma}^\top \boldsymbol{\Sigma} + \boldsymbol{\Gamma})^{-1} \boldsymbol{\Sigma}^\top &= \left(\begin{bmatrix} \bar{\boldsymbol{\Sigma}}^2 & 0 \\ 0 & 0 \end{bmatrix} + \boldsymbol{\Gamma} \right)^{-1} \begin{bmatrix} \bar{\boldsymbol{\Sigma}} \\ 0 \end{bmatrix} \\ &= \left(\begin{bmatrix} \sigma_1^2 + \gamma_1 & & & & & & \\ & \ddots & & & & & \\ & & \sigma_m^2 + \gamma_m & & & & \\ & & & \gamma_{m+1} & & & \\ & & & & \ddots & & \\ & & & & & \gamma_n \end{bmatrix} \right)^{-1} \begin{bmatrix} \bar{\boldsymbol{\Sigma}} \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} \frac{\sigma_1}{\sigma_1^2 + \gamma_1} & & \\ & \ddots & \\ & & \frac{\sigma_m}{\sigma_m^2 + \gamma_m} \\ & 0 & \end{bmatrix} \end{aligned} \tag{A.31}$$

If $m > n$

$$\begin{aligned} (\boldsymbol{\Sigma}^\top \boldsymbol{\Sigma} + \boldsymbol{\Gamma})^{-1} \boldsymbol{\Sigma}^\top &= \left(\bar{\boldsymbol{\Sigma}}^2 + \boldsymbol{\Gamma} \right)^{-1} \begin{bmatrix} \bar{\boldsymbol{\Sigma}} & 0 \end{bmatrix} \\ &= \left(\begin{bmatrix} \sigma_1^2 + \gamma_1 & & \\ & \ddots & \\ & & \sigma_n^2 + \gamma_n \end{bmatrix} \right)^{-1} \begin{bmatrix} \bar{\boldsymbol{\Sigma}} & 0 \end{bmatrix} \\ &= \begin{bmatrix} \frac{\sigma_1}{\sigma_1^2 + \gamma_1} & & & \\ & \ddots & & 0 \\ & & \frac{\sigma_n}{\sigma_n^2 + \gamma_n} & \end{bmatrix} \end{aligned} \tag{A.32}$$

In both cases, with the addition of matrix $\boldsymbol{\Gamma}$, we are performing a regularization oriented on the specific singular value below threshold. All the direction corresponding to singular values above threshold will be unaffected by regularization on the others.

Equivalent results can be obtained, for both cases, if one starts with

$$\boldsymbol{J}^{\#} = (\boldsymbol{J}^\top \boldsymbol{J} + \boldsymbol{K})^{-1} \boldsymbol{J}^\top \tag{A.33}$$

and

$$\boldsymbol{K} = \boldsymbol{U}\boldsymbol{\Gamma}\boldsymbol{U}^\top. \tag{A.34}$$

Appendix B

Inverse Kinematics: a Geometric Interpretation

Let us focus on the problem of finding out which joint velocities $\dot{\boldsymbol{q}}$ allow to obtain a desired end-effector Cartesian $\dot{\tilde{\boldsymbol{x}}}$ velocity, which is known as the inverse kinematic problem, by exploiting the Jacobian relationship

$$\dot{\boldsymbol{x}} = \boldsymbol{J}\dot{\boldsymbol{q}} \tag{B.1}$$

B.1 Full Row Rank Jacobian case

Let us suppose to have $n \geq 6$. If $\partial(J) = 6$, then matrix $\boldsymbol{J}$, interpreted as a linear transformation, maps all of $\mathbb{R}^6$. Thus, it will certainly exists a $\dot{\boldsymbol{q}}$ that is mapped in $\dot{\tilde{\boldsymbol{x}}}$. We can write the solution as two separate parts:

$$\dot{\boldsymbol{q}} = \dot{\boldsymbol{q}}_0 + \dot{\boldsymbol{q}}_1 \tag{B.2}$$

where $\dot{\boldsymbol{q}}_1$ is any solution of the (B.1), and $\dot{\boldsymbol{q}}_0$ is any solution of the homogeneous equation $\boldsymbol{0} = \boldsymbol{J}\dot{\boldsymbol{q}}_0$., i.e. $\dot{\boldsymbol{q}}_0 \in N(\boldsymbol{J})$, as depicted in Fig. B.1.

Which criteria exists to choose one of the solution of the equation? Minimum norm criterion:

$$\begin{cases} \min\limits_{\dot{\boldsymbol{q}}} \|\dot{\boldsymbol{q}}\|^2 \\ \dot{\tilde{\boldsymbol{x}}} = \boldsymbol{J}\dot{\boldsymbol{q}} \end{cases} \tag{B.3}$$

The minimum norm solution is orthogonal to the null of $\boldsymbol{J}$. Suppose to choose $\dot{\boldsymbol{q}}_1 \in I(\boldsymbol{J}^\top)$. Since $N(\boldsymbol{J}) \perp I(\boldsymbol{J}^\top)$, then $\dot{\boldsymbol{q}}_0 \perp \dot{\boldsymbol{q}}_1$. So it follows:

$$\begin{aligned} \|\dot{\boldsymbol{q}}\|^2 &= (\dot{\boldsymbol{q}}_0 + \dot{\boldsymbol{q}}_1)^\top (\dot{\boldsymbol{q}}_0 + \dot{\boldsymbol{q}}_1) \\ &= \dot{\boldsymbol{q}}_0^\top \dot{\boldsymbol{q}}_0 + \cancel{\dot{\boldsymbol{q}}_0^\top \dot{\boldsymbol{q}}_1} + \cancel{\dot{\boldsymbol{q}}_1^\top \dot{\boldsymbol{q}}_0} + \dot{\boldsymbol{q}}_1^\top \dot{\boldsymbol{q}}_1 \\ &= \|\dot{\boldsymbol{q}}_0\|^2 + \|\dot{\boldsymbol{q}}_1\|^2 \end{aligned} \tag{B.4}$$

Since $\dot{\boldsymbol{q}}_0$ does not contribute to the solution of the problem, then the minimum norm for $\dot{\boldsymbol{q}}$ is reached when $\dot{\boldsymbol{q}}_0 = \boldsymbol{0}$ and thus $\dot{\boldsymbol{q}} = \dot{\boldsymbol{q}}_1 \in I(\boldsymbol{J}^\top)$.

For the above reasons, we should look for $\dot{\tilde{\boldsymbol{q}}}^*$ directly in the space orthogonal to the null of $\boldsymbol{J}$. Let us consider

$$\begin{cases} \dot{\boldsymbol{q}} = \boldsymbol{J}^\top \boldsymbol{\alpha} \\ \dot{\tilde{\boldsymbol{x}}} = \boldsymbol{J}\dot{\boldsymbol{q}} \end{cases} \tag{B.5}$$

Substituting we have

$$\begin{aligned} \dot{\tilde{\boldsymbol{x}}} &= \boldsymbol{J}\boldsymbol{J}^\top \boldsymbol{\alpha} \\ \boldsymbol{\alpha} &= (\boldsymbol{J}\boldsymbol{J}^\top)^{-1} \dot{\tilde{\boldsymbol{x}}} \\ \dot{\boldsymbol{q}} &= \boldsymbol{J}^\top (\boldsymbol{J}\boldsymbol{J}^\top)^{-1} \dot{\tilde{\boldsymbol{x}}} \end{aligned} \tag{B.6}$$

In the above equation we identify the right inverse (A.6), therefore

$$\dot{\boldsymbol{q}} = \boldsymbol{J}^{\#} \dot{\tilde{\boldsymbol{x}}} \tag{B.7}$$

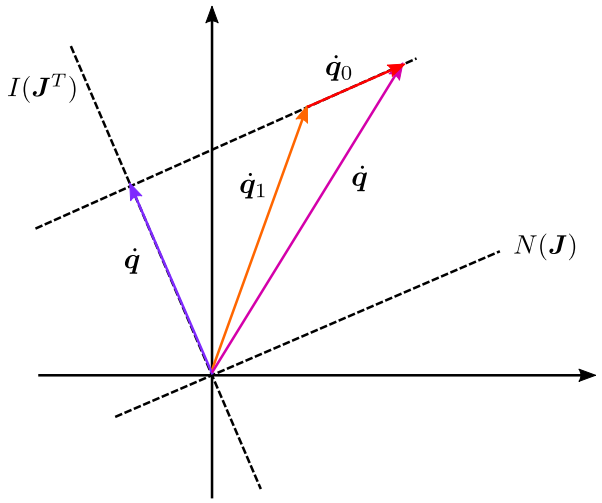


Figure B.1: Representation of two possible solutions $\dot{\boldsymbol{q}}$. The magenta one is decomposed in two parts, $\dot{\boldsymbol{q}}_0 \in N(\boldsymbol{J})$ and another solution $\dot{\boldsymbol{q}}_1$. The purple one instead lies in $I(\boldsymbol{J}^\top)$.

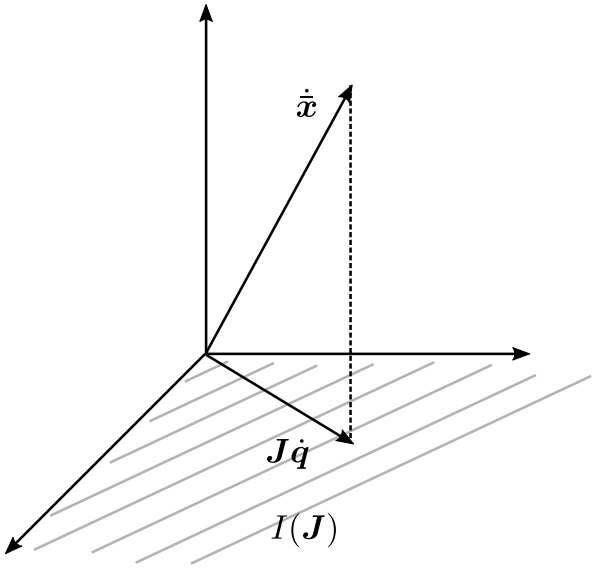


Figure B.2: When the Jacobian matrix is not full row rank, then it only maps a subspace. The best solution is the one such that the error is orthogonal to the $I(\boldsymbol{J})$ ad depicted in the figure.

B.2 Full Column Rank Jacobian Case

Let us now consider the case where $n < 6$. Since $\partial(J) < 6$, from the Jacobian relationship

$$\dot{\boldsymbol{x}} = \boldsymbol{J}\dot{\boldsymbol{q}} \tag{B.8}$$

it follows that $\dot{\boldsymbol{x}}$ is found within a subspace of $\mathbb{R}^6$. In particular, $\dot{\boldsymbol{x}} \in I(\boldsymbol{J}) \subset \mathbb{R}^6$

Then we want to find out the best approximation that gives us

$$\dot{\tilde{\boldsymbol{x}}} \approx \boldsymbol{J}\dot{\tilde{\boldsymbol{q}}} \tag{B.9}$$

From the figure, it is clear that the error must be orthogonal to the image space of $\boldsymbol{J}$, thus

$$(\dot{\tilde{\boldsymbol{x}}} - \boldsymbol{J}\dot{\tilde{\boldsymbol{q}}})^\top \perp I(\boldsymbol{J}) \tag{B.10}$$

thus

$$\begin{aligned}(\dot{\boldsymbol{x}} - \boldsymbol{J}\dot{\boldsymbol{q}})^\top \boldsymbol{J} &= 0 \\ \boldsymbol{J}^\top(\dot{\boldsymbol{x}} - \boldsymbol{J}\dot{\boldsymbol{q}}) &= 0 \\ \boldsymbol{J}^\top \dot{\boldsymbol{x}} &= \boldsymbol{J}^\top \boldsymbol{J}\dot{\boldsymbol{q}} \\ \dot{\boldsymbol{q}} &= (\boldsymbol{J}^\top \boldsymbol{J})^{-1}\boldsymbol{J}^\top \dot{\boldsymbol{x}} \\ \dot{\boldsymbol{q}} &= \boldsymbol{J}^{\#}\dot{\boldsymbol{x}}\end{aligned} \tag{B.11}$$

In the above equation we identify the left inverse (A.5), therefore

$$\dot{\boldsymbol{q}} = \boldsymbol{J}^{\#}\dot{\boldsymbol{x}} \tag{B.12}$$

Appendix C

Task Priority Inverse Kinematics: Detailed Developments

C.1 Solving the First Priority Level

Let us consider a single task. Our goal is to find out a solution that tracks at best the rows corresponding to *active* tasks, while not over-constraining the system whenever the activation value is zero and the task is *inactive*. The straightforward initial idea is to consider the following minimization problem

$$\min_{\dot{\boldsymbol{q}}} \left\| \boldsymbol{A}(\dot{\tilde{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{q}}) \right\|^2, \tag{C.1}$$

whose corresponding manifold of non-regularized solutions is

$$\dot{\boldsymbol{q}} = (\boldsymbol{A}\boldsymbol{J})^{\#}\boldsymbol{A}\dot{\tilde{\boldsymbol{x}}} + (\boldsymbol{I} - (\boldsymbol{A}\boldsymbol{J})^{\#}\boldsymbol{A}\boldsymbol{J})\dot{\boldsymbol{z}},\ \forall \dot{\boldsymbol{z}}. \tag{C.2}$$

Equation (C.2) unfortunately exhibits discontinuities in the $\dot{\boldsymbol{q}}$ when $\boldsymbol{A}$ is varied, since the resulting weighted pseudo-inverse $(\boldsymbol{A}\boldsymbol{J})^{\#}\boldsymbol{A}$ is invariant to the weights on the rows of $\boldsymbol{J}$ that are linearly independent [**?**]. More specifically, the discontinuity occurs whenever a value of $A_{(i,i)}$ changes from 0 to $\epsilon > 0$ and vice versa.

C.1.1 Task Oriented Regularization

Since solely imposing a weight $\boldsymbol{A}$ is insufficient to obtain the desired continuous behavior of activating and deactivating some rows of $\boldsymbol{J}$ without discontinuity, and that both the DLS and SVO regularizations, even if assuring a theoretical continuity, are however affected by practical discontinuities, the additional idea is therefore to modify the original minimization problem (C.1) by introducing a novel regularization, the here called *task oriented regularization*:

$$\min_{\dot{\boldsymbol{q}}} \left[\left\| \boldsymbol{A}(\dot{\tilde{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{q}}) \right\|^2 + \|\boldsymbol{J}\dot{\boldsymbol{q}}\|^2_{\boldsymbol{A}(\boldsymbol{I}-\boldsymbol{A})} \right]. \tag{C.3}$$

The above problem is a special instance of the general regularized problem (3.34), where the regularization matrix $\boldsymbol{R}$ results to be

$$\boldsymbol{R} = \boldsymbol{J}^{\top}\boldsymbol{A}(\boldsymbol{I} - \boldsymbol{A})\boldsymbol{J}. \tag{C.4}$$

The rationale of this choice is that we want to preserve the rows that have their corresponding $A_{(i,i)} = 1$, while at the same time we want to introduce a regularization on those with $A_{(i,i)} < 1$. The choice of using $\boldsymbol{A}(\boldsymbol{I} - \boldsymbol{A})$ guarantees that a regularization is added only to the rows in transition. Indeed, it is easy to see that the cost vanishes for all the rows with $A_{(i,i)} = 0$ or $A_{(i,i)} = 1$. Thus, if $\boldsymbol{A}$ is made of only such values, the regularization cost vanishes and the solution just corresponds to the pseudo inverse of the active rows, as it will be evident in the following discussion.

The definition (C.4), substituted into (3.39), gives the following form for the manifold of regularized solutions

$$\begin{aligned} \dot{\boldsymbol{q}} &= (\boldsymbol{J}^{\top}\boldsymbol{A}\boldsymbol{J})^{\#}\boldsymbol{J}^{\top}\boldsymbol{A}\boldsymbol{A}\dot{\tilde{\boldsymbol{x}}} + (\boldsymbol{I} - (\boldsymbol{J}^{\top}\boldsymbol{A}\boldsymbol{J})^{\#}\boldsymbol{J}^{\top}\boldsymbol{A}\boldsymbol{A}\boldsymbol{J})\dot{\boldsymbol{z}} \\ &\triangleq \boldsymbol{\rho} + \boldsymbol{Q}\dot{\boldsymbol{z}},\ \forall \dot{\boldsymbol{z}}. \end{aligned} \tag{C.5}$$

The main results are (proof in section C.3):

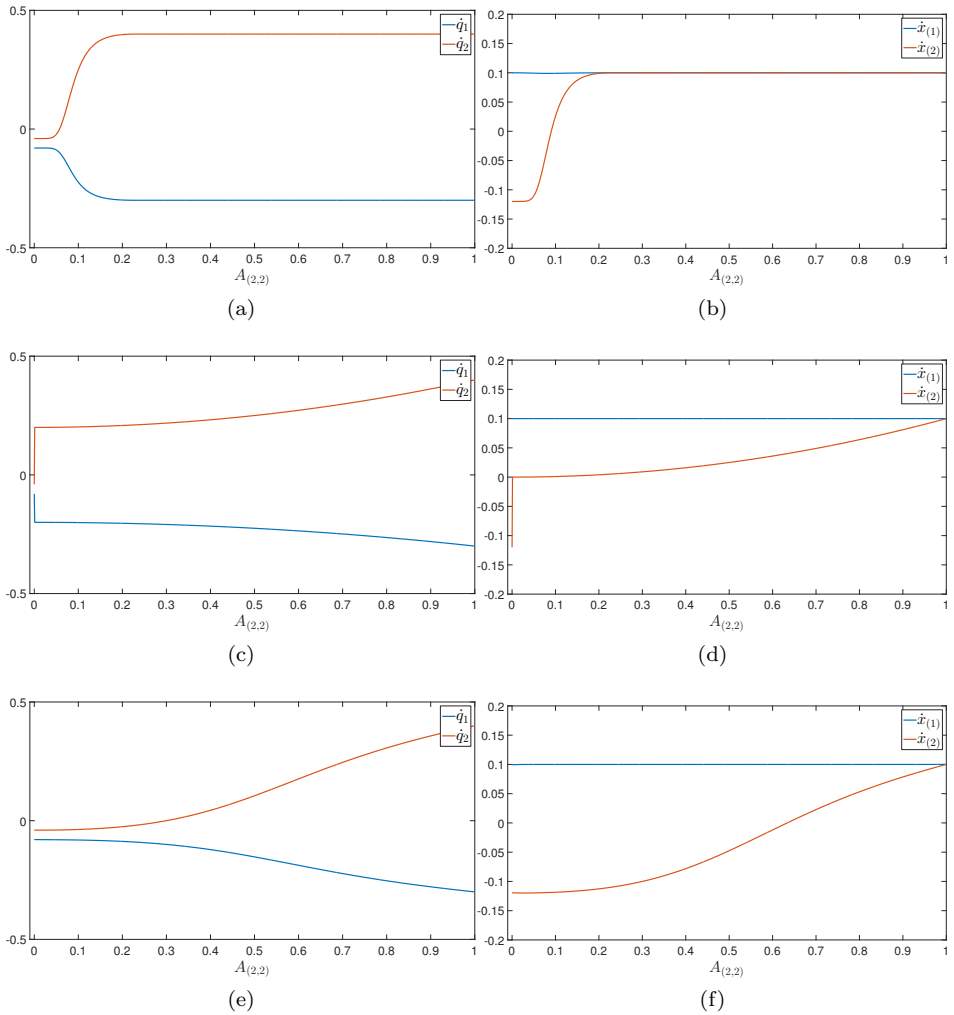


Figure C.1: Comparison between the different regularization techniques, as the second row of task (C.9) is deactivated. (a) and (b) show what happens to control vector $\dot{\boldsymbol{q}}$ and (b) task velocities $\dot{\boldsymbol{x}}$ in the case of SVO regularization. As it is evident, the transition does not occur over the whole variation of $A_{(2,2)}$. (c) and (d) show what happens when only the task oriented regularization is used: a discontinuity occurs near $A_{(2,2)} = 0$. Finally, (e) and (f) show the final result with the proposed approach

- if a row i is linearly independent from all the others, and its $A_{(i,i)} = 1$, then $(\boldsymbol{JQ})_{\{i\}} = \boldsymbol{0}$;
- for a set of rows of $\boldsymbol{J}$ that are linearly dependent between each other, but linearly independent from the others, the orthogonality property holds if and only if all the corresponding $A_{(i,i)}$ are equal to one.

However, the task oriented regularization alone is insufficient to remove all the actual and practical discontinuities. It needs to be coupled with the SVO regularization, leading to the following problem:

$$\min_{\dot{\boldsymbol{q}}} \left[\left\| \boldsymbol{A}(\dot{\tilde{\boldsymbol{x}}} - \boldsymbol{J}\dot{\boldsymbol{q}}) \right\|^2 + \|\boldsymbol{J}\dot{\boldsymbol{q}}\|^2_{\boldsymbol{A}(\boldsymbol{I}-\boldsymbol{A})} + \left\| \boldsymbol{V}^\top \dot{\boldsymbol{q}} \right\|^2_{\boldsymbol{P}} \right], \tag{C.6}$$

where $\boldsymbol{V}^\top$ is the right orthonormal matrix of the SVD decomposition of $\boldsymbol{J}^\top \boldsymbol{A}\boldsymbol{J} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$, and $\boldsymbol{P}$ is the SVO regularization matrix described in Section 3.8.1.2. The manifold of solutions of the above problem can be thus written as

$$\begin{aligned} \dot{\boldsymbol{q}} &= (\boldsymbol{J}^\top \boldsymbol{A}\boldsymbol{J} + \boldsymbol{V}^\top \boldsymbol{P}\boldsymbol{V})^{\#} \boldsymbol{J}^\top \boldsymbol{A}\boldsymbol{A}\dot{\tilde{\boldsymbol{x}}} + (\boldsymbol{I} - (\boldsymbol{J}^\top \boldsymbol{A}\boldsymbol{J} + \boldsymbol{V}^\top \boldsymbol{P}\boldsymbol{V})^{\#} \boldsymbol{J}^\top \boldsymbol{A}\boldsymbol{A}\boldsymbol{J})\dot{\boldsymbol{z}} \\ &\triangleq \boldsymbol{\rho} + \boldsymbol{Q}\dot{\boldsymbol{z}}, \ \forall \dot{\boldsymbol{z}}, \end{aligned} \tag{C.7}$$

and coincides with (C.5) whenever $\boldsymbol{P} = \boldsymbol{0}$.

Finally, the residual arbitrariness $\dot{\boldsymbol{z}}$ should be exploited to ensure the continuity by minimizing the following cost

$$\min_{\dot{\boldsymbol{z}}} \left[\|\boldsymbol{\rho} + \boldsymbol{Q}\dot{\boldsymbol{z}}\|^2 + \eta \, \|(\boldsymbol{I} - \boldsymbol{Q})\dot{\boldsymbol{z}}\|^2 \right]. \tag{C.8}$$

An example of the resulting behavior is reported in Fig. C.1 for the case of

$$\boldsymbol{J} = \begin{bmatrix} -1 & -0.5 \\ 1 & 1 \end{bmatrix}, \tag{C.9}$$

with a velocity reference $\dot{\tilde{\boldsymbol{x}}} = \begin{bmatrix} 0.1 & 0.1 \end{bmatrix}^\top$ and by varying the value of $A_{(2,2)}$, i.e. activating/deactivating the second task, represented by the second row of $\boldsymbol{J}$. Details on the minimization problems (C.6) and (C.8) are not given here. The interested reader can find all the details in the paper [**?**].

C.2 Extension to Multiple Priority Levels

In the previous section, the methodology for activating and deactivating rows of a single task has been presented, with the introduction of the so called task oriented regularization, the use of the SVO one and a final minimization on the control vector to ensure the continuity of the solution. In this section, we shall first tackle the possible discontinuity that can arise when a second level of priority is introduced, and then generalize the proposed framework to any number of priority levels.

C.2.1 Removing the Discontinuities in the Prioritized Control

Let us now consider another task that has to be executed with lower priority, represented by the following Jacobian relationship

$$\dot{\boldsymbol{x}}_2 = \boldsymbol{J}_2 \dot{\boldsymbol{q}}, \tag{C.10}$$

with $\boldsymbol{J}_2 \in \mathbb{R}^{m_2 \times n}$, $\dot{\boldsymbol{q}} \in \mathbb{R}^n$ and $\dot{\boldsymbol{x}}_2 \in \mathbb{R}^{m_2}$. The minimization for this second task must be performed taking into account that $\dot{\boldsymbol{q}}$ has been partially fixed by the higher priority task. The manifold of solutions of the first task is given by (C.7) and is $\dot{\boldsymbol{q}} = \boldsymbol{\rho}_1 + \boldsymbol{Q}_1 \dot{\boldsymbol{z}}_1$. Let us remark how, for the time being, we are not considering the minimization on the control vector, because that completely consumes any residual arbitrariness. We shall later see in Section C.2.2 how that minimization is used in a hierarchy of tasks.

With that in mind, the second minimization problem can only exploit the arbitrariness of $\dot{\boldsymbol{z}}_1$, leading to

$$\min_{\dot{\boldsymbol{z}}_1} \left[\left\| \boldsymbol{A}_2(\dot{\tilde{\boldsymbol{x}}}_2 - \boldsymbol{J}_2\boldsymbol{Q}_1\dot{\boldsymbol{z}}_1) \right\|^2 + \|\boldsymbol{J}_2\boldsymbol{Q}_1\dot{\boldsymbol{z}}_1\|^2_{\boldsymbol{A}_2(\boldsymbol{I}-\boldsymbol{A}_2)} + \left\| \boldsymbol{V}_2^\top \dot{\boldsymbol{z}}_1 \right\|^2_{\boldsymbol{P}_2} \right], \tag{C.11}$$

with the definition $\dot{\tilde{\boldsymbol{x}}}_2 \triangleq \dot{\tilde{\boldsymbol{x}}}_2 - \boldsymbol{J}_2\boldsymbol{\rho}_1$, and where the same task oriented and SVO regularizations have been employed to deal with the activation matrix $\boldsymbol{A}_2$.

Whenever a second level of priority is considered, a new source of possible discontinuities is represented by the non-orthogonal projection matrix $\boldsymbol{Q}_1$. In this case we have the dual problem that was present in (C.1), i.e. the fact that the minimization problem can be invariant to the weights on the columns (which is how $\boldsymbol{Q}_1$ is acting). Thus, even if $\boldsymbol{Q}_1$ varies smoothly, the solution will not, except abruptly near zero.

To solve this additional problem, the idea is to compute a new task reference in lieu of $\dot{\tilde{\boldsymbol{x}}}_2$. In particular, we want to find which is the best velocity obtainable minimizing the use of control directions in transition. Toward that end, we exploit the following auxiliary problem

$$\min_{\dot{\boldsymbol{u}}_1} \left[\left\| \boldsymbol{A}_2(\dot{\tilde{\boldsymbol{x}}}_2 - \boldsymbol{J}_2\boldsymbol{Q}_1\dot{\boldsymbol{u}}_1) \right\|^2 + \|\boldsymbol{J}_2\boldsymbol{Q}_1\dot{\boldsymbol{u}}_1\|^2_{\boldsymbol{A}_2(\boldsymbol{I}-\boldsymbol{A}_2)} + \eta \, \|(\boldsymbol{I} - \boldsymbol{Q}_1)\dot{\boldsymbol{u}}_1\|^2 + \left\| \hat{\boldsymbol{V}}_2^\top \dot{\boldsymbol{u}}_1 \right\|^2_{\hat{\boldsymbol{P}}_2} \right], \tag{C.12}$$

where this time $\hat{\boldsymbol{V}}_2^\top$ is the right orthonormal matrix of the SVD decomposition of

$$\boldsymbol{Q}_1^\top \boldsymbol{J}_2^\top \boldsymbol{A}_2 \boldsymbol{J}_2 \boldsymbol{Q}_1 + \eta (\boldsymbol{I} - \boldsymbol{Q}_1)^\top (\boldsymbol{I} - \boldsymbol{Q}_1).$$

The corresponding task velocity is

$$\dot{\boldsymbol{x}}_2^* = \boldsymbol{J}_2\boldsymbol{Q}_1\dot{\boldsymbol{u}}_1 \triangleq \boldsymbol{W}_2\dot{\tilde{\boldsymbol{x}}}_2, \tag{C.13}$$

which is then used as a reference velocity in (C.11). In practice, $\boldsymbol{Q}_1(\boldsymbol{J}_2\boldsymbol{Q}_1)^{\#}\boldsymbol{W}_2\dot{\tilde{\boldsymbol{x}}}_2$ operates in this way: first it finds the best $\dot{\boldsymbol{x}}_2^* = \boldsymbol{W}_2\dot{\tilde{\boldsymbol{x}}}_2$ that can be obtained minimizing the use of control directions in transition, and then it exploits the standard weighted pseudo-inverse to obtain the corresponding weighted minimum norm solution (i.e. $\boldsymbol{Q}_1(\boldsymbol{J}_2\boldsymbol{Q}_1)^{\#}\dot{\boldsymbol{x}}_2^*$).

C.2.2 Minimization of the Control Vector as the Final Task

In the previous section we have seen how to deal with a secondary task, exploiting (C.12) to cope with fact that the standard weighted pseudo inverse $\boldsymbol{Q}_1(\boldsymbol{J}_2\boldsymbol{Q}_1)^{\#}$ is insufficient.

Let us now go back to the minimization of the control vector (C.8). It is clear that such a minimization can actually be seen as another task to be executed. This can be simply done considering $\boldsymbol{J}_2 = \boldsymbol{I}$, $\boldsymbol{A}_2 = \boldsymbol{I}$ and $\dot{\tilde{\boldsymbol{x}}}_2 = 0$. Then, we can see how (C.8) is just a special instance of (C.12).

Then, this task should be placed as the very last task of the hierarchy, consuming all the residual arbitrariness for minimizing the control vector, eliminating any practical discontinuities during task activations.

C.2.3 Unifying Formula for the Pseudo Inverse

Before proceeding to the extension to any number of priority levels, let us first introduce a more compact notation by introducing the operator $(\boldsymbol{X})^{\#,\boldsymbol{A},\boldsymbol{Q}}$ as in the following:

$$\boldsymbol{X}^{\#,\boldsymbol{A},\boldsymbol{Q}} \triangleq \left(\boldsymbol{X}^\top\boldsymbol{A}\boldsymbol{X} + \eta(\boldsymbol{I}-\boldsymbol{Q})^\top(\boldsymbol{I}-\boldsymbol{Q}) + \boldsymbol{V}^\top\boldsymbol{P}\boldsymbol{V}\right)^{\#}\boldsymbol{X}^\top\boldsymbol{A}\boldsymbol{A} \tag{C.14}$$

where $\boldsymbol{V}$ is the right orthonormal matrix of the SVD decomposition of $\boldsymbol{X}^\top\boldsymbol{A}\boldsymbol{X} + \eta(\boldsymbol{I}-\boldsymbol{Q})^\top(\boldsymbol{I}-\boldsymbol{Q})$.

The task oriented regularization (C.6) can be simply obtained by writing $\boldsymbol{J}^{\#,\boldsymbol{A},\boldsymbol{I}}$ and the auxiliary problem (C.12) can be obtained by writing $(\boldsymbol{J}\boldsymbol{Q})^{\#,\boldsymbol{A},\boldsymbol{Q}}$.

C.2.4 Extension to Any Number of Priority Levels

Putting all the pieces together, and with the definition of the pseudo inverse given in the previous section, the extension to any number of priority levels is straightforward. With the initializations

$$\boldsymbol{\rho}_0 = \boldsymbol{0}, \quad \boldsymbol{Q}_0 = \boldsymbol{I}, \tag{C.15}$$

then for $k = 1, \ldots, p$, where p is the total number of priority levels:

$$\begin{aligned}
\boldsymbol{W}_k &= \boldsymbol{J}_k\boldsymbol{Q}_{k-1}(\boldsymbol{J}_k\boldsymbol{Q}_{k-1})^{\#,\boldsymbol{A}_k,\boldsymbol{Q}_{k-1}} \\
\boldsymbol{Q}_k &= \boldsymbol{Q}_{k-1}(\boldsymbol{I} - (\boldsymbol{J}_k\boldsymbol{Q}_{k-1})^{\#,\boldsymbol{A}_k,\boldsymbol{I}}\boldsymbol{J}_k\boldsymbol{Q}_{k-1}) \\
\boldsymbol{\rho}_k &= \boldsymbol{\rho}_{k-1} + \boldsymbol{Q}_{k-1}(\boldsymbol{J}_k\boldsymbol{Q}_{k-1})^{\#,\boldsymbol{A}_k,\boldsymbol{I}}\boldsymbol{W}_k\left(\dot{\tilde{\boldsymbol{x}}}_k - \boldsymbol{J}_k\boldsymbol{\rho}_{k-1}\right)
\end{aligned} \tag{C.16}$$

thus ending up with the final control law

$$\dot{\boldsymbol{q}} = \boldsymbol{\rho}_p \tag{C.17}$$

because, according to Section C.2.2, the p-th and final task should be the minimization of the control vector $\dot{\boldsymbol{q}}$, which consumes any residual arbitrariness.

C.3 Residual Orthogonality Properties of the Projector Matrix

In the classical task priority framework the projection operator is orthogonal, and guarantees that lower priority task can only act inside the null space of the higher priority ones. This ensures the invariance of the main task w.r.t. lower priority ones. However, in the here proposed case, the projection matrix is not orthogonal whenever any of the activation values are different from 0 and 1, i.e. whenever some task is in the transition zone. As we shall see, this is actually a positive fact, since the d.o.f. that are being released from the higher priority tasks can be exploited, at least partially, by the lower priority ones. For the time being, let us prove the fact that, under some assumptions, the projection matrix admits some residual orthogonality property: it is orthogonal with respect to the active rows of $\boldsymbol{J}$, i.e. $(\boldsymbol{J}\boldsymbol{Q})_{\{i\}} = \boldsymbol{0}$ for every i-th row for which $A_{(i,i)} = 1$.

To simplify the subsequent analysis, it is convenient to note that

$$\begin{aligned}&(\boldsymbol{J}^\top \boldsymbol{A}\boldsymbol{J})^{\#}\boldsymbol{J}^\top \boldsymbol{A}\\ =&(\boldsymbol{J}^\top \sqrt{\boldsymbol{A}}\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}\boldsymbol{J}^\top \sqrt{\boldsymbol{A}}\sqrt{\boldsymbol{A}}\\ =&(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}\sqrt{\boldsymbol{A}},\end{aligned} \tag{C.18}$$

where the identity (A.7) has been used. Substituting this relationship in the manifold (C.5) yields

$$\begin{aligned}\dot{\boldsymbol{q}} &= (\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}\sqrt{\boldsymbol{A}}\boldsymbol{A}\dot{\tilde{\boldsymbol{x}}} + (\boldsymbol{I} - (\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}\sqrt{\boldsymbol{A}}\boldsymbol{A}\boldsymbol{J})\dot{\boldsymbol{z}}\\ &\triangleq \boldsymbol{\rho} + \boldsymbol{Q}\dot{\boldsymbol{z}},\ \forall \dot{\boldsymbol{z}}.\end{aligned} \tag{C.19}$$

Let us start the discussion by considering $A_{(i,i)} > 0, \forall i$ and $\boldsymbol{J}$ full rank, which implies that $\sqrt{\boldsymbol{A}}\boldsymbol{J}$ is full row rank. Let us compute $(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}$:

$$\begin{aligned}(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#} &= \boldsymbol{J}^\top \sqrt{\boldsymbol{A}}(\sqrt{\boldsymbol{A}}\boldsymbol{J}\boldsymbol{J}^\top \sqrt{\boldsymbol{A}})^{-1}\\ &= \boldsymbol{J}^\top \sqrt{\boldsymbol{A}}\sqrt{\boldsymbol{A}}^{-1}(\boldsymbol{J}\boldsymbol{J}^\top)^{-1}\sqrt{\boldsymbol{A}}^{-1}\\ &= \boldsymbol{J}^\top (\boldsymbol{J}\boldsymbol{J}^\top)^{-1}\sqrt{\boldsymbol{A}}^{-1}\\ &= \boldsymbol{J}^{\#}\sqrt{\boldsymbol{A}}^{-1}\end{aligned} \tag{C.20}$$

which substituted into (C.19) gives

$$\begin{aligned}\dot{\boldsymbol{q}} &= \boldsymbol{J}^{\#}\boldsymbol{A}\dot{\tilde{\boldsymbol{x}}} + (\boldsymbol{I} - \boldsymbol{J}^{\#}\boldsymbol{A}\boldsymbol{J})\dot{\boldsymbol{z}}\\ &\triangleq \boldsymbol{\rho} + \boldsymbol{Q}\dot{\boldsymbol{z}},\ \forall \dot{\boldsymbol{z}}.\end{aligned} \tag{C.21}$$

It is easy to see that the projection matrix $(\boldsymbol{I} - \boldsymbol{J}^{\#}\boldsymbol{A}\boldsymbol{J})$ is actually orthogonal to the active rows. Indeed, multiplying the projector by $\boldsymbol{J}$ yields

$$\boldsymbol{J}(\boldsymbol{I} - \boldsymbol{J}^{\#}\boldsymbol{A}\boldsymbol{J}) = (\boldsymbol{J} - \boldsymbol{J}\boldsymbol{J}^{\#}\boldsymbol{A}\boldsymbol{J}) = (\boldsymbol{I} - \boldsymbol{A})\boldsymbol{J} \tag{C.22}$$

since $\boldsymbol{J}\boldsymbol{J}^{\#} = \boldsymbol{I}$ under the above assumptions. The above result implies that, for every row where $A_{(i,i)} = 1$, then $(\boldsymbol{J}\boldsymbol{Q})_{\{i\}} = \boldsymbol{0}$ as it was claimed. However, the projection matrix $\boldsymbol{Q}$ does not prevent $\dot{\boldsymbol{z}}$ from influencing the other rows. As said before, this is a positive fact, as those rows are being deactivated and thus there is not anymore any need to guarantee the fulfillment of their corresponding velocity reference. Indeed, when $A_{(i,i)}$ reaches zero, the corresponding velocity should be unconstrained.

Let us now drop the two assumptions of full rankness of $\boldsymbol{A}$ and $\boldsymbol{J}$. If $\boldsymbol{A}$ contains any value equal to zero in its diagonal, then the above formula cannot be used, since it contains $(\sqrt{\boldsymbol{A}})^{-1}$ which cannot be computed. Instead, without losing generality, let us suppose that the rows with $A_{(i,i)} = 0$ are at the bottom, and let us partition $\boldsymbol{A}$ and $\boldsymbol{J}$ in the following way

$$\boldsymbol{A} = \begin{bmatrix}\bar{\boldsymbol{A}} & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{0}\end{bmatrix}\ ;\ \boldsymbol{J} = \begin{bmatrix}\bar{\boldsymbol{J}}\\ \hat{\boldsymbol{J}}\end{bmatrix}, \tag{C.23}$$

where $\bar{\boldsymbol{J}}$ only contains the i-th rows for which $A_{(i,i)} \neq 0$.

Let us still compute the value of $(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}$. Using (A.8) let us write it as

$$(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#} = \boldsymbol{J}^\top \sqrt{\boldsymbol{A}}(\sqrt{\boldsymbol{A}}\boldsymbol{J}\boldsymbol{J}^\top \sqrt{\boldsymbol{A}})^{\#}. \tag{C.24}$$

Then using the definition (C.23) it is simple to compute

$$\sqrt{\boldsymbol{A}}\boldsymbol{J} = \begin{bmatrix}\sqrt{\bar{\boldsymbol{A}}}\bar{\boldsymbol{J}}\\ \boldsymbol{0}\end{bmatrix} \tag{C.25}$$

and

$$\sqrt{\boldsymbol{A}}\boldsymbol{J}\boldsymbol{J}^\top \sqrt{\boldsymbol{A}} = \begin{bmatrix}\sqrt{\bar{\boldsymbol{A}}}\bar{\boldsymbol{J}}\bar{\boldsymbol{J}}^\top \sqrt{\bar{\boldsymbol{A}}} & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{0}\end{bmatrix} \tag{C.26}$$

After some simple algebra, the formula for the pseudo-inverse $(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#}$ becomes:

$$\begin{aligned}(\sqrt{\boldsymbol{A}}\boldsymbol{J})^{\#} &= \boldsymbol{J}^{\top}\sqrt{\boldsymbol{A}}(\sqrt{\boldsymbol{A}}\boldsymbol{J}\boldsymbol{J}^{\top}\sqrt{\boldsymbol{A}})^{\#} \\ &= \begin{bmatrix}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0}\end{bmatrix}\sqrt{\boldsymbol{A}}^{\#}\end{aligned} \tag{C.27}$$

which substituted into the control law (C.19) yields

$$\begin{aligned}\dot{\boldsymbol{q}} &= \begin{bmatrix}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0}\end{bmatrix}\boldsymbol{A}\dot{\boldsymbol{x}} + (\boldsymbol{I} - \begin{bmatrix}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0}\end{bmatrix}\boldsymbol{A}\boldsymbol{J})\dot{\boldsymbol{z}} \\ &\triangleq \boldsymbol{\rho} + \boldsymbol{Q}\dot{\boldsymbol{z}}, \forall \dot{\boldsymbol{z}}.\end{aligned} \tag{C.28}$$

This could not be otherwise, as the added cost in (C.3) vanishes whenever $\boldsymbol{A}$ is composed only by ones and zeros, and thus the obtained solution is just the pseudo inverse of $\boldsymbol{J}$ with only the relevant rows.

The projection matrix $(\boldsymbol{I} - \begin{bmatrix}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0}\end{bmatrix}\boldsymbol{A}\boldsymbol{J})$ is still orthogonal to the active rows. Indeed,

$$\begin{aligned}\boldsymbol{J}\boldsymbol{Q} &= (\boldsymbol{I} - \begin{bmatrix}\bar{\boldsymbol{J}} \\ \hat{\boldsymbol{J}}\end{bmatrix}\begin{bmatrix}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0}\end{bmatrix}\boldsymbol{A})\boldsymbol{J} \\ &= (\boldsymbol{I} - \begin{bmatrix}\bar{\boldsymbol{J}}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0} \\ \hat{\boldsymbol{J}}\bar{\boldsymbol{J}}^{\#} & \boldsymbol{0}\end{bmatrix}\boldsymbol{A})\boldsymbol{J} \\ &= \begin{bmatrix}(\boldsymbol{I} - \bar{\boldsymbol{J}}\bar{\boldsymbol{J}}^{\#}\bar{\boldsymbol{A}})\bar{\boldsymbol{J}} \\ (\boldsymbol{I} - \hat{\boldsymbol{J}}\bar{\boldsymbol{J}}^{\#}\bar{\boldsymbol{A}})\bar{\boldsymbol{J}}\end{bmatrix}.\end{aligned} \tag{C.29}$$

We have the following results:

- if a row i is linearly independent from all the others, and its $A_{(i,i)} = 1$, then $(\boldsymbol{J}\boldsymbol{Q})_{\{i\}} = \boldsymbol{0}$ as it was claimed;
- for a set of rows of $\boldsymbol{J}$ that are linearly dependent between each other, but linearly independent from the others, the orthogonality property holds if and only if all the corresponding $A_{(i,i)}$ are equal to one.

The clear advantage of the proposed regularization method is that the activation value is directly linked with the regularization itself, unlike in the SVO case. This allows to exploit the full range of variation of the activation value to perform the transition, clearly mitigating the possible practical discontinuities. Furthermore, the above analysis shows that the regularization is oriented along the task in transitions, preserving those that are active (whenever they are linearly independent from those in transition).